Ballistic Trajectory Calculator

5.3

Generated by Doxygen 1.15.0

1 Ballistic Trajectory Calculator 1
1.1 Overview . . . 1
1.2 Features . . . 1
1.3 Architecture . . . 1
1.4 Usage . . . 2
1.4.0.1 Basic Operation: . . . 2
1.4.0.2 Data Format Examples: . . . 2
1.5 Technical Details . . . 2
1.6 References . . . 2
1.6.0.1 Primary References . . . 2
1.6.0.2 Technical Standards . . . 3
1.6.0.3 Online Resources . . . 3
1.6.0.4 Implementation Notes . . . 3

2 Todo List 5

3 Topic Index 7
3.1 Topics . . . 7

4 Directory Hierarchy 9
4.1 Directories . . . 9

5 Data Structure Index 11
5.1 Data Structures . . . 11

6 File Index 13
6.1 File List . . . 13

7 Topic Documentation 15
7.1 Ballistic Calculations . . . 15
7.1.1 Detailed Description . . . 15
7.1.2 Function Documentation . . . 16
7.1.2.1 compute_fire_solution() . . . 16
7.1.2.2 SuggestBestBaseAndAmmo() . . . 17
7.1.2.3 SuggestBestBaseAndAmmo_RegionAware() . . . 19
7.2 Command Processing . . . 20
7.2.1 Detailed Description . . . 22
7.2.2 Macro Definition Documentation . . . 22
7.2.2.1 CMD_C_ALL . . . 22
7.2.3 Enumeration Type Documentation . . . 22
7.2.3.1 Command_case . . . 22
7.2.3.2 Command_t . . . 23
7.2.4 Function Documentation . . . 24
7.2.4.1 add_seperator() . . . 24

7.2.4.2 clear_terminal() . . . 24
7.2.4.3 cmd_ammo() . . . 25
7.2.4.4 cmd_base() . . . 26
7.2.4.5 cmd_calculate() . . . 27
7.2.4.6 cmd_clr_ter() . . . 28
7.2.4.7 cmd_colour() . . . 29
7.2.4.8 cmd_config() . . . 29
7.2.4.9 cmd_exit() . . . 30
7.2.4.10 cmd_help() . . . 30
7.2.4.11 cmd_mono() . . . 31
7.2.4.12 cmd_target() . . . 32
7.2.4.13 cmd_test() . . . 33
7.2.4.14 GetCommand() . . . 33
7.2.4.15 NormalizeInput() . . . 35
7.2.4.16 ParseCommandFromBuffer() . . . 36
7.2.5 Variable Documentation . . . 37
7.2.5.1 command_count . . . 37
7.2.5.2 commands . . . 37
7.3 Utility Functions . . . 38
7.3.1 Detailed Description . . . 38
7.3.2 Enumeration Type Documentation . . . 38
7.3.2.1 HRESULT . . . 38
7.3.2.2 PROCESS . . . 39
7.3.3 Function Documentation . . . 39
7.3.3.1 FAILED() . . . 39
7.3.3.2 SUCCEEDED() . . . 40
7.4 File Loading and Configuration . . . 40
7.4.1 Detailed Description . . . 41
7.4.2 Typedef Documentation . . . 42
7.4.2.1 RecordHandler . . . 42
7.4.3 Enumeration Type Documentation . . . 42
7.4.3.1 ParseState . . . 42
7.4.4 Function Documentation . . . 42
7.4.4.1 format_basic_error_string() . . . 42
7.4.4.2 free_folder_path() . . . 43
7.4.4.3 generate_config() . . . 43
7.4.4.4 handle_ammo_t() . . . 44
7.4.4.5 handle_base() . . . 45
7.4.4.6 handle_config() . . . 47
7.4.4.7 handle_target() . . . 48
7.4.4.8 HandleParameterSelection() . . . 49
7.4.4.9 LogResult() . . . 50

7.4.4.10 peek() . . . 50
7.4.4.11 read_config() . . . 51
7.4.4.12 recreate_config() . . . 52
7.4.4.13 scan() . . . 54
7.4.4.14 set_folder_path() . . . 55
7.4.4.15 test_and_run_config() . . . 55
7.4.5 Variable Documentation . . . 56
7.4.5.1 folder_path . . . 56
7.5 Main Program . . . 57
7.5.1 Detailed Description . . . 57
7.5.2 Function Documentation . . . 57
7.5.2.1 Setup() . . . 57
7.6 Data Storage Structures . . . 58
7.6.1 Detailed Description . . . 63
7.6.1.1 Data Structure Hierarchy . . . 64
7.6.1.2 Data File Formats . . . 64
7.6.1.3 Memory Management . . . 64
7.6.2 Typedef Documentation . . . 65
7.6.2.1 Ammo_Stock . . . 65
7.6.2.2 Ammo_t . . . 65
7.6.2.3 Ammo_t_Node . . . 65
7.6.2.4 Base . . . 65
7.6.2.5 Base_Node . . . 65
7.6.2.6 Target . . . 66
7.6.2.7 Target_Node . . . 66
7.6.3 Enumeration Type Documentation . . . 66
7.6.3.1 Default_flags . . . 66
7.6.3.2 InputFlags . . . 66
7.6.3.3 RangeType . . . 67
7.6.4 Function Documentation . . . 67
7.6.4.1 add_Ammo_Stock() . . . 67
7.6.4.2 add_Ammo_t() . . . 68
7.6.4.3 add_Base() . . . 69
7.6.4.4 add_Target() . . . 70
7.6.4.5 delete_Ammo_Stock() . . . 70
7.6.4.6 delete_Ammo_t() . . . 71
7.6.4.7 delete_Base() . . . 72
7.6.4.8 delete_Target() . . . 73
7.6.4.9 FindAmmoByID() . . . 74
7.6.4.10 FindBaseByID() . . . 75
7.6.4.11 FindTargetByID() . . . 76
7.6.4.12 free_Ammo_Stock_List() . . . 77

7.6.4.13 free_Ammo_t_List() . . . 77
7.6.4.14 free_Base_List() . . . 78
7.6.4.15 free_Target_List() . . . 79
7.6.4.16 GetAmmoID() . . . 80
7.6.4.17 GetNode_Ammot_t_List() . . . 81
7.6.4.18 init_Ammo_Stock_List() . . . 81
7.6.4.19 init_Ammo_t_List() . . . 82
7.6.4.20 init_Base_List() . . . 83
7.6.4.21 init_Target_List() . . . 84
7.6.4.22 popBack_Ammo_Stock() . . . 85
7.6.4.23 popBack_Ammo_t() . . . 85
7.6.4.24 popBack_Base() . . . 85
7.6.4.25 popBack_Target() . . . 86
7.6.4.26 popFront_Ammo_Stock() . . . 86
7.6.4.27 popFront_Ammo_t() . . . 87
7.6.4.28 popFront_Base() . . . 87
7.6.4.29 popFront_Target() . . . 88
7.6.4.30 print_Ammo_Stock_List() . . . 89
7.6.4.31 print_Ammo_t_List() . . . 89
7.6.4.32 print_Base_List() . . . 90
7.6.4.33 print_Target_List() . . . 91
7.6.4.34 Save_Ammo_t_List() . . . 91
7.6.4.35 Save_Ammo_t_List_file() . . . 92
7.6.4.36 Save_BaseList() . . . 93
7.6.4.37 Save_BaseList_file() . . . 94
7.6.4.38 Save_TargetList() . . . 94
7.6.4.39 Save_TargetList_file() . . . 95
7.6.5 Variable Documentation . . . 96
7.6.5.1 altitude . . . 96
7.6.5.2 ammo_t_list . . . 96
7.6.5.3 ammoItem . . . 96
7.6.5.4 ammoStock . . . 96
7.6.5.5 area . . . 96
7.6.5.6 ballistic_coefficient . . . 96
7.6.5.7 base_list . . . 97
7.6.5.8 baseItem . . . 97
7.6.5.9 blastRadius . . . 97
7.6.5.10 coord [1/2] . . . 97
7.6.5.11 coord [2/2] . . . 97
7.6.5.12 diameter . . . 97
7.6.5.13 head [1/4] . . . 98
7.6.5.14 head [2/4] . . . 98

7.6.5.15 head [3/4] . . . 98
7.6.5.16 head [4/4] . . . 98
7.6.5.17 id [1/3] . . . 98
7.6.5.18 id [2/3] . . . 98
7.6.5.19 id [3/3] . . . 99
7.6.5.20 label [1/2] . . . 99
7.6.5.21 label [2/2] . . . 99
7.6.5.22 latitude . . . 99
7.6.5.23 longitude . . . 99
7.6.5.24 muzzleVelocity . . . 99
7.6.5.25 name . . . 100
7.6.5.26 next [1/4] . . . 100
7.6.5.27 next [2/4] . . . 100
7.6.5.28 next [3/4] . . . 100
7.6.5.29 next [4/4] . . . 100
7.6.5.30 prev . . . 100
7.6.5.31 quantity . . . 101
7.6.5.32 tail [1/4] . . . 101
7.6.5.33 tail [2/4] . . . 101
7.6.5.34 tail [3/4] . . . 101
7.6.5.35 tail [4/4] . . . 101
7.6.5.36 target_list . . . 101
7.6.5.37 targetItem . . . 102
7.6.5.38 typePtr . . . 102
7.6.5.39 weight . . . 102

8 Directory Documentation **103**

8.1 C:/vscode/ballistic Directory Reference . . . 103
8.2 C:/vscode/ballistic/doxygen Directory Reference . . . 104
8.3 C:/vscode Directory Reference . . . 104

9 Data Structure Documentation **105**

9.1 Ammo_Stock Struct Reference . . . 105
9.1.1 Detailed Description . . . 106
9.2 Ammo_Stock_List Struct Reference . . . 106
9.2.1 Detailed Description . . . 107
9.3 Ammo_t Struct Reference . . . 107
9.3.1 Detailed Description . . . 107
9.4 Ammo_t_List Struct Reference . . . 108
9.4.1 Detailed Description . . . 108
9.5 Ammo_t_Node Struct Reference . . . 109
9.5.1 Detailed Description . . . 109
9.6 BallisticResult Struct Reference . . . 109

9.6.1 Detailed Description . . . 110
9.6.2 Field Documentation . . . 110
9.6.2.1 azimuth_deg . . . 110
9.6.2.2 elevation_deg . . . 110
9.6.2.3 impact_velocity . . . 110
9.6.2.4 time_of_flight . . . 111
9.6.2.5 valid . . . 111
9.7 Base Struct Reference . . . 111
9.7.1 Detailed Description . . . 112
9.8 Base_List Struct Reference . . . 112
9.8.1 Detailed Description . . . 113
9.9 Base_Node Struct Reference . . . 114
9.9.1 Detailed Description . . . 115
9.10 Command Struct Reference . . . 115
9.10.1 Detailed Description . . . 115
9.10.2 Field Documentation . . . 115
9.10.2.1 case_mask . . . 115
9.10.2.2 description . . . 116
9.10.2.3 handler . . . 116
9.10.2.4 name . . . 116
9.10.2.5 type . . . 116
9.11 Coordinate Struct Reference . . . 116
9.11.1 Detailed Description . . . 117
9.12 DebugmallocData Struct Reference . . . 117
9.12.1 Detailed Description . . . 117
9.12.2 Field Documentation . . . 118
9.12.2.1 all_alloc_bytes . . . 118
9.12.2.2 all_alloc_count . . . 118
9.12.2.3 alloc_bytes . . . 118
9.12.2.4 alloc_count . . . 118
9.12.2.5 head . . . 118
9.12.2.6 logfile . . . 118
9.12.2.7 max_block_size . . . 118
9.12.2.8 tail . . . 119
9.13 DebugmallocEntry Struct Reference . . . 119
9.13.1 Detailed Description . . . 119
9.13.2 Field Documentation . . . 119
9.13.2.1 expr . . . 119
9.13.2.2 file . . . 120
9.13.2.3 func . . . 120
9.13.2.4 line . . . 120
9.13.2.5 next . . . 120

9.13.2.6 prev . . . 120
9.13.2.7 real_mem . . . 120
9.13.2.8 size . . . 120
9.13.2.9 user_mem . . . 121
9.14 State Struct Reference . . . 121
9.14.1 Detailed Description . . . 121
9.14.2 Field Documentation . . . 121
9.14.2.1 vx . . . 121
9.14.2.2 vy . . . 121
9.14.2.3 vz . . . 122
9.14.2.4 x . . . 122
9.14.2.5 y . . . 122
9.14.2.6 z . . . 122
9.15 Target Struct Reference . . . 122
9.15.1 Detailed Description . . . 123
9.16 Target_List Struct Reference . . . 123
9.16.1 Detailed Description . . . 124
9.17 Target_Node Struct Reference . . . 124
9.17.1 Detailed Description . . . 125
9.18 Vec3 Struct Reference . . . 125
9.18.1 Detailed Description . . . 125
9.18.2 Field Documentation . . . 125
9.18.2.1 x . . . 125
9.18.2.2 y . . . 125
9.18.2.3 z . . . 125

10 File Documentation **127**

10.1 C:/vscode/ballistic/doxygen/calculations.c File Reference . . . 127
10.1.1 Detailed Description . . . 128
10.1.2 Macro Definition Documentation . . . 128
10.1.2.1 _USE_MATH_DEFINES . . . 128
10.1.2.2 AIR_DENSITY . . . 129
10.1.2.3 DEG2RAD . . . 129
10.1.2.4 G . . . 129
10.1.2.5 RAD2DEG . . . 129
10.1.2.6 sq . . . 129
10.2 calculations.c . . . 130
10.3 C:/vscode/ballistic/doxygen/calculations.h File Reference . . . 135
10.4 calculations.h . . . 136
10.5 C:/vscode/ballistic/doxygen/color.c File Reference . . . 136
10.5.1 Function Documentation . . . 137
10.5.1.1 get_color_mode() . . . 137

10.5.1.2 print_colored() . . . 137
10.5.1.3 set_color_mode() . . . 138
10.6 color.c . . . 138
10.7 C:/vscode/ballistic/doxygen/color.h File Reference . . . 139
10.7.1 Macro Definition Documentation . . . 140
10.7.1.1 ANSI_COLOR_BLUE . . . 140
10.7.1.2 ANSI_COLOR_CYAN . . . 140
10.7.1.3 ANSI_COLOR_GREEN . . . 140
10.7.1.4 ANSI_COLOR_MAGENTA . . . 140
10.7.1.5 ANSI_COLOR_RED . . . 140
10.7.1.6 ANSI_COLOR_RESET . . . 140
10.7.1.7 ANSI_COLOR_YELLOW . . . 140
10.7.2 Enumeration Type Documentation . . . 140
10.7.2.1 ColorMode . . . 140
10.7.3 Function Documentation . . . 141
10.7.3.1 get_color_mode() . . . 141
10.7.3.2 print_colored() . . . 141
10.7.3.3 set_color_mode() . . . 142
10.8 color.h . . . 142
10.9 C:/vscode/ballistic/doxygen/commands.c File Reference . . . 142
10.10 commands.c . . . 144
10.11 C:/vscode/ballistic/doxygen/commands.h File Reference . . . 148
10.12 commands.h . . . 150
10.13 C:/vscode/ballistic/doxygen/data_structures.dox File Reference . . . 151
10.14 C:/vscode/ballistic/doxygen/debug.c File Reference . . . 151
10.14.1 Function Documentation . . . 152
10.14.1.1 run_pre_test() . . . 152
10.15 debug.c . . . 153
10.16 C:/vscode/ballistic/doxygen/debug.h File Reference . . . 154
10.16.1 Macro Definition Documentation . . . 154
10.16.1.1 EMPTY_MESSAGE . . . 154
10.16.2 Enumeration Type Documentation . . . 155
10.16.2.1 bool . . . 155
10.16.3 Function Documentation . . . 155
10.16.3.1 run_pre_test() . . . 155
10.17 debug.h . . . 156
10.18 C:/vscode/ballistic/doxygen/debugmalloc.h File Reference . . . 156
10.18.1 Macro Definition Documentation . . . 157
10.18.1.1 calloc . . . 157
10.18.1.2 free . . . 158
10.18.1.3 malloc . . . 158
10.18.1.4 realloc . . . 158

10.18.1.5 strdup . . . 158
10.18.1.6 strndup . . . 158
10.18.2 Typedef Documentation . . . 159
10.18.2.1 DebugmallocData . . . 159
10.18.2.2 DebugmallocEntry . . . 159
10.18.3 Enumeration Type Documentation . . . 159
10.18.3.1 anonymous enum . . . 159
10.18.4 Function Documentation . . . 159
10.18.4.1 putenv() . . . 159
10.19 debugmalloc.h . . . 159
10.20 C:/vscode/ballistic/doxygen/doxygen_main.h File Reference . . . 166
10.21 doxygen_main.h . . . 166
10.22 C:/vscode/ballistic/doxygen/error.c File Reference . . . 166
10.23 error.c . . . 167
10.24 C:/vscode/ballistic/doxygen/error.h File Reference . . . 167
10.24.1 Detailed Description . . . 167
10.25 error.h . . . 168
10.26 C:/vscode/ballistic/doxygen/helpers.c File Reference . . . 168
10.26.1 Function Documentation . . . 169
10.26.1.1 ensure_trailing_backslash() . . . 169
10.26.1.2 norm_strncmp() . . . 169
10.26.1.3 Normalize_path() . . . 170
10.26.1.4 remove_quotes() . . . 170
10.26.1.5 strtoken() . . . 170
10.27 helpers.c . . . 171
10.28 C:/vscode/ballistic/doxygen/helpers.h File Reference . . . 172
10.28.1 Function Documentation . . . 174
10.28.1.1 ensure_trailing_backslash() . . . 174
10.28.1.2 norm_strncmp() . . . 174
10.28.1.3 Normalize_path() . . . 175
10.28.1.4 remove_quotes() . . . 175
10.28.1.5 strtoken() . . . 175
10.29 helpers.h . . . 176
10.30 C:/vscode/ballistic/doxygen/load.c File Reference . . . 176
10.31 load.c . . . 178
10.32 C:/vscode/ballistic/doxygen/load.h File Reference . . . 187
10.33 load.h . . . 189
10.34 C:/vscode/ballistic/doxygen/main.c File Reference . . . 189
10.34.1 Function Documentation . . . 190
10.34.1.1 CleanUp() . . . 190
10.34.1.2 main() . . . 191
10.35 main.c . . . 192

10.36 C:/vscode/ballistic/doxygen/main.h File Reference 194
10.36.1 Detailed Description 195
10.37 main.h 195
10.38 C:/vscode/ballistic/doxygen/storage.c File Reference 195
10.38.1 Function Documentation 198
10.38.1.1 pop_BackAmmo_t() 198
10.38.1.2 pop_FrontAmmo_t() 198
10.39 storage.c 198
10.40 C:/vscode/ballistic/doxygen/storage.h File Reference 210
10.41 storage.h 214

Index **217**

Chapter 1

Ballistic Trajectory Calculator

1.1 Overview

A comprehensive ballistic calculation system for artillery fire control. The program computes firing solutions, manages military bases, targets, and ammunition inventory, and provides optimal base-ammunition selection.

1.2 Features

- Ballistic trajectory simulation using WGS84 Earth model
- G7 drag coefficient model for projectile aerodynamics
- 4th-order Runge-Kutta numerical integration
- Geographic coordinate transformations (WGS84 ECEF ENU)
- Linked list data structures for all entities
- Configuration file support with persistent settings
- Color-coded terminal output (ANSI escape codes)
- Memory leak detection via debugmalloc
- Interactive command-line interface
- File I/O for data persistence
- Regional awareness for base selection (ID-based regions)

1.3 Architecture

The system is organized into the following modules:

- Storage: Core data structures and linked list implementations
- Loading: File parsing, configuration, and data loading
- Calculations: Ballistic computations and coordinate transformations
- CommandHandling: Interactive command processing and user interface
- Utilities: Helper functions, error handling, and color management
- Main: Program lifecycle and main control flow

1.4 Usage

1.4.0.1 Basic Operation:

1. Program starts and loads configuration from `config.txt`
2. Data files (`ammo_catalouge.txt`, `bases.txt`, `targets.txt`) are loaded
3. Interactive command prompt appears
4. Use commands like `calc` to compute firing solutions
5. Type `help` for available commands, `exit` to quit

1.4.0.2 Data Format Examples:

- Ammo: `I0 NHE W50.0 D120.0 BR10.0 MV800.0 BC0.5;`
- Base: `I0 LBase1 CLA47.4729283 CLO19.0531429 CA100.0 AMS:{0:100,1:50};`
- Target: `I0 LTarget1 CLA47.5 CLO19.1 CA50.0 A1.5;`

1.5 Technical Details

- **Earth Model**: WGS84 ellipsoid with ECEF/ENU coordinate systems
- **Ballistics**: G7 drag model, constant gravity (9.80665 m/s^2)
- **Integration**: 4th-order Runge-Kutta with 20ms time steps
- **Search**: 0.1° elevation increments from 5° to 80°
- **Tolerance**: 1 meter for distance and altitude matching
- **Memory**: Sentinel-linked lists with head/tail nodes

/**

1.6 References

1.6.0.1 Primary References

1. **World Geodetic System 1984 (WGS84)**
   National Geospatial-Intelligence Agency. (2020).
   Department of Defense World Geodetic System 1984.
   https://earth-info.nga.mil/index.php?dir=wgs84&action=wgs84
2. **G7 Ballistic Standard**
   Sierra Bullets. (2019).
   Understanding Ballistic Coefficients: G1 vs. G7.
   https://www.snipershide.com/shooting/threads/understanding-g1-and-g7-ballistic-coeff
   6984110/

1.6.0.2 Technical Standards

- ISO 80000-3:2006, *Quantities and units — Part 3: Space and time*
- ICAO Doc 7488/3, *Manual of the ICAO Standard Atmosphere*
- MIL-STD-2525D, *Common Warfighting Symbology*

1.6.0.3 Online Resources

- Earth-centered, Earth-fixed (ECEF) coordinate system: https://en.wikipedia.org/wiki/Earth-centered,_Earth-fixed_coordinate_system
- Haversine formula for great-circle distances: https://en.wikipedia.org/wiki/Haversine_formula
- Runge-Kutta methods for numerical integration: https://en.wikipedia.org/wiki/Runge%E2%80%93Kutta_methods
- Ballistic coefficient definitions: https://en.wikipedia.org/wiki/Ballistic_coefficient

1.6.0.4 Implementation Notes

This implementation uses:

- WGS84 ellipsoid parameters: a = 6378137.0 m, e^2 = 6.69437999014e-3
- G7 drag coefficient approximation as piecewise function of Mach number
- 4th-order Runge-Kutta with fixed 20 ms time steps
- Constant gravity assumption (9.80665 m/s^2)
- Sea-level atmospheric density (1.225 kg/m^3)

Author

Horváth Ádám József

Date

2025.12.01

Version

5.2

Todo Add wind effect modeling

Implement Coriolis force for long-range trajectories

Add binary search for elevation optimization

Implement solution caching for performance

Chapter 2

Todo List

page Ballistic Trajectory Calculator

Add wind effect modeling

Implement Coriolis force for long-range trajectories

Add binary search for elevation optimization

Implement solution caching for performance

File calculations.c

Add caching for frequently computed firing solutions

Implement binary search for elevation angle optimization

Add Coriolis effect for long-range trajectories

Add wind effect modeling

Global compute_fire_solution (Base *b, Target *t, Ammo_t *ammo)

Cache results for frequently used base-target-ammo combinations

Global SuggestBestBaseAndAmmo (Base_List *list, Target *tgt, Base **outBase, Ammo_t **outAmmo)

Implement early exit when distance is significantly worse than current best

Add weight factors for different ammunition types (lethality, cost, etc.)

Global SuggestBestBaseAndAmmo_RegionAware (Base_List *list, Target *tgt, Base **outBase, Ammo_t **outAmmo)

Make region calculation configurable (not hardcoded to 1000-based blocks)

Add region weighting factor (e.g., prefer same region but allow cross-region if much better)

Chapter 3

Topic Index

3.1 Topics

Here is a list of all topics with brief descriptions:

Ballistic Calculations 15
Command Processing 20
Utility Functions 38
File Loading and Configuration 40
Main Program 57
Data Storage Structures 58

Chapter 4

Directory Hierarchy

4.1 Directories

ballistic . . . 103
doxygen . . . 104
calculations.c . . . 127
calculations.h . . . 135
color.c . . . 136
color.h . . . 139
commands.c . . . 142
commands.h . . . 148
debug.c . . . 151
debug.h . . . 154
debugmalloc.h . . . 156
doxygen_main.h . . . 166
error.c . . . 166
error.h . . . 167
helpers.c . . . 168
helpers.h . . . 172
load.c . . . 176
load.h . . . 187
main.c . . . 189
main.h . . . 194
storage.c . . . 195
storage.h . . . 210
doxygen . . . 104
calculations.c . . . 127
calculations.h . . . 135
color.c . . . 136
color.h . . . 139
commands.c . . . 142
commands.h . . . 148
debug.c . . . 151
debug.h . . . 154
debugmalloc.h . . . 156
doxygen_main.h . . . 166
error.c . . . 166
error.h . . . 167
helpers.c . . . 168
helpers.h . . . 172
load.c . . . 176

load.h . . . 187
main.c . . . 189
main.h . . . 194
storage.c . . . 195
storage.h . . . 210
vscode . . . 104
ballistic . . . 103
doxygen . . . 104
calculations.c . . . 127
calculations.h . . . 135
color.c . . . 136
color.h . . . 139
commands.c . . . 142
commands.h . . . 148
debug.c . . . 151
debug.h . . . 154
debugmalloc.h . . . 156
doxygen_main.h . . . 166
error.c . . . 166
error.h . . . 167
helpers.c . . . 168
helpers.h . . . 172
load.c . . . 176
load.h . . . 187
main.c . . . 189
main.h . . . 194
storage.c . . . 195
storage.h . . . 210

Chapter 5

Data Structure Index

5.1 Data Structures

Here are the data structures with brief descriptions:

Ammo_Stock
: Base ammunition stock entry . . . 105

Ammo_Stock_List
: Ammunition stock list for a single base (sorted by ammo ID) . . . 106

Ammo_t
: Ammunition type definition . . . 107

Ammo_t_List
: Global ammunition type list (sorted by ID) . . . 108

Ammo_t_Node
: Node for global ammunition type linked list . . . 109

BallisticResult
: Ballistic calculation results . . . 109

Base
: Military base structure . . . 111

Base_List
: Base list (sorted by ID) . . . 112

Base_Node
: Node for base linked list . . . 114

Command
: Command definition structure . . . 115

Coordinate
: Geographic coordinate structure . . . 116

DebugmallocData . . . 117

DebugmallocEntry . . . 119

State
: State vector for ballistic trajectory simulation . . . 121

Target
: Target structure . . . 122

Target_List
: Target list (sorted by ID) . . . 123

Target_Node
: Node for target linked list . . . 124

Vec3
: 3D vector structure for coordinate calculations . . . 125

Chapter 6

File Index

6.1 File List

Here is a list of all files with brief descriptions:

C:/vscode/ballistic/doxygen/calculations.c
Implementation of ballistic calculation functions . . . 127
C:/vscode/ballistic/doxygen/calculations.h . . . 135
C:/vscode/ballistic/doxygen/color.c . . . 136
C:/vscode/ballistic/doxygen/color.h . . . 139
C:/vscode/ballistic/doxygen/commands.c . . . 142
C:/vscode/ballistic/doxygen/commands.h . . . 148
C:/vscode/ballistic/doxygen/debug.c . . . 151
C:/vscode/ballistic/doxygen/debug.h . . . 154
C:/vscode/ballistic/doxygen/debugmalloc.h . . . 156
C:/vscode/ballistic/doxygen/doxygen_main.h . . . 166
C:/vscode/ballistic/doxygen/error.c . . . 166
C:/vscode/ballistic/doxygen/error.h
Error codes and process states . . . 167
C:/vscode/ballistic/doxygen/helpers.c . . . 168
C:/vscode/ballistic/doxygen/helpers.h . . . 172
C:/vscode/ballistic/doxygen/load.c . . . 176
C:/vscode/ballistic/doxygen/load.h . . . 187
C:/vscode/ballistic/doxygen/main.c . . . 189
C:/vscode/ballistic/doxygen/main.h
Main program declarations . . . 194
C:/vscode/ballistic/doxygen/storage.c . . . 195
C:/vscode/ballistic/doxygen/storage.h . . . 210

Chapter 7

Topic Documentation

7.1 Ballistic Calculations

Ballistic trajectory simulation and coordinate transformations.

Data Structures

- struct BallisticResult

  Ballistic calculation results.

Functions

- BallisticResult compute_fire_solution (Base ∗b, Target ∗t, Ammo_t ∗ammo)

  Compute firing solution for base-target-ammunition combination.
- int SuggestBestBaseAndAmmo (Base_List ∗list, Target ∗tgt, Base ∗∗outBase, Ammo_t ∗∗outAmmo)

  Suggest optimal base and ammunition for a target.
- int SuggestBestBaseAndAmmo_RegionAware (Base_List ∗list, Target ∗tgt, Base ∗∗outBase, Ammo_t ∗∗outAmmo)

  Suggest optimal base and ammunition with regional preference.

7.1.1 Detailed Description

Ballistic trajectory simulation and coordinate transformations.

This module performs all ballistic calculations including:

- WGS84 to ECEF coordinate transformations
- ECEF to local ENU coordinate transformations
- G7 drag coefficient modeling
- 4th-order Runge-Kutta trajectory simulation
- Optimal base and ammunition selection algorithms

7.1.2 Function Documentation

7.1.2.1 compute_fire_solution()

```
BallisticResult compute_fire_solution (
            Base * b,
            Target * t,
            Ammo_t * ammo)
```

```
#include <calculations.h>
```

Compute firing solution for base-target-ammunition combination.

Main ballistic calculation function. Performs coordinate transformations and trajectory simulation to determine firing parameters.

Parameters

b	Pointer to firing base
t	Pointer to target
ammo	Pointer to ammunition type

Returns

BallisticResult structure with all calculated parameters

Note

Uses 1 meter tolerance for altitude matching

Searches elevation angles from 5° to 80° in 0.1° increments

Maximum simulated flight time: 200 seconds

Compute firing solution for base-target-ammunition combination.

Main function that orchestrates coordinate transformations and trajectory calculations to determine firing parameters.

Parameters

b	Pointer to the firing base structure
t	Pointer to the target structure
ammo	Pointer to the ammunition type structure

Returns

BallisticResult structure containing all calculated firing parameters

See also

BallisticResult

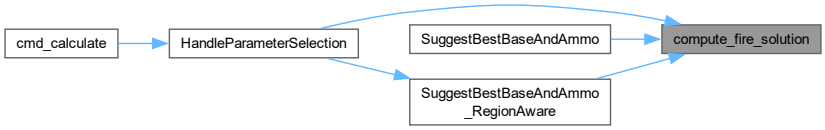
Note

Firing solution is invalid if no elevation angle produces a hit within tolerance

Todo Cache results for frequently used base-target-ammo combinations

Definition at line 318 of file calculations.c.

Here is the caller graph for this function:



7.1.2.2 SuggestBestBaseAndAmmo()

```
int SuggestBestBaseAndAmmo (
            Base_List * list,
            Target * tgt,
            Base ** outBase,
            Ammo_t ** outAmmo)
```

```
#include <calculations.h>
```

Suggest optimal base and ammunition for a target.

Evaluates all bases and ammunition combinations to find the best match based on distance and ammunition availability.

Selection criteria (in order):

1. Valid firing solution exists
2. Minimum 3D distance to target (primary)
3. Maximum ammunition quantity (tie-breaker)

Parameters

list	Pointer to base list
tgt	Pointer to target

outBase	Double pointer to receive selected base
outAmmo	Double pointer to receive selected ammunition

Returns

1 if valid combination found, 0 otherwise

Note

Uses 1 meter tolerance for distance comparisons

Complexity: O(n×m) where n=bases, m=ammunition types per base

Suggest optimal base and ammunition for a target.

Evaluates all bases and their available ammunition to find the optimal combination based on:

1. Closest distance to target (primary criterion)
2. Ammunition quantity (tie-breaker)

Parameters

list	Pointer to the list of available bases
tgt	Pointer to the target structure
outBase	Double pointer to return the selected base (output parameter)
outAmmo	Double pointer to return the selected ammunition (output parameter)

Returns

1 if a valid combination was found, 0 otherwise

Note

Only considers ammunition with valid firing solutions

Uses 1 meter tolerance for distance comparisons

O(n∗m) complexity where n = number of bases, m = ammunition types per base

Todo Implement early exit when distance is significantly worse than current best

Add weight factors for different ammunition types (lethality, cost, etc.)

Definition at line 423 of file calculations.c.

Here is the call graph for this function:



7.1.2.3 SuggestBestBaseAndAmmo_RegionAware()

```
int SuggestBestBaseAndAmmo_RegionAware (
            Base_List * list,
            Target * tgt,
            Base ** outBase,
            Ammo_t ** outAmmo)
```

```
#include <calculations.h>
```

Suggest optimal base and ammunition with regional preference.

Similar to SuggestBestBaseAndAmmo but prefers bases in the same region as the target. Region is determined by ID blocks of 1000.

Algorithm:

1. First pass: Only consider bases with IDs in same 1000-block as target
2. If no solution in same region, second pass: Consider all bases

Parameters

list	Pointer to base list
tgt	Pointer to target
outBase	Double pointer to receive selected base
outAmmo	Double pointer to receive selected ammunition

Returns

1 if valid combination found, 0 otherwise

Note

Region: target.id/1000 determines the 1000-based block

Uses same selection criteria as SuggestBestBaseAndAmmo

Suggest optimal base and ammunition with regional preference.

Similar to SuggestBestBaseAndAmmo but prioritizes bases in the same region as the target. Region is determined by ID ranges (1000-based blocks).

Algorithm:

1. First pass: only consider bases in same region as target
2. If no solution found in same region, second pass: consider all bases

Parameters

list	Pointer to the list of available bases
tgt	Pointer to the target structure
outBase	Double pointer to return the selected base (output parameter)
outAmmo	Double pointer to return the selected ammunition (output parameter)

Returns

1 if a valid combination was found, 0 otherwise

Note

Region is determined by ID: target.id/1000 defines region block

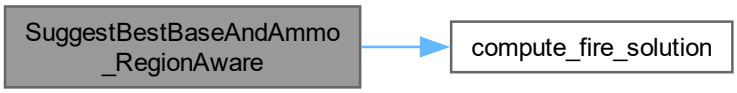
Uses 1 meter tolerance for distance comparisons

Todo Make region calculation configurable (not hardcoded to 1000-based blocks)

Add region weighting factor (e.g., prefer same region but allow cross-region if much better)

Definition at line 514 of file calculations.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.2 Command Processing

Interactive command-line interface and user interaction.

Data Structures

- struct Command

  Command definition structure.

Macros

- #define CMD_C_ALL 0xFFFFFFFFu

  All command contexts combined.

Enumerations

- enum Command_case {
  CMD_C_DEFAULT = 1 << 0 , CMD_C_GENERIC = 1 << 1 , CMD_C_SYSTEM = 1 << 2 ,
  CMD_C_YESNO = 1 << 3 ,
  CMD_C_LOAD = 1 << 4 , CMD_C_SETUP = 1 << 5 , CMD_C_EXIT = 1 << 6 , CMD_C_CALC = 1 << 7
  ,
  CMD_C_SELECT = 1 << 8 , CMD_C_BACK = 1 << 9 , CMD_C_CANCEL = 1 << 10 , CMD_C_LOOP =
  1 << 20 }

  Command context flags for modal operation.
- enum Command_t {
  CMD_INVALID , CMD_EXIT , CMD_HELP , CMD_CONTINUE ,
  CMD_YES , CMD_NO , CMD_SAVE , CMD_CANCEL ,
  CMD_SELECT , CMD_BACK , CMD_CALCULATE , CMD_RESET ,
  CMD_TARGET , CMD_BASE , CMD_AMMO , CMD_TEST ,
  CMD_LIST , CMD_ADD , CMD_DELETE , CMD_COLOUR ,
  CMD_MONO , CMD_CLEAR_TERMINAL }

  Command type identifiers.

Variables

- int command_count

  Number of commands in command table.
- Command commands [ ]

  Global command table.

Core Command Functions

- Command_t GetCommand (Command_case case_flags, PROCESS *running)

  Get command from user with modal context.
- Command_t ParseCommandFromBuffer (char *input, Command_case case_flags, PROCESS *running)

  Parse command from buffer (non-interactive).
- void NormalizeInput (char *input)

  Normalize input string to lowercase.
- void add_seperator (const char *str, char symbol, unsigned sep_length)

  Print formatted separator line.
- void clear_terminal ()

  Clear terminal screen.

Command Handler Functions

Each handler corresponds to a specific command. Handlers receive arguments string and current context flags.

- void cmd_exit (const char *args, Command_case case_flags, PROCESS *running)
  Exit program handler.
- void cmd_help (const char *args, Command_case case_flags, PROCESS *running)
  Help command handler.
- void cmd_base (const char *args, Command_case case_flags, PROCESS *running)
  Base operations handler.
- void cmd_calculate (const char *args, Command_case case_flags, PROCESS *running)
  Calculate trajectory handler.
- void cmd_ammo (const char *args, Command_case case_flags, PROCESS *running)
  Ammunition operations handler.
- void cmd_target (const char *args, Command_case case_flags, PROCESS *running)
  Target operations handler.
- void cmd_test (const char *args, Command_case case_flags, PROCESS *running)
  Test system handler.
- void cmd_config (const char *args, Command_case case_flags, PROCESS *running)
  Configuration handler.
- void cmd_colour (const char *args, Command_case case_flags, PROCESS *running)
  Set color mode handler.
- void cmd_mono (const char *args, Command_case case_flags, PROCESS *running)
  Set monochrome mode handler.
- void cmd_clr_ter (const char *args, Command_case case_flags, PROCESS *running)
  Clear terminal handler.

7.2.1 Detailed Description

Interactive command-line interface and user interaction.

This module handles all user commands, provides help system, and manages the interactive command loop with modal support.

7.2.2 Macro Definition Documentation

7.2.2.1 CMD_C_ALL

```
#define CMD_C_ALL 0xFFFFFFFFu
```

```
#include <commands.h>
```

All command contexts combined.

Definition at line 71 of file commands.h.

7.2.3 Enumeration Type Documentation

7.2.3.1 Command_case

```
enum Command_case
```

```
#include <commands.h>
```

Command context flags for modal operation.

Bitmask flags defining which commands are available in each context.

Enumerator

CMD_C_DEFAULT	Default commands with handlers.
CMD_C_GENERIC	Generic commands without handlers.
CMD_C_SYSTEM	System commands (exit, help, etc.).
CMD_C_YESNO	Yes/No commands.
CMD_C_LOAD	Data loading commands.
CMD_C_SETUP	Setup/configuration commands.
CMD_C_EXIT	Exit-related commands.
CMD_C_CALC	Calculation commands.
CMD_C_SELECT	Selection commands.
CMD_C_BACK	Back/return commands.
CMD_C_CANCEL	Cancel commands.
CMD_C_LOOP	Special: loop until valid input.

Definition at line 51 of file commands.h.

7.2.3.2 Command_t

```
enum Command_t
```

```
#include <commands.h>
```

Command type identifiers.

Enumerator

CMD_INVALID	Invalid or unknown command.
CMD_EXIT	Exit program.
CMD_HELP	Show help.
CMD_CONTINUE	Continue operation.
CMD_YES	Yes response.
CMD_NO	No response.
CMD_SAVE	Save data.
CMD_CANCEL	Cancel operation.
CMD_SELECT	Select option.
CMD_BACK	Go back/return.
CMD_CALCULATE	Calculate trajectory.
CMD_RESET	Reset system.
CMD_TARGET	Target operations.
CMD_BASE	Base operations.
CMD_AMMO	Ammunition operations.
CMD_TEST	Run test.
CMD_LIST	List items.
CMD_ADD	Add item.

CMD_DELETE	Delete item.
CMD_COLOUR	Set color mode.
CMD_MONO	Set monochrome mode.
CMD_CLEAR_TERMINAL	Clear terminal.

Definition at line 19 of file commands.h.

7.2.4 Function Documentation

7.2.4.1 add_seperator()

```
void add_seperator (
            const char * str,
            char symbol,
            unsigned sep_length)
```

```
#include <commands.h>
```

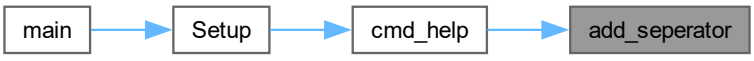
Print formatted separator line.

Parameters

str	Text to center in separator (NULL for line only)
symbol	Character to use for line
sep_length	Total length of separator

Definition at line 324 of file commands.c.

Here is the caller graph for this function:



7.2.4.2 clear_terminal()

```
void clear_terminal ()
```

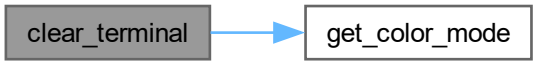
```
#include <commands.h>
```

Clear terminal screen.

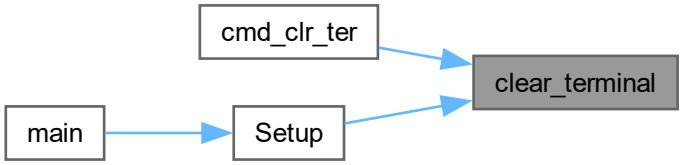
Uses ANSI escape codes if color mode enabled, otherwise no effect.

Definition at line 343 of file commands.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.2.4.3 cmd_ammo()

```
void cmd_ammo (
            const char * args,
            Command_case case_flags,
            PROCESS * running)
```

```
#include <commands.h>
```

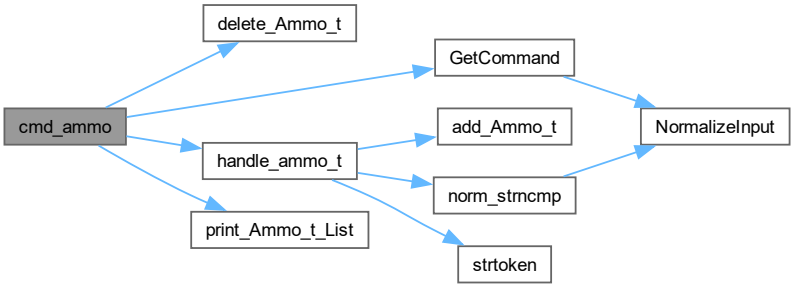
Ammunition operations handler.

Parameters

args	Command arguments
case_flags	Current context flags
running	Pointer to program state

Definition at line 130 of file commands.c.

Here is the call graph for this function:



7.2.4.4 cmd_base()

```
void cmd_base (
            const char * args,
            Command_case case_flags,
            PROCESS * running)
```

```
#include <commands.h>
```

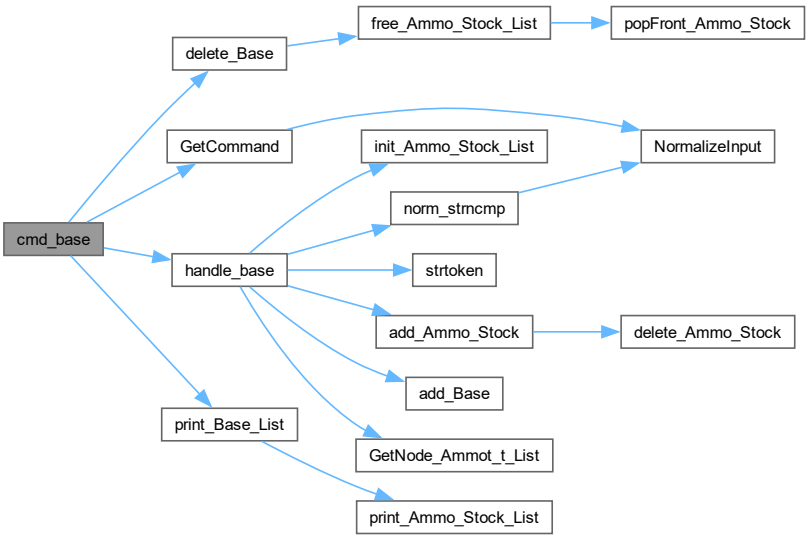
Base operations handler.

Parameters

args	Command arguments
case_flags	Current context flags
running	Pointer to program state

Definition at line 101 of file commands.c.

Here is the call graph for this function:



7.2.4.5 cmd_calculate()

```
void cmd_calculate (
            const char * args,
            Command_case case_flags,
            PROCESS * running)
```

```
#include <commands.h>
```

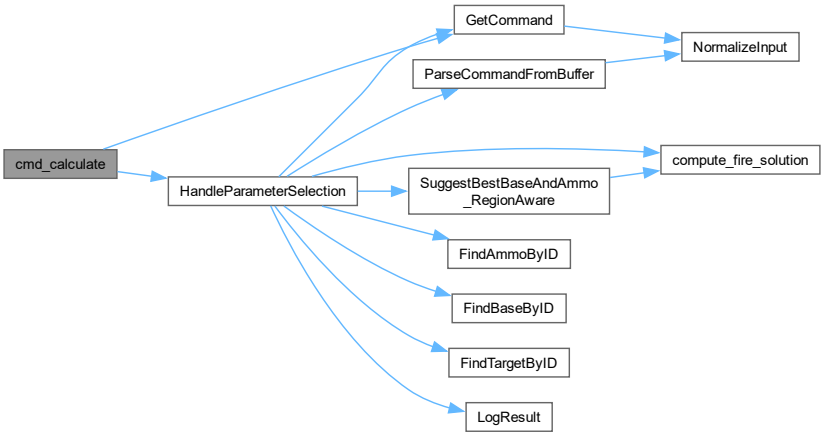
Calculate trajectory handler.

Parameters

args	Command arguments
case_flags	Current context flags
running	Pointer to program state

Definition at line 189 of file commands.c.

Here is the call graph for this function:



7.2.4.6 cmd_clr_ter()

```
void cmd_clr_ter (
            const char * args,
            Command_case case_flags,
            PROCESS * running)
```

```
#include <commands.h>
```

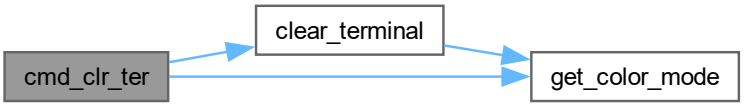
Clear terminal handler.

Parameters

args	Command arguments
case_flags	Current context flags
running	Pointer to program state

Definition at line 178 of file commands.c.

Here is the call graph for this function:



7.2.4.7 cmd_colour()

```
void cmd_colour (
            const char * args,
            Command_case case_flags,
            PROCESS * running)
```

```
#include <commands.h>
```

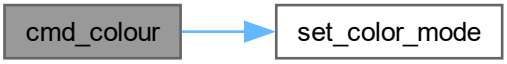
Set color mode handler.

Parameters

args	Command arguments
case_flags	Current context flags
running	Pointer to program state

Definition at line 167 of file commands.c.

Here is the call graph for this function:



7.2.4.8 cmd_config()

```
void cmd_config (
            const char * args,
            Command_case case_flags,
            PROCESS * running)
```

```
#include <commands.h>
```

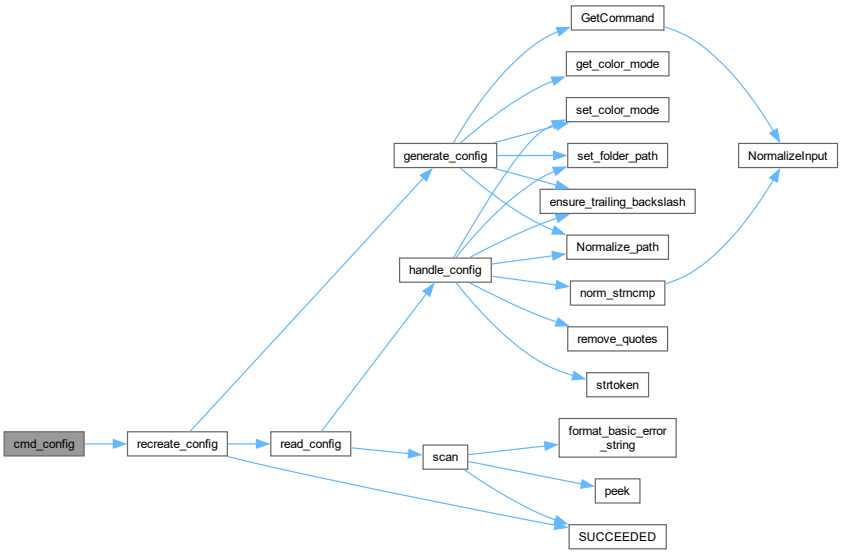
Configuration handler.

Parameters

args	Command arguments
case_flags	Current context flags
running	Pointer to program state

Definition at line 184 of file commands.c.

Here is the call graph for this function:



7.2.4.9 cmd_exit()

```
void cmd_exit (
            const char * args,
            Command_case case_flags,
            PROCESS * running)
```

```
#include <commands.h>
```

Exit program handler.

Parameters

args	Command arguments (unused)
case_flags	Current context flags
running	Pointer to program state (set to S_END)

Definition at line 46 of file commands.c.

7.2.4.10 cmd_help()

```
void cmd_help (
            const char * args,
            Command_case case_flags,
            PROCESS * running)
```

```
#include <commands.h>
```

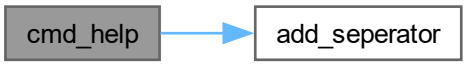
Help command handler.

Parameters

args	Command arguments (unused)
case_flags	Current context flags (determines which commands to show)
running	Pointer to program state (unused)

Definition at line 57 of file commands.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.2.4.11 cmd_mono()

```
void cmd_mono (
            const char * args,
            Command_case case_flags,
            PROCESS * running)
```

```
#include <commands.h>
```

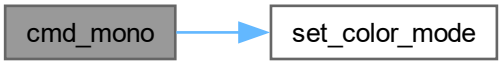
Set monochrome mode handler.

Parameters

args	Command arguments
case_flags	Current context flags
running	Pointer to program state

Definition at line 172 of file commands.c.

Here is the call graph for this function:



7.2.4.12 cmd_target()

```
void cmd_target (
            const char * args,
            Command_case case_flags,
            PROCESS * running)
```

```
#include <commands.h>
```

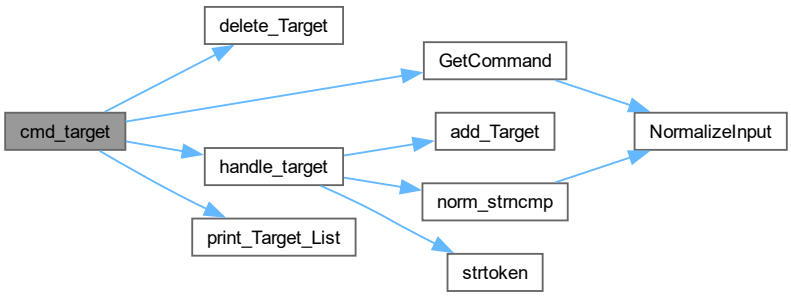
Target operations handler.

Parameters

args	Command arguments
case_flags	Current context flags
running	Pointer to program state

Definition at line 71 of file commands.c.

Here is the call graph for this function:



7.2.4.13 cmd_test()

```
void cmd_test (
            const char * args,
            Command_case case_flags,
            PROCESS * running)
```

```
#include <commands.h>
```

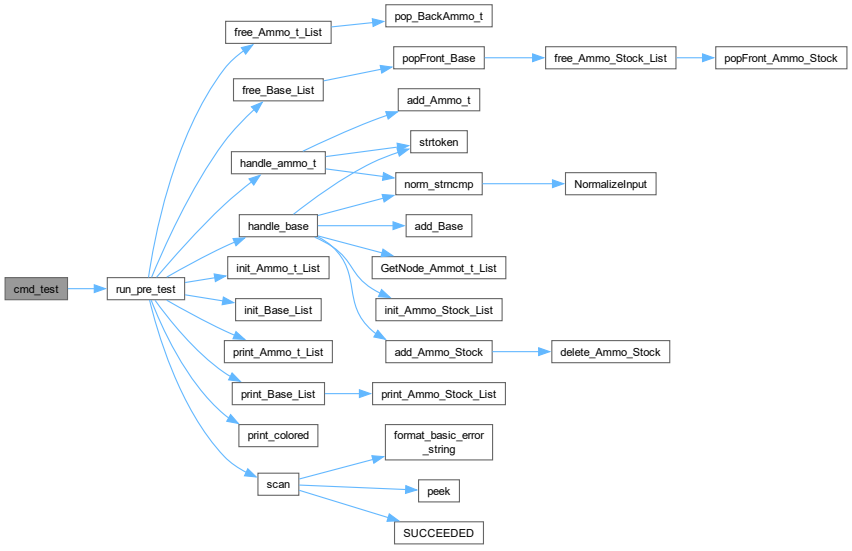
Test system handler.

Parameters

args	Command arguments
case_flags	Current context flags
running	Pointer to program state

Definition at line 161 of file commands.c.

Here is the call graph for this function:



7.2.4.14 GetCommand()

```
Command_t GetCommand (
            Command_case case_flags,
            PROCESS * running)
```

```
#include <commands.h>
```

Get command from user with modal context.

Reads user input, validates against available commands in current context, and executes the appropriate handler.

Parameters

case_flags	Bitmask defining available commands in current context
running	Pointer to program state (may be modified by handlers)

Returns

Command_t identifier of executed command

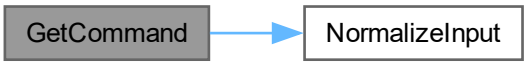
Note

If CMD_C_LOOP is set, function loops until valid command received

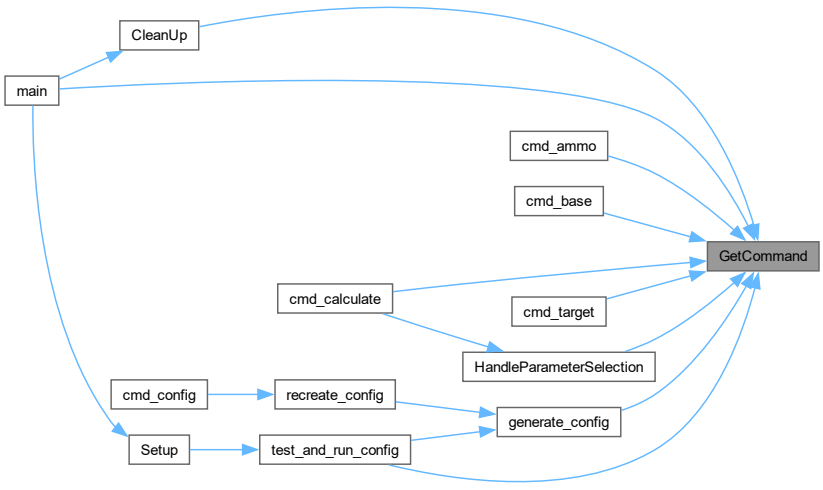
Normalizes input to lowercase before comparison

Definition at line 217 of file commands.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.2.4.15 NormalizeInput()

```
void NormalizeInput (
            char * input)
```

```
#include <commands.h>
```

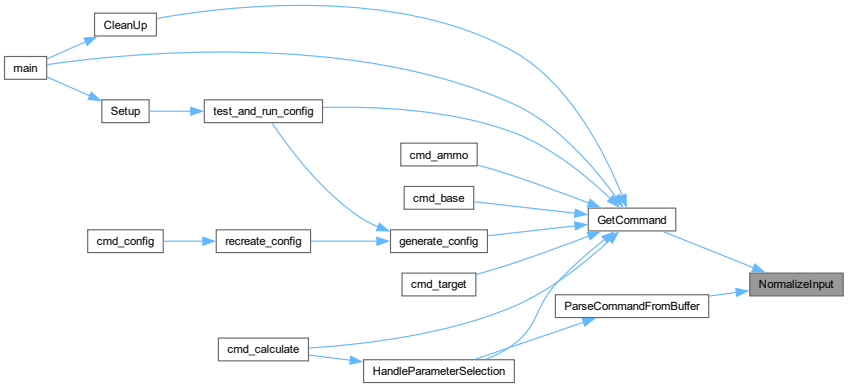
Normalize input string to lowercase.

Parameters

input	String to normalize (modified in-place)

Definition at line 103 of file helpers.c.

Here is the caller graph for this function:



7.2.4.16 ParseCommandFromBuffer()

```
Command_t ParseCommandFromBuffer (
            char * input,
            Command_case case_flags,
            PROCESS * running)
```

```
#include <commands.h>
```

Parse command from buffer (non-interactive).

Similar to GetCommand but works with pre-existing input buffer. Useful for parsing commands from files or other sources.

Parameters

input	Input buffer containing command
case_flags	Bitmask of allowed commands
running	Pointer to program state

Returns

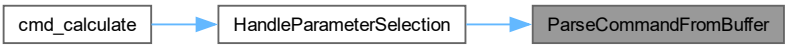
Command_t identifier

Definition at line 274 of file commands.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.2.5 Variable Documentation

7.2.5.1 command_count

```
int command_count  [extern]
```

```
#include <commands.h>
```

Number of commands in command table.

Definition at line 42 of file commands.c.

7.2.5.2 commands

```
Command commands[]  [extern]
```

```
#include <commands.h>
```

Global command table.

Array of all available commands with their handlers and contexts.

Definition at line 14 of file commands.c.

7.3 Utility Functions

Error handling, color management, helper functions, and debugging.

Files

- file error.h

  Error codes and process states.

Enumerations

- enum HRESULT { S_OK = 0 , S_FALSE = 1 , S_PARTIAL = 2 , E_FAIL = -1 }

  Result codes (similar to Windows HRESULT).
- enum PROCESS { S_RUNNING , S_END , S_TERMINATE , S_DEBUG }

  Program process states.

Functions

- int FAILED (HRESULT hr)

  Check if result indicates failure.
- int SUCCEEDED (HRESULT hr)

  Check if result indicates success.

7.3.1 Detailed Description

Error handling, color management, helper functions, and debugging.

This module contains utility functions used throughout the system.

7.3.2 Enumeration Type Documentation

7.3.2.1 HRESULT

```
enum HRESULT
```

```
#include <error.h>
```

Result codes (similar to Windows HRESULT).

Enumerator

S_OK	Operation successful.
S_FALSE	Operation completed but with non-fatal issue.
S_PARTIAL	Operation partially completed (missing parameters).

E_FAIL	Fatal error occurred.

Definition at line 21 of file error.h.

7.3.2.2 PROCESS

```
enum PROCESS
```

```
#include <error.h>
```

Program process states.

Enumerator

S_RUNNING	Program is running normally.
S_END	Program is ending gracefully.
S_TERMINATE	Program is terminating due to error.
S_DEBUG	Program is in debug mode.

Definition at line 33 of file error.h.

7.3.3 Function Documentation

7.3.3.1 FAILED()

```
int FAILED (
            HRESULT hr)
```

```
#include <error.h>
```

Check if result indicates failure.

Parameters

hr	Result code to check

Returns

1 if failure (hr $<$ 0), 0 otherwise

Definition at line 8 of file error.c.

Here is the caller graph for this function:



7.3.3.2 SUCCEEDED()

```
int SUCCEEDED (
            HRESULT hr)
```

```
#include <error.h>
```

Check if result indicates success.

Parameters

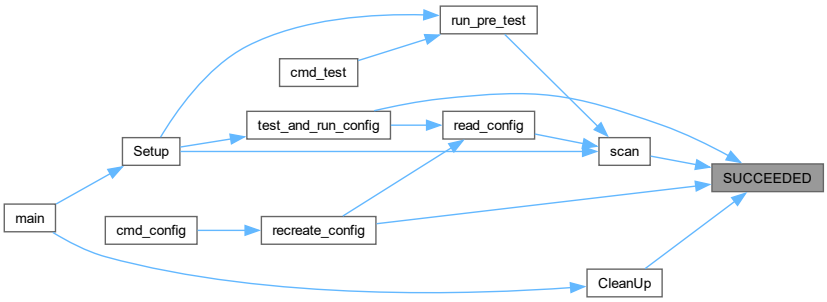
hr	Result code to check

Returns

1 if success (hr >= 0), 0 otherwise

Definition at line 4 of file error.c.

Here is the caller graph for this function:



7.4 File Loading and Configuration

File parsing, configuration management, and data loading.

Typedefs

- typedef HRESULT(∗ RecordHandler) (char ∗record, void ∗data, int ∗error_flags)

  Record handler function type for parser.

Enumerations

- enum ParseState { NORMAL , LINE_COMMENT , BLOCK_COMMENT , RECORD }

  Parser state machine states.

Functions

- HRESULT format_basic_error_string (char *error_buffer, int *error_flags)

  Format error string for missing parameters.
- void free_folder_path (void)

  Free the working directory path memory.
- int HandleParameterSelection (void)

  Interactive parameter selection for calculations.
- void LogResult (const Base *base, const Target *tgt, const Ammo_t *ammo, const BallisticResult *res)

  Log calculation result to file.
- int peek (FILE *file)

  Peek next character from file without consuming it.
- HRESULT scan (const char *filename, RecordHandler handler, void *data, PROCESS mode)

  Parse a file and call handler for each record.
- void set_folder_path (const char *new_path)

  Set the working directory path.

Variables

- char * folder_path

  Pointer to current working directory path.

Configuration Functions

- HRESULT generate_config (FILE *configfile, const char *config_file)

  Generate new configuration file interactively.
- HRESULT recreate_config (const char *config_file, PROCESS mode)

  Recreate configuration file with new settings.
- HRESULT read_config (const char *config_file)

  Read and apply existing configuration file.
- HRESULT test_and_run_config (const char *config_file, PROCESS mode)

  Test configuration and create if missing.
- HRESULT handle_config (char *record, void *data, int *error_flags)

  Handle configuration file record parsing.

Data File Handlers

- HRESULT handle_ammo_t (char *record, void *data, int *error_flags)

  Parse ammunition type record.
- HRESULT handle_base (char *record, void *data, int *error_flags)

  Parse base record.
- HRESULT handle_target (char *record, void *data, int *error_flags)

  Parse target record.

7.4.1 Detailed Description

File parsing, configuration management, and data loading.

This module handles all file I/O operations including configuration, data file parsing with custom format, and logging of results.

7.4.2 Typedef Documentation

7.4.2.1 RecordHandler

```
typedef HRESULT(* RecordHandler) (char *record, void *data, int *error_flags)
```

```
#include <load.h>
```

Record handler function type for parser.

Callback function called by parser for each complete record.

Parameters

record	Complete record string (without trailing ';')
data	User data passed to parser
error_flags	Array to indicate missing required fields

Returns

HRESULT status code

Definition at line 60 of file load.h.

7.4.3 Enumeration Type Documentation

7.4.3.1 ParseState

```
enum ParseState
```

```
#include <load.h>
```

Parser state machine states.

Enumerator

NORMAL	Reading normal data.
LINE_COMMENT	Inside // comment.
BLOCK_COMMENT	Inside /* ... */ comment.
RECORD	Collecting record text until ';'.

Definition at line 21 of file load.h.

7.4.4 Function Documentation

7.4.4.1 format_basic_error_string()

```
HRESULT format_basic_error_string (
            char * error_buffer,
            int * error_flags)
```

```
#include <load.h>
```

Format error string for missing parameters.

Parameters

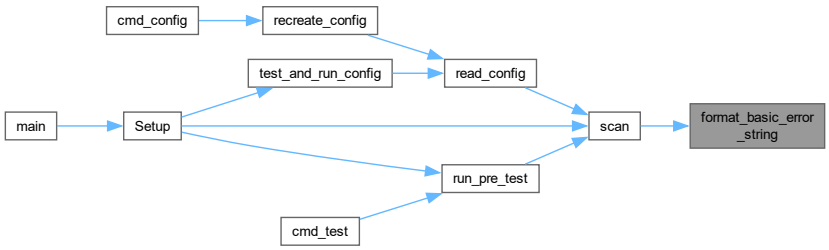
error_buffer	Buffer to write error message
error_flags	Array indicating which fields are missing

Returns

S_OK on success, S_FALSE on buffer overflow

Definition at line 177 of file load.c.

Here is the caller graph for this function:



7.4.4.2 free_folder_path()

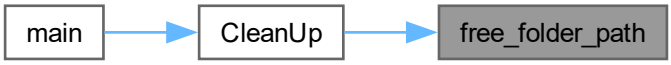
```
void free_folder_path (
            void )
```

```
#include <load.h>
```

Free the working directory path memory.

Definition at line 39 of file load.c.

Here is the caller graph for this function:



7.4.4.3 generate_config()

```
HRESULT generate_config (
            FILE * configfile,
            const char * config_file)
```

```
#include <load.h>
```

Generate new configuration file interactively.

Parameters

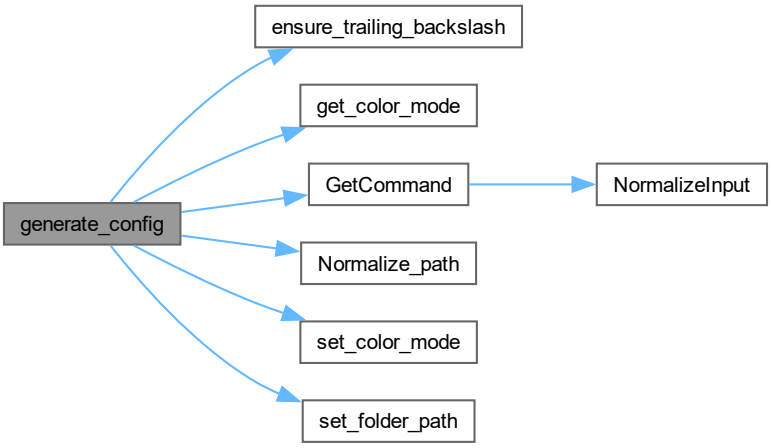
configfile	FILE pointer (can be NULL)
config_file	Configuration file path

Returns

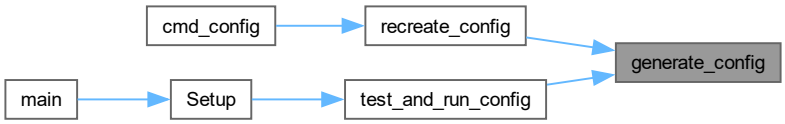
S_OK on success, S_FALSE on error

Definition at line 297 of file load.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.4.4.4 handle_ammo_t()

```
HRESULT handle_ammo_t (
            char * record,
            void * data,
            int * error_flags)
```

```
#include <load.h>
```

Parse ammunition type record.

Parameters

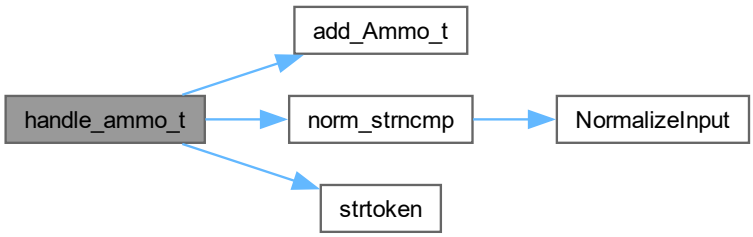
record	Ammunition record string
data	Pointer to Ammo_t_List
error_flags	Array to indicate missing fields

Returns

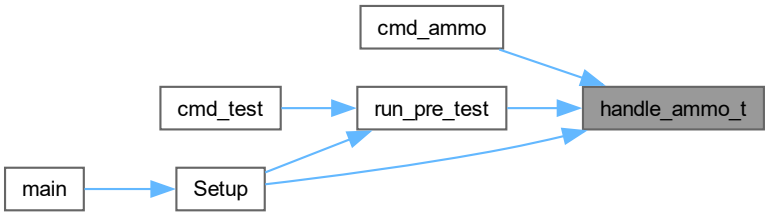
S_OK on success

Definition at line 388 of file load.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.4.4.5 handle_base()

```
HRESULT handle_base (
            char * record,
            void * data,
            int * error_flags)
```

```
#include <load.h>
```

Parse base record.

Parameters

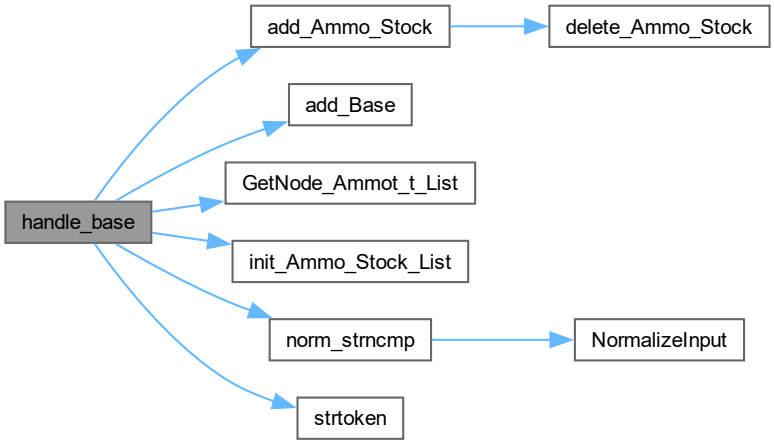
record	Base record string
data	Pointer to Base_List
error_flags	Array to indicate missing fields

Returns

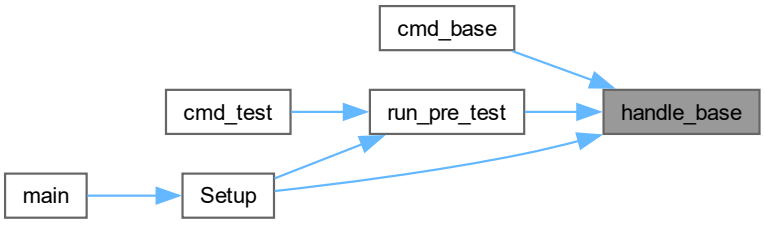
S_OK on success

Definition at line 465 of file load.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.4.4.6 handle_config()

```
HRESULT handle_config (
            char * record,
            void * data,
            int * error_flags)
```

```
#include <load.h>
```

Handle configuration file record parsing.

Parameters

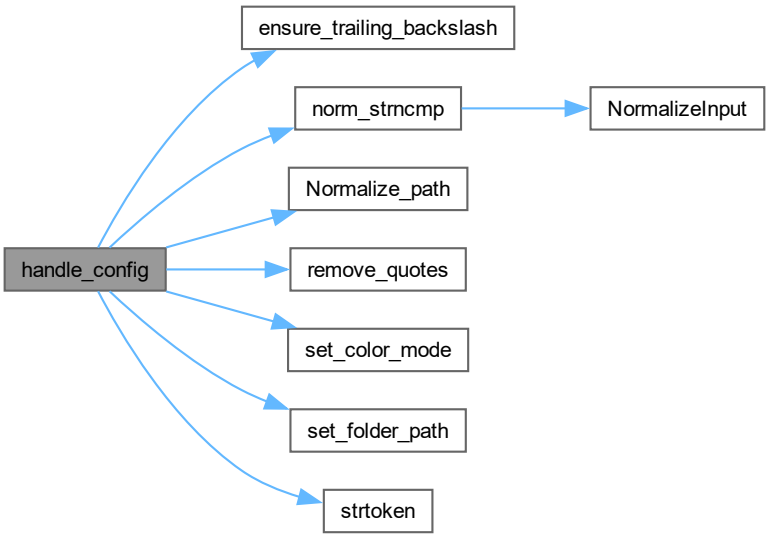
record	Configuration record string
data	User data (unused)
error_flags	Error flags array

Returns

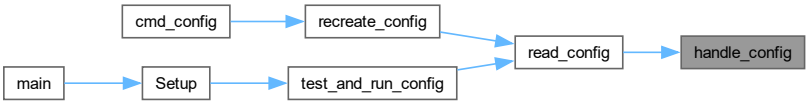
S_OK always (errors are non-fatal for config)

Definition at line 369 of file load.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.4.4.7 handle_target()

```
HRESULT handle_target (
            char * record,
            void * data,
            int * error_flags)
```

```
#include <load.h>
```

Parse target record.

Parameters

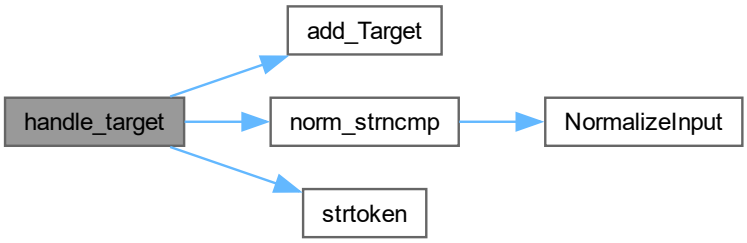
record	Target record string
data	Pointer to Target_List
error_flags	Array to indicate missing fields

Returns

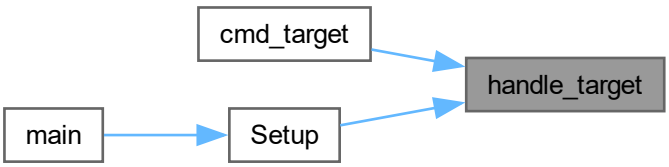
S_OK on success

Definition at line 549 of file load.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.4.4.8 HandleParameterSelection()

```
int HandleParameterSelection (
            void )
```

```
#include <load.h>
```

Interactive parameter selection for calculations.

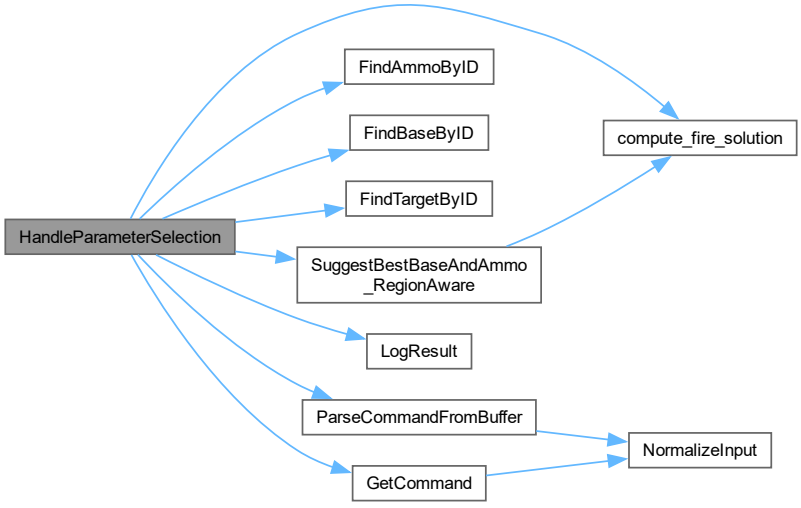
Guides user through selecting base, target, and ammunition IDs. Provides suggestions if invalid combinations are selected.

Returns

0 on success, -1 to return to menu

Definition at line 693 of file load.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.4.4.9 LogResult()

```
void LogResult (
            const Base * base,
            const Target * tgt,
            const Ammo_t * ammo,
            const BallisticResult * res)
```

```
#include <load.h>
```

Log calculation result to file.

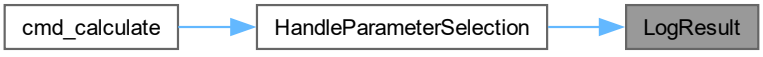
Appends detailed firing solution to log.txt in working directory.

Parameters

base	Pointer to firing base
tgt	Pointer to target
ammo	Pointer to ammunition type
res	Pointer to ballistic result

Definition at line 617 of file load.c.

Here is the caller graph for this function:



7.4.4.10 peek()

```
int peek (
            FILE * file)
```

```
#include <load.h>
```

Peek next character from file without consuming it.

Parameters

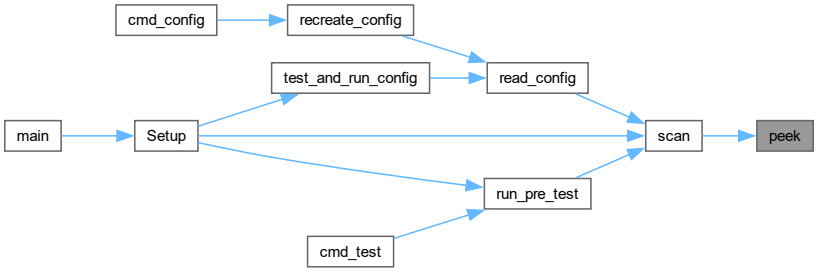
file	FILE pointer to peek from

Returns

Next character, EOF if at end

Definition at line 169 of file load.c.

Here is the caller graph for this function:



7.4.4.11 read_config()

```
HRESULT read_config (
            const char * config_file)
```

```
#include <load.h>
```

Read and apply existing configuration file.

Parameters

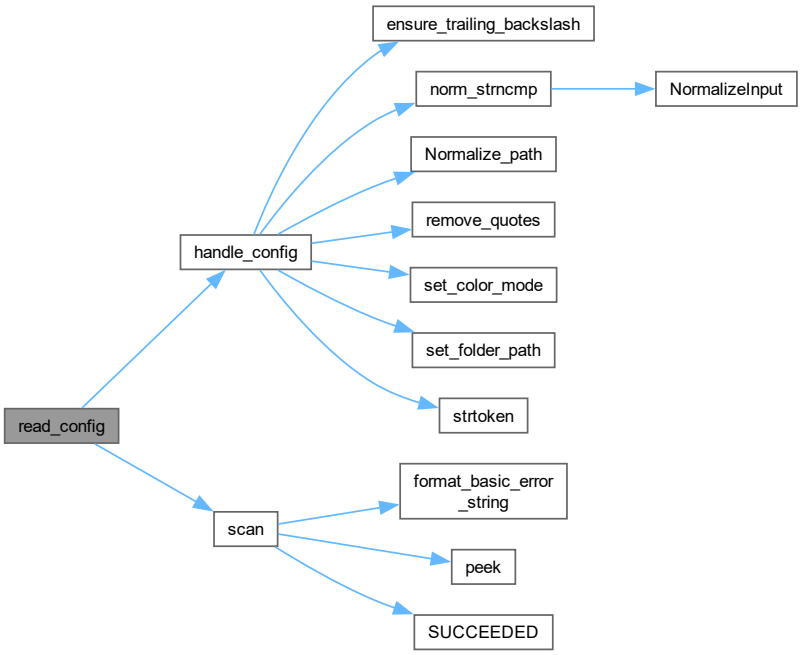
config_file	Configuration file path

Returns

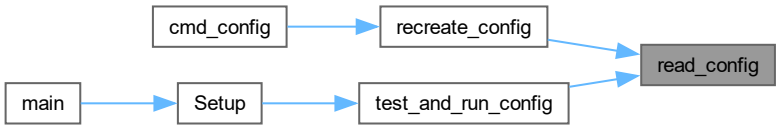
S_OK on success, S_FALSE if file doesn't exist

Definition at line 350 of file load.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.4.4.12 recreate_config()

```
HRESULT recreate_config (
            const char * config_file,
            PROCESS mode)
```

```
#include <load.h>
```

Recreate configuration file with new settings.

Parameters

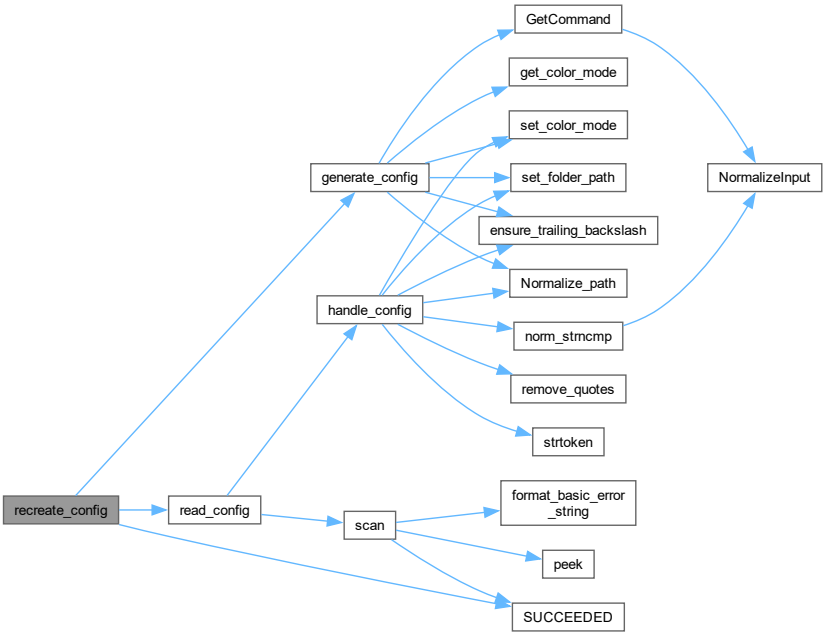
config_file	Configuration file path
mode	Process mode

Returns

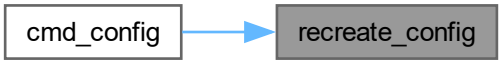
S_OK on success, error code on failure

Definition at line 277 of file load.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.4.4.13 scan()

```
HRESULT scan (
            const char * filename,
            RecordHandler handler,
            void * data,
            PROCESS mode)
```

```
#include <load.h>
```

Parse a file and call handler for each record.

Handles comments (// and * *) and record separation by ';'.

Parameters

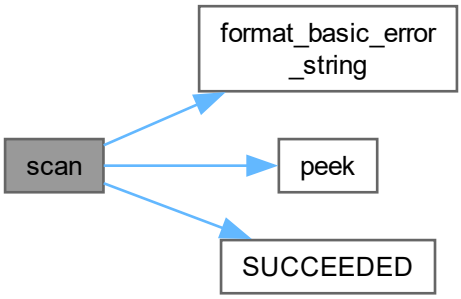
filename	File to parse
handler	Callback function for each record
data	User data passed to handler
mode	Process mode (S_DEBUG enables debugging output)

Returns

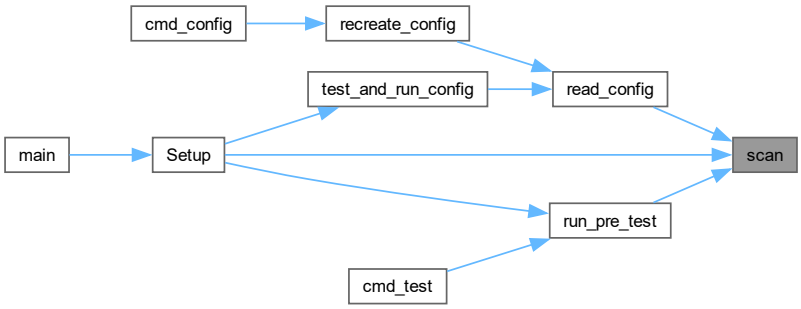
S_OK on success, S_FALSE on file error

Definition at line 46 of file load.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.4.4.14 set_folder_path()

```
void set_folder_path (
            const char * new_path)
```

```
#include <load.h>
```

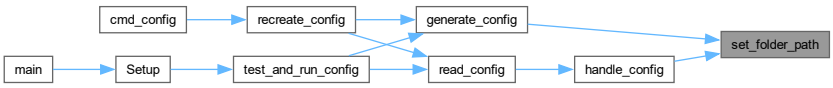
Set the working directory path.

Parameters

new_path	New directory path

Definition at line 20 of file load.c.

Here is the caller graph for this function:



7.4.4.15 test_and_run_config()

```
HRESULT test_and_run_config (
            const char * config_file,
            PROCESS mode)
```

```
#include <load.h>
```

Test configuration and create if missing.

Parameters

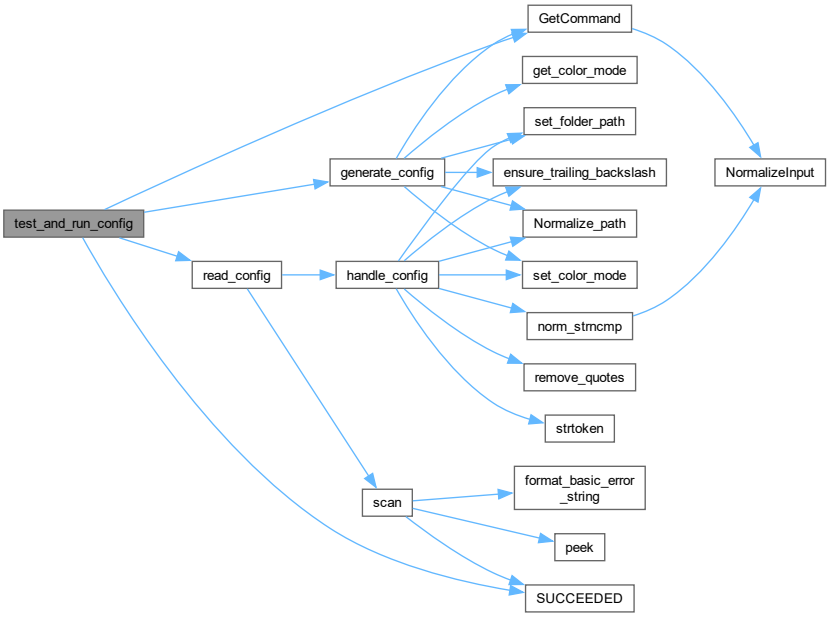
config_file	Configuration file path
mode	Process mode

Returns

S_OK on success, E_FAIL if user declines creation

Definition at line 238 of file load.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.4.5 Variable Documentation

7.4.5.1 folder_path

```
char* folder_path  [extern]
```

```
#include <load.h>
```

Pointer to current working directory path.

Set via configuration file. All data files are loaded relative to this path.

Definition at line 18 of file load.c.

7.5 Main Program

Program entry point and lifecycle management.

Functions

- PROCESS Setup ()

  Program initialization and setup.

7.5.1 Detailed Description

Program entry point and lifecycle management.

7.5.2 Function Documentation

7.5.2.1 Setup()

```
PROCESS Setup ()
```

```
#include <main.h>
```

Program initialization and setup.

Performs all startup tasks:

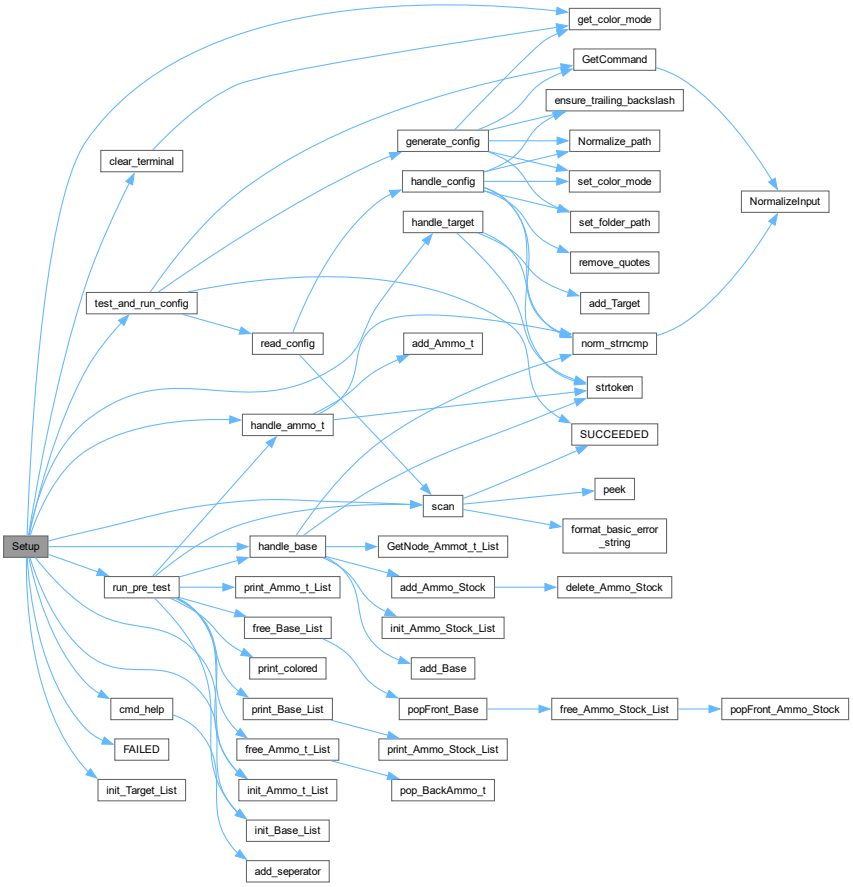
1. Loads or creates configuration
2. Runs system tests
3. Loads data files (ammo, bases, targets)
4. Displays welcome message
5. Shows help

Returns

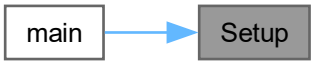
Initial process state (S_RUNNING on success, S_TERMINATE on failure)

Definition at line 21 of file main.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.6 Data Storage Structures

Core data structures and linked list implementations.

Data Structures

- struct Ammo_Stock

  Base ammunition stock entry.
- struct Ammo_Stock_List

  Ammunition stock list for a single base (sorted by ammo ID).
- struct Ammo_t

  Ammunition type definition.
- struct Ammo_t_List

  Global ammunition type list (sorted by ID).
- struct Ammo_t_Node

  Node for global ammunition type linked list.
- struct Base

  Military base structure.
- struct Base_List

  Base list (sorted by ID).
- struct Base_Node

  Node for base linked list.
- struct Coordinate

  Geographic coordinate structure.
- struct Target

  Target structure.
- struct Target_List

  Target list (sorted by ID).
- struct Target_Node

  Node for target linked list.

Typedefs

- typedef struct Ammo_Stock Ammo_Stock

  Base ammunition stock entry.
- typedef struct Ammo_t Ammo_t

  Ammunition type definition.
- typedef struct Ammo_t_Node Ammo_t_Node

  Node for global ammunition type linked list.
- typedef struct Base Base

  Military base structure.
- typedef struct Base_Node Base_Node

  Node for base linked list.
- typedef struct Target Target

  Target structure.
- typedef struct Target_Node Target_Node

  Node for target linked list.

Enumerations

- enum Default_flags {
  F_ID = 0 , F_LABEL = 1 , F_LONGITUDE = 2 , F_LATITUDE = 3 ,
  F_ALTITUDE = 4 }

  Default data field flags for parsing.
- enum InputFlags { T_ID , T_LABEL , T_DATA , T_WHITESPACE }

  Input parsing flags.
- enum RangeType {
  RANGE_NONE = 0 , RANGE_EUROPE , RANGE_ASIA , RANGE_AMERICAS ,
  RANGE_AFRICA , RANGE_OCEANIA , RANGE_ANTARCTICA }

  Regional categorization based on ID ranges.

Variables

- double altitude

  Altitude in meters above WGS84 ellipsoid.
- Ammo_t_List * ammo_t_list

  Global ammunition type database.
- Ammo_t * ammoItem

  Pointer to ammunition data.
- Ammo_Stock_List * ammoStock

  Inventory of ammunition types.
- double area

  Target area in square kilometers.
- double ballistic_coefficient

  G7 ballistic coefficient (kg/m^2).
- Base_List * base_list

  All military bases.
- Base * baseItem

  Pointer to base data.
- double blastRadius

  Explosive effect radius in meters.
- Coordinate coord

  Geographic coordinates.
- Coordinate coord

  Geographic coordinates.
- double diameter

  Caliber in millimeters.
- Ammo_Stock * head

  Sentinel head node (typePtr = NULL).
- Ammo_t_Node * head

  Sentinel head node (ammoItem = NULL).
- Base_Node * head

  Sentinel head node (baseItem = NULL).
- Target_Node * head

  Sentinel head node (targetItem = NULL).
- unsigned id

  Unique identifier.
- unsigned id

  Unique identifier.

- unsigned id

  Unique identifier.
- char * label

  Dynamically allocated base name.
- char * label

  Dynamically allocated target name.
- double latitude

  Latitude in degrees (-90 to 90).
- double longitude

  Longitude in degrees (-180 to 180).
- double muzzleVelocity

  Initial velocity in m/s.
- char * name

  Dynamically allocated name string.
- struct Ammo_Stock * next

  Next stock entry in list.
- struct Ammo_t_Node * next

  Next node in list.
- struct Base_Node * next

  Next base in list.
- struct Target_Node * next

  Next target in list.
- struct Ammo_t_Node * prev

  Previous node in list.
- unsigned quantity

  Available quantity at base.
- Ammo_Stock * tail

  Sentinel tail node (typePtr = NULL).
- Ammo_t_Node * tail

  Sentinel tail node (ammoItem = NULL).
- Base_Node * tail

  Sentinel tail node (baseItem = NULL).
- Target_Node * tail

  Sentinel tail node (targetItem = NULL).
- Target_List * target_list

  All targets.
- Target * targetItem

  Pointer to target data.
- Ammo_t * typePtr

  Pointer to global ammunition type.
- double weight

  Mass in kilograms.

Ammunition List Functions

- HRESULT init_Ammo_t_List (Ammo_t_List **list)

  Initialize an empty ammunition type list.
- Ammo_t_Node * add_Ammo_t (Ammo_t_List *list, Ammo_t ammo)

  Add ammunition type to list (sorted by ID).
- int delete_Ammo_t (Ammo_t_List *list, unsigned id)

Delete ammunition type by ID.

- Ammo_t_Node popFront_Ammo_t (Ammo_t_List *list)

  Remove and return first ammunition from list.
- Ammo_t_Node popBack_Ammo_t (Ammo_t_List *list)

  Remove and return last ammunition from list.
- void print_Ammo_t_List (Ammo_t_List *list)

  Print all ammunition types to stdout.
- HRESULT free_Ammo_t_List (Ammo_t_List **list)

  Free all memory associated with ammunition list.
- Ammo_t * FindAmmoByID (Ammo_t_List *list, unsigned id)

  Find ammunition by ID.

Base List Functions

- HRESULT init_Base_List (Base_List **list)

  Initialize an empty base list.
- Base_Node * add_Base (Base_List *list, Base base)

  Add base to list (sorted by ID).
- int delete_Base (Base_List *list, unsigned id)

  Delete base by ID.
- Base_Node popFront_Base (Base_List *list)

  Remove and return first base from list.
- Base_Node popBack_Base (Base_List *list)

  Remove and return last base from list.
- void print_Base_List (Base_List *list, PROCESS mode)

  Print all bases to stdout.
- Ammo_t * GetNode_Ammot_t_List (Ammo_t_List *list, unsigned id)

  Get ammunition type by ID from ammunition list.
- HRESULT free_Base_List (Base_List **list)

  Free all memory associated with base list.
- Base * FindBaseByID (Base_List *list, unsigned id)

  Find base by ID.

Ammunition Stock List Functions

- HRESULT init_Ammo_Stock_List (Ammo_Stock_List **list)

  Initialize an empty ammunition stock list.
- Ammo_Stock * add_Ammo_Stock (Ammo_Stock_List *list, Ammo_t *ammoRef, unsigned quantity)

  Add ammunition to base stock (sorted by ammo ID).
- int delete_Ammo_Stock (Ammo_Stock_List *list, Ammo_t *ammoRef)

  Delete ammunition from stock by reference.
- Ammo_Stock popFront_Ammo_Stock (Ammo_Stock_List *list)

  Remove and return first ammunition from stock.
- Ammo_Stock popBack_Ammo_Stock (Ammo_Stock_List *list)

  Remove and return last ammunition from stock.
- void print_Ammo_Stock_List (Ammo_Stock_List *list)

  Print ammunition stock to stdout.
- HRESULT free_Ammo_Stock_List (Ammo_Stock_List **list)

  Free all memory associated with ammunition stock list.

Target List Functions

- HRESULT init_Target_List (Target_List **list)

  Initialize an empty target list.
- Target_Node * add_Target (Target_List *list, Target target)

  Add target to list (sorted by ID).
- int delete_Target (Target_List *list, unsigned id)

  Delete target by ID.
- Target_Node popFront_Target (Target_List *list)

  Remove and return first target from list.
- Target_Node popBack_Target (Target_List *list)

  Remove and return last target from list.
- void print_Target_List (Target_List *list)

  Print all targets to stdout.
- HRESULT free_Target_List (Target_List **list)

  Free all memory associated with target list.
- Target * FindTargetByID (Target_List *list, unsigned id)

  Find target by ID.

File I/O Functions

- unsigned GetAmmoID (const Ammo_t *a)

  Get ammunition ID (safe wrapper).
- void Save_BaseList_file (FILE *fp)

  Save base list to open file.
- HRESULT Save_BaseList (Base_List *list, char *path)

  Save base list to file path.
- void Save_TargetList_file (FILE *fp)

  Save target list to open file.
- HRESULT Save_TargetList (Target_List *list, char *path)

  Save target list to file path.
- void Save_Ammo_t_List_file (FILE *fp)

  Save ammunition list to open file.
- HRESULT Save_Ammo_t_List (Ammo_t_List *list, char *path)

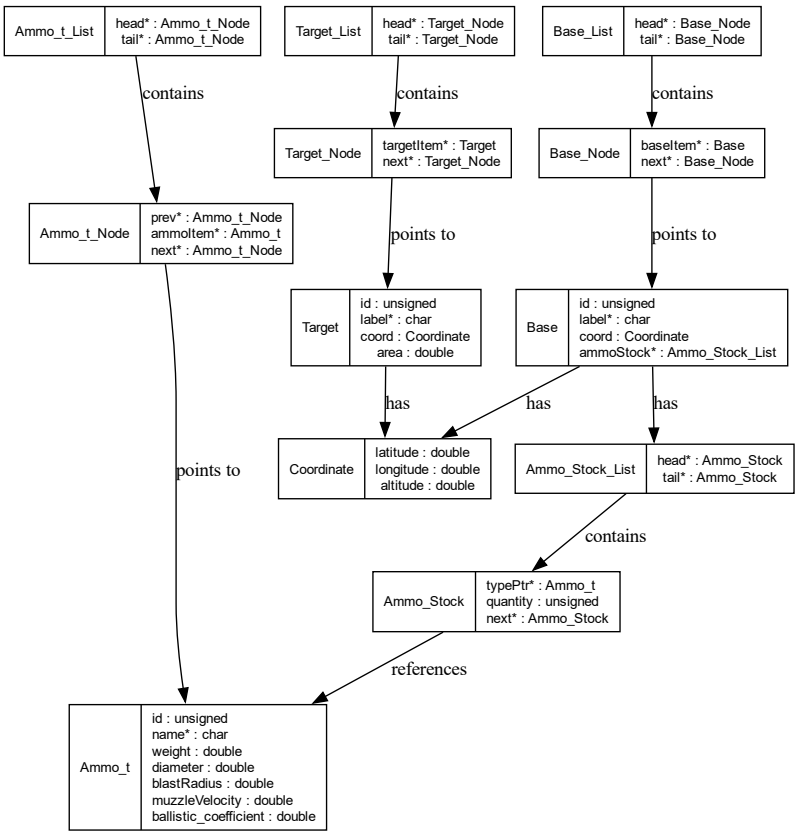
  Save ammunition list to file path.

7.6.1 Detailed Description

Core data structures and linked list implementations.

This module contains all data structures for storing bases, targets, ammunition types, and their relationships. It implements sentinel-linked lists with head and tail nodes for efficient insertion and traversal.

7.6.1.1 Data Structure Hierarchy



7.6.1.2 Data File Formats

7.6.1.2.1 Ammunition Catalog (ammo_catalouge.txt)

```
I0   NHE   W50.0  D120.0   BR10.0   MV800.0   BC0.5;
I1   NAP   W40.0  D105.0   BR8.0    MV850.0   BC0.48;
```

7.6.1.2.2 Bases (bases.txt)

```
I0   LBase1   CLA47.4729283  CLO19.0531429   CA100.0  AMS:{0:100,1:50};
I1   LBase2   CLA48.123456   CLO20.987654    CA150.0  AMS:{0:75};
```

7.6.1.2.3 Targets (targets.txt)

```
I0   LTarget1   CLA47.5  CLO19.1   CA50.0  A1.5;
I1   LTarget2   CLA48.0  CLO20.0   CA75.0  A2.0;
```

7.6.1.3 Memory Management

All dynamically allocated strings (name, label) are freed automatically when lists are destroyed. The system uses sentinel nodes (head/tail) to simplify boundary conditions in list operations.

7.6.2 Typedef Documentation

7.6.2.1 Ammo_Stock

```
typedef struct Ammo_Stock Ammo_Stock
```

```
#include <storage.h>
```

Base ammunition stock entry.

Links to global ammunition type and stores quantity.

7.6.2.2 Ammo_t

```
typedef struct Ammo_t Ammo_t
```

```
#include <storage.h>
```

Ammunition type definition.

Contains physical and ballistic properties of ammunition types. These are globally defined and referenced by bases' stock lists.

7.6.2.3 Ammo_t_Node

```
typedef struct Ammo_t_Node Ammo_t_Node
```

```
#include <storage.h>
```

Node for global ammunition type linked list.

7.6.2.4 Base

```
typedef struct Base Base
```

```
#include <storage.h>
```

Military base structure.

Contains base identification, location, and ammunition inventory.

7.6.2.5 Base_Node

```
typedef struct Base_Node Base_Node
```

```
#include <storage.h>
```

Node for base linked list.

7.6.2.6 Target

```
typedef struct Target Target
```

```
#include <storage.h>
```

Target structure.

Contains target identification, location, and area.

7.6.2.7 Target_Node

```
typedef struct Target_Node Target_Node
```

```
#include <storage.h>
```

Node for target linked list.

7.6.3 Enumeration Type Documentation

7.6.3.1 Default_flags

```
enum Default_flags
```

```
#include <storage.h>
```

Default data field flags for parsing.

Enumerator

F_ID	ID field flag.
F_LABEL	Label field flag.
F_LONGITUDE	Longitude field flag.
F_LATITUDE	Latitude field flag.
F_ALTITUDE	Altitude field flag.

Definition at line 650 of file storage.h.

7.6.3.2 InputFlags

```
enum InputFlags
```

```
#include <storage.h>
```

Input parsing flags.

Enumerator

T_ID	Identifier token.
T_LABEL	Label/name token.
T_DATA	Data value token.
T_WHITESPACE	Whitespace token.

Definition at line 638 of file storage.h.

7.6.3.3 RangeType

```
enum RangeType
```

```
#include <storage.h>
```

Regional categorization based on ID ranges.

Used for organizing entities geographically and for region-aware base selection algorithms.

Enumerator

RANGE_NONE	No region assigned.
RANGE_EUROPE	IDs 0-999.
RANGE_ASIA	IDs 1000-1999.
RANGE_AMERICAS	IDs 2000-2999.
RANGE_AFRICA	IDs 3000-3999.
RANGE_OCEANIA	IDs 4000-4999.
RANGE_ANTARCTICA	IDs 5000+.

Definition at line 94 of file storage.h.

7.6.4 Function Documentation

7.6.4.1 add_Ammo_Stock()

```
Ammo_Stock * add_Ammo_Stock (
             Ammo_Stock_List * list,
             Ammo_t * ammoRef,
             unsigned quantity)
```

```
#include <storage.h>
```

Add ammunition to base stock (sorted by ammo ID).

Parameters

list	Pointer to stock list

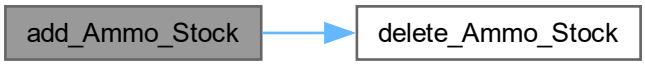
ammoRef	Pointer to global ammunition type
quantity	Amount to add

Returns

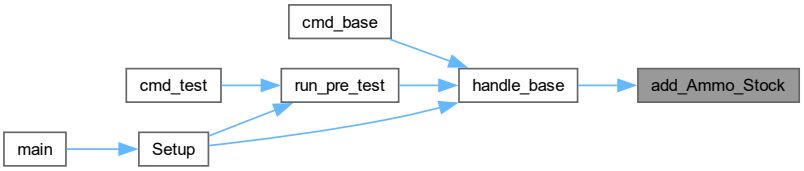
Pointer to new stock entry, NULL on error

Definition at line 435 of file storage.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.6.4.2 add_Ammo_t()

```
Ammo_t_Node * add_Ammo_t (
              Ammo_t_List * list,
              Ammo_t ammo)
```

```
#include <storage.h>
```

Add ammunition type to list (sorted by ID).

Parameters

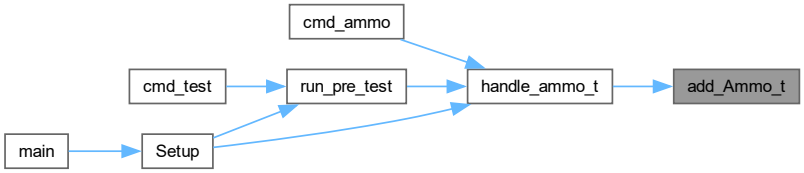
list	Pointer to ammunition list
ammo	Ammunition data to add

Returns

Pointer to new node, NULL on error

Definition at line 37 of file storage.c.

Here is the caller graph for this function:



7.6.4.3 add_Base()

```
Base_Node * add_Base (
            Base_List * list,
            Base base)
```

```
#include <storage.h>
```

Add base to list (sorted by ID).

Parameters

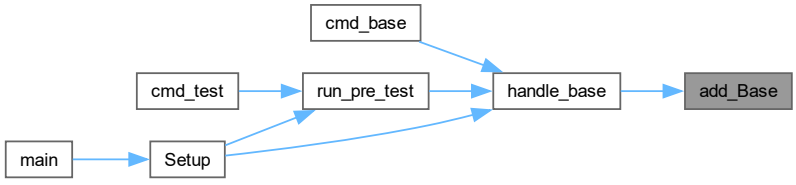
list	Pointer to base list
base	Base data to add

Returns

Pointer to new node, NULL on error

Definition at line 248 of file storage.c.

Here is the caller graph for this function:



7.6.4.4 add_Target()

```
Target_Node * add_Target (
            Target_List * list,
            Target target)
```

```
#include <storage.h>
```

Add target to list (sorted by ID).

Parameters

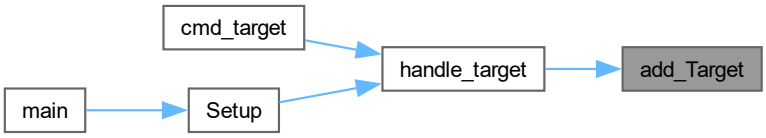
list	Pointer to target list
target	Target data to add

Returns

Pointer to new node, NULL on error

Definition at line 598 of file storage.c.

Here is the caller graph for this function:



7.6.4.5 delete_Ammo_Stock()

```
int delete_Ammo_Stock (
            Ammo_Stock_List * list,
            Ammo_t * ammoRef)
```

```
#include <storage.h>
```

Delete ammunition from stock by reference.

Parameters

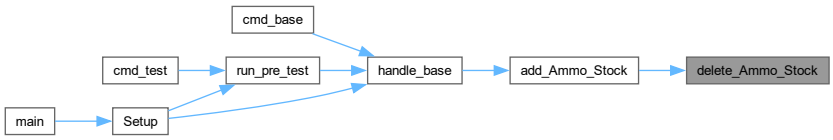
list	Pointer to stock list
ammoRef	Pointer to ammunition type to remove

Returns

Ammunition ID if found and deleted, -1 if not found

Definition at line 461 of file storage.c.

Here is the caller graph for this function:



7.6.4.6 delete_Ammo_t()

```
int delete_Ammo_t (
            Ammo_t_List * list,
            unsigned id)
```

```
#include <storage.h>
```

Delete ammunition type by ID.

Parameters

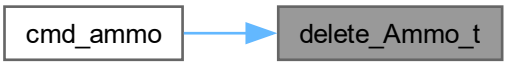
list	Pointer to ammunition list
id	ID of ammunition to delete

Returns

ID if found and deleted, -1 if not found

Definition at line 79 of file storage.c.

Here is the caller graph for this function:



7.6.4.7 delete_Base()

```
int delete_Base (
            Base_List * list,
            unsigned id)
```

```
#include <storage.h>
```

Delete base by ID.

Parameters

list	Pointer to base list
id	ID of base to delete

Returns

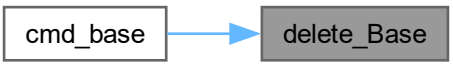
ID if found and deleted, -1 if not found

Definition at line 298 of file storage.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.6.4.8 delete_Target()

```
int delete_Target (
            Target_List * list,
            unsigned id)
```

```
#include <storage.h>
```

Delete target by ID.

Parameters

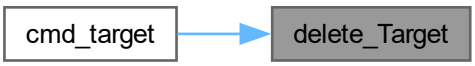
list	Pointer to target list
id	ID of target to delete

Returns

ID if found and deleted, -1 if not found

Definition at line 632 of file storage.c.

Here is the caller graph for this function:



7.6.4.9 FindAmmoByID()

```
Ammo_t * FindAmmoByID (
            Ammo_t_List * list,
            unsigned id)
```

```
#include <storage.h>
```

Find ammunition by ID.

Parameters

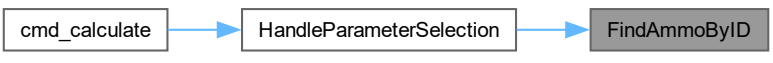
list	Pointer to ammunition list
id	ID to search for

Returns

Pointer to ammunition, NULL if not found

Definition at line 199 of file storage.c.

Here is the caller graph for this function:



7.6.4.10 FindBaseByID()

```
Base * FindBaseByID (
            Base_List * list,
            unsigned id)
```

```
#include <storage.h>
```

Find base by ID.

Parameters

list	Pointer to base list
id	ID to search for

Returns

Pointer to base, NULL if not found

Definition at line 392 of file storage.c.

Here is the caller graph for this function:



7.6.4.11 FindTargetByID()

```
Target * FindTargetByID (
            Target_List * list,
            unsigned id)
```

```
#include <storage.h>
```

Find target by ID.

Parameters

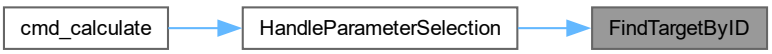
list	Pointer to target list
id	ID to search for

Returns

Pointer to target, NULL if not found

Definition at line 722 of file storage.c.

Here is the caller graph for this function:



7.6.4.12 free_Ammo_Stock_List()

```
HRESULT free_Ammo_Stock_List (
            Ammo_Stock_List ** list)
```

```
#include <storage.h>
```

Free all memory associated with ammunition stock list.

Parameters

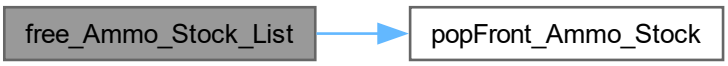
list	Double pointer to stock list

Returns

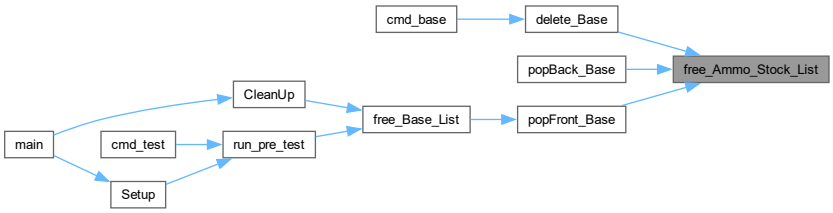
S_OK on success, S_FALSE if list was NULL

Definition at line 559 of file storage.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.6.4.13 free_Ammo_t_List()

```
HRESULT free_Ammo_t_List (
            Ammo_t_List ** list)
```

```
#include <storage.h>
```

Free all memory associated with ammunition list.

Parameters

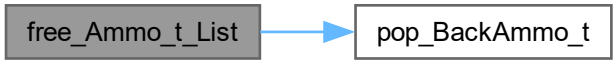
list	Double pointer to ammunition list

Returns

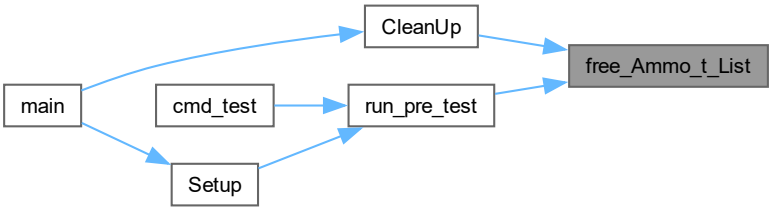
S_OK on success, S_FALSE if list was NULL

Definition at line 183 of file storage.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.6.4.14 free_Base_List()

```
HRESULT free_Base_List (
            Base_List ** list)
```

```
#include <storage.h>
```

Free all memory associated with base list.

Parameters

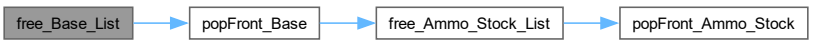
list	Double pointer to base list

Returns

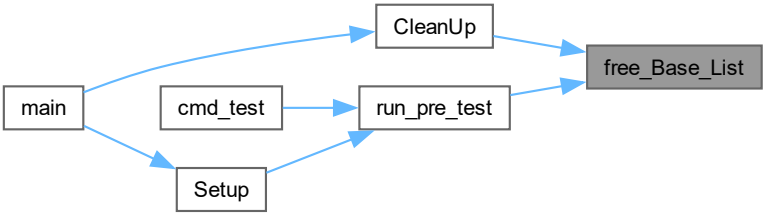
S_OK on success, S_FALSE if list was NULL

Definition at line 376 of file storage.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.6.4.15 free_Target_List()

```
HRESULT free_Target_List (
            Target_List ** list)
```

```
#include <storage.h>
```

Free all memory associated with target list.

Parameters

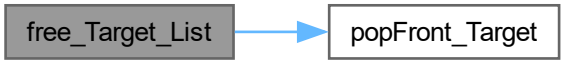
list	Double pointer to target list

Returns

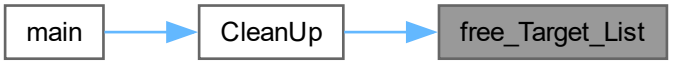
S_OK on success, S_FALSE if list was NULL

Definition at line 706 of file storage.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.6.4.16 GetAmmoID()

```
unsigned GetAmmoID (
            const Ammo_t * a)
```

```
#include <storage.h>
```

Get ammunition ID (safe wrapper).

Parameters

a	Pointer to ammunition

Returns

ID if ammunition exists, 0 otherwise

Definition at line 786 of file storage.c.

Here is the caller graph for this function:



7.6.4.17 GetNode_Ammot_t_List()

```
Ammo_t * GetNode_Ammot_t_List (
            Ammo_t_List * list,
            unsigned id)
```

```
#include <storage.h>
```

Get ammunition type by ID from ammunition list.

Parameters

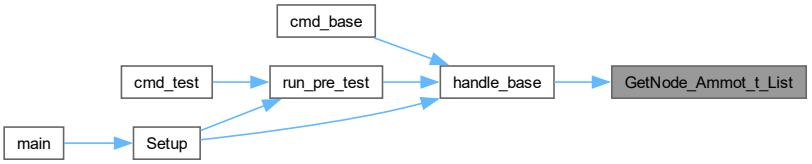
list	Pointer to ammunition list
id	ID to search for

Returns

Pointer to ammunition, NULL if not found

Definition at line 138 of file storage.c.

Here is the caller graph for this function:



7.6.4.18 init_Ammo_Stock_List()

```
HRESULT init_Ammo_Stock_List (
            Ammo_Stock_List ** list)
```

```
#include <storage.h>
```

Initialize an empty ammunition stock list.

Parameters

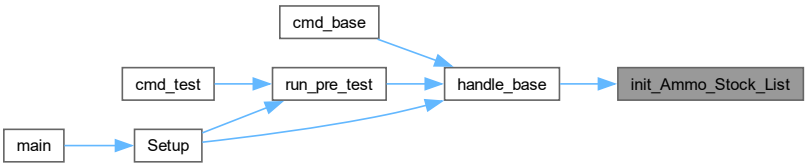
list	Double pointer to Ammo_Stock_List

Returns

S_OK on success, S_FALSE if list already initialized

Definition at line 412 of file storage.c.

Here is the caller graph for this function:



7.6.4.19 init_Ammo_t_List()

```
HRESULT init_Ammo_t_List (
            Ammo_t_List ** list)
```

```
#include <storage.h>
```

Initialize an empty ammunition type list.

Parameters

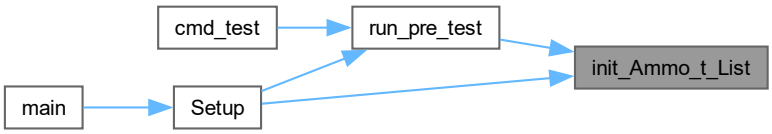
list	Double pointer to Ammo_t_List

Returns

S_OK on success, S_FALSE if list already initialized

Definition at line 17 of file storage.c.

Here is the caller graph for this function:



7.6.4.20 init_Base_List()

```
HRESULT init_Base_List (
            Base_List ** list)
```

```
#include <storage.h>
```

Initialize an empty base list.

Parameters

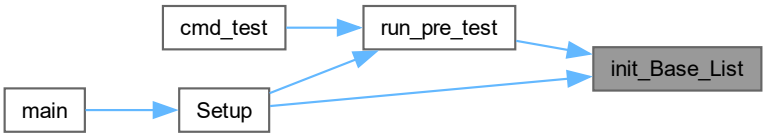
list	Double pointer to Base_List

Returns

S_OK on success, S_FALSE if list already initialized

Definition at line 230 of file storage.c.

Here is the caller graph for this function:



7.6.4.21 init_Target_List()

```
HRESULT init_Target_List (
            Target_List ** list)
```

```
#include <storage.h>
```

Initialize an empty target list.

Parameters

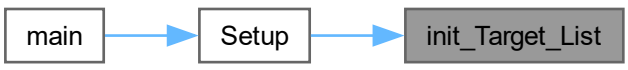
list	Double pointer to Target_List

Returns

S_OK on success, S_FALSE if list already initialized

Definition at line 580 of file storage.c.

Here is the caller graph for this function:



7.6.4.22 popBack_Ammo_Stock()

```
Ammo_Stock popBack_Ammo_Stock (
            Ammo_Stock_List * list)
```

```
#include <storage.h>
```

Remove and return last ammunition from stock.

Parameters

list	Pointer to stock list

Returns

Copy of removed stock entry (empty if list was empty)

Definition at line 509 of file storage.c.

7.6.4.23 popBack_Ammo_t()

```
Ammo_t_Node popBack_Ammo_t (
            Ammo_t_List * list)
```

```
#include <storage.h>
```

Remove and return last ammunition from list.

Parameters

list	Pointer to ammunition list

Returns

Copy of removed node (empty if list was empty)

7.6.4.24 popBack_Base()

```
Base_Node popBack_Base (
            Base_List * list)
```

```
#include <storage.h>
```

Remove and return last base from list.

Parameters

list	Pointer to base list

Returns

Copy of removed node (empty if list was empty)

Definition at line 339 of file storage.c.

Here is the call graph for this function:



7.6.4.25 popBack_Target()

```
Target_Node popBack_Target (
            Target_List * list)
```

```
#include <storage.h>
```

Remove and return last target from list.

Parameters

list	Pointer to target list

Returns

Copy of removed node (empty if list was empty)

Definition at line 671 of file storage.c.

7.6.4.26 popFront_Ammo_Stock()

```
Ammo_Stock popFront_Ammo_Stock (
            Ammo_Stock_List * list)
```

```
#include <storage.h>
```

Remove and return first ammunition from stock.

Parameters

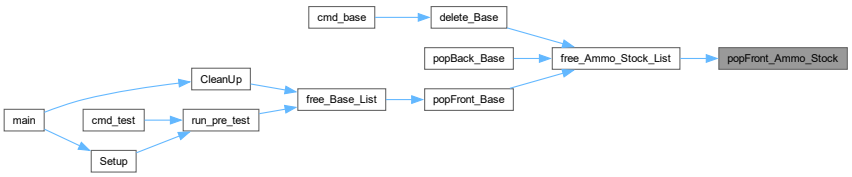
list	Pointer to stock list

Returns

Copy of removed stock entry (empty if list was empty)

Definition at line 485 of file storage.c.

Here is the caller graph for this function:



7.6.4.27 popFront_Ammo_t()

```
Ammo_t_Node popFront_Ammo_t (
            Ammo_t_List * list)
```

```
#include <storage.h>
```

Remove and return first ammunition from list.

Parameters

list	Pointer to ammunition list

Returns

Copy of removed node (empty if list was empty)

7.6.4.28 popFront_Base()

```
Base_Node popFront_Base (
            Base_List * list)
```

```
#include <storage.h>
```

Remove and return first base from list.

Parameters

list	Pointer to base list

Returns

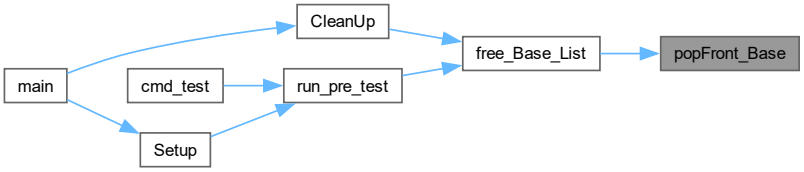
Copy of removed node (empty if list was empty)

Definition at line 324 of file storage.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.6.4.29 popFront_Target()

```
Target_Node popFront_Target (
            Target_List * list)
```

```
#include <storage.h>
```

Remove and return first target from list.

Parameters

list	Pointer to target list

Returns

Copy of removed node (empty if list was empty)

Definition at line 657 of file storage.c.

Here is the caller graph for this function:



7.6.4.30 print_Ammo_Stock_List()

```
void print_Ammo_Stock_List (
            Ammo_Stock_List * list)
```

```
#include <storage.h>
```

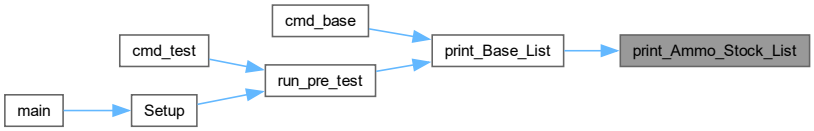
Print ammunition stock to stdout.

Parameters

list	Pointer to stock list

Definition at line 539 of file storage.c.

Here is the caller graph for this function:



7.6.4.31 print_Ammo_t_List()

```
void print_Ammo_t_List (
            Ammo_t_List * list)
```

```
#include <storage.h>
```

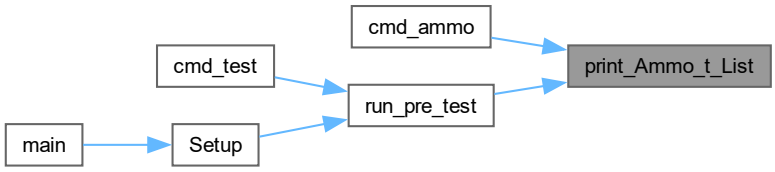
Print all ammunition types to stdout.

Parameters

list	Pointer to ammunition list

Definition at line 168 of file storage.c.

Here is the caller graph for this function:



7.6.4.32 print_Base_List()

```
void print_Base_List (
            Base_List * list,
            PROCESS mode)
```

```
#include <storage.h>
```

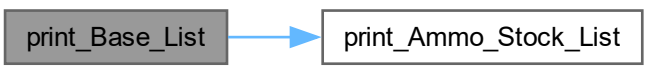
Print all bases to stdout.

Parameters

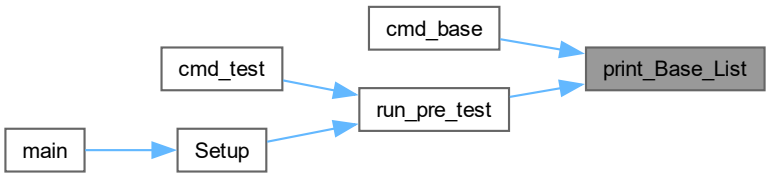
list	Pointer to base list
mode	Process mode (S_DEBUG shows ammunition stock details)

Definition at line 357 of file storage.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.6.4.33 print_Target_List()

```
void print_Target_List (
            Target_List * list)
```

```
#include <storage.h>
```

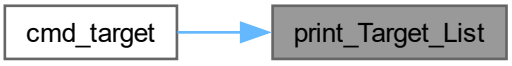
Print all targets to stdout.

Parameters

list	Pointer to target list

Definition at line 689 of file storage.c.

Here is the caller graph for this function:



7.6.4.34 Save_Ammo_t_List()

```
HRESULT Save_Ammo_t_List (
            Ammo_t_List * list,
            char * path)
```

```
#include <storage.h>
```

Save ammunition list to file path.

Parameters

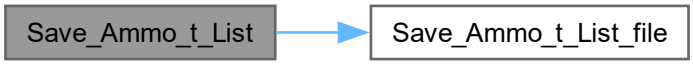
list	Pointer to ammunition list
path	File path

Returns

S_OK on success, E_FAIL on file error

Definition at line 771 of file storage.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.6.4.35 Save_Ammo_t_List_file()

```
void Save_Ammo_t_List_file (
            FILE * fp)
```

```
#include <storage.h>
```

Save ammunition list to open file.

Parameters

fp	Open FILE pointer for writing

Definition at line 919 of file storage.c.

Here is the caller graph for this function:



7.6.4.36 Save_BaseList()

```
HRESULT Save_BaseList (
            Base_List * list,
            char * path)
```

```
#include <storage.h>
```

Save base list to file path.

Parameters

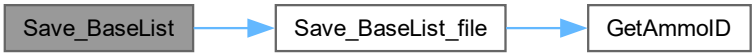
list	Pointer to base list
path	File path

Returns

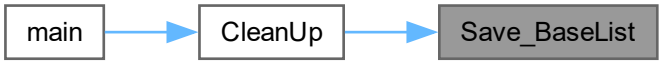
S_OK on success, E_FAIL on file error

Definition at line 741 of file storage.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.6.4.37 Save_BaseList_file()

```
void Save_BaseList_file (
            FILE * fp)
```

```
#include <storage.h>
```

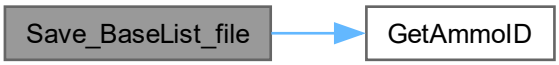
Save base list to open file.

Parameters

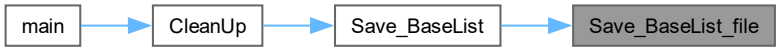
fp	Open FILE pointer for writing

Definition at line 791 of file storage.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.6.4.38 Save_TargetList()

```
HRESULT Save_TargetList (
            Target_List * list,
            char * path)
```

```
#include <storage.h>
```

Save target list to file path.

Parameters

list	Pointer to target list

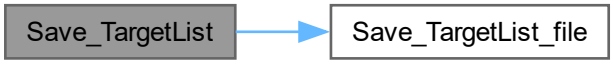
path	File path

Returns

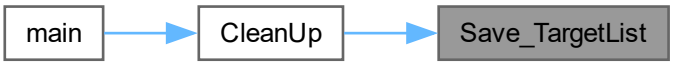
S_OK on success, E_FAIL on file error

Definition at line 756 of file storage.c.

Here is the call graph for this function:



Here is the caller graph for this function:



7.6.4.39 Save_TargetList_file()

```
void Save_TargetList_file (
            FILE * fp)
```

```
#include <storage.h>
```

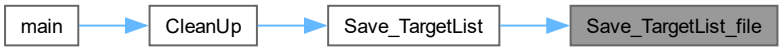
Save target list to open file.

Parameters

fp	Open FILE pointer for writing

Definition at line 864 of file storage.c.

Here is the caller graph for this function:



7.6.5 Variable Documentation

7.6.5.1 altitude

```
double altitude
```

Altitude in meters above WGS84 ellipsoid.

Definition at line 137 of file storage.h.

7.6.5.2 ammo_t_list

```
Ammo_t_List* ammo_t_list  [extern]
```

```
#include <storage.h>
```

Global ammunition type database.

Definition at line 12 of file storage.c.

7.6.5.3 ammoItem

```
Ammo_t* ammoItem
```

Pointer to ammunition data.

Definition at line 164 of file storage.h.

7.6.5.4 ammoStock

```
Ammo_Stock_List* ammoStock
```

Inventory of ammunition types.

Definition at line 215 of file storage.h.

7.6.5.5 area

```
double area
```

Target area in square kilometers.

Definition at line 249 of file storage.h.

7.6.5.6 ballistic_coefficient

```
double ballistic_coefficient
```

G7 ballistic coefficient (kg/m^2).

Definition at line 155 of file storage.h.

7.6.5.7 base_list

```
Base_List* base_list  [extern]
```

```
#include <storage.h>
```

All military bases.

Definition at line 13 of file storage.c.

7.6.5.8 baseItem

```
Base* baseItem
```

Pointer to base data.

Definition at line 224 of file storage.h.

7.6.5.9 blastRadius

```
double blastRadius
```

Explosive effect radius in meters.

Definition at line 153 of file storage.h.

7.6.5.10 coord [1/2]

```
Coordinate coord
```

Geographic coordinates.

Definition at line 214 of file storage.h.

7.6.5.11 coord [2/2]

```
Coordinate coord
```

Geographic coordinates.

Definition at line 248 of file storage.h.

7.6.5.12 diameter

```
double diameter
```

Caliber in millimeters.

Definition at line 152 of file storage.h.

7.6.5.13 head [1/4]

```
Ammo_Stock* head
```

Sentinel head node (typePtr = NULL).

Definition at line 200 of file storage.h.

7.6.5.14 head [2/4]

```
Ammo_t_Node* head
```

Sentinel head node (ammoItem = NULL).

Definition at line 177 of file storage.h.

7.6.5.15 head [3/4]

```
Base_Node* head
```

Sentinel head node (baseItem = NULL).

Definition at line 234 of file storage.h.

7.6.5.16 head [4/4]

```
Target_Node* head
```

Sentinel head node (targetItem = NULL).

Definition at line 268 of file storage.h.

7.6.5.17 id [1/3]

```
unsigned id
```

Unique identifier.

Definition at line 149 of file storage.h.

7.6.5.18 id [2/3]

```
unsigned id
```

Unique identifier.

Definition at line 212 of file storage.h.

7.6.5.19 id [3/3]

```
unsigned id
```

Unique identifier.

Definition at line 246 of file storage.h.

7.6.5.20 label [1/2]

```
char* label
```

Dynamically allocated base name.

Definition at line 213 of file storage.h.

7.6.5.21 label [2/2]

```
char* label
```

Dynamically allocated target name.

Definition at line 247 of file storage.h.

7.6.5.22 latitude

```
double latitude
```

Latitude in degrees (-90 to 90).

Definition at line 135 of file storage.h.

7.6.5.23 longitude

```
double longitude
```

Longitude in degrees (-180 to 180).

Definition at line 136 of file storage.h.

7.6.5.24 muzzleVelocity

```
double muzzleVelocity
```

Initial velocity in m/s.

Definition at line 154 of file storage.h.

7.6.5.25 name

```
char* name
```

Dynamically allocated name string.

Definition at line 150 of file storage.h.

7.6.5.26 next [1/4]

```
struct Ammo_Stock* next
```

Next stock entry in list.

Definition at line 191 of file storage.h.

7.6.5.27 next [2/4]

```
struct Ammo_t_Node* next
```

Next node in list.

Definition at line 166 of file storage.h.

7.6.5.28 next [3/4]

```
struct Base_Node* next
```

Next base in list.

Definition at line 225 of file storage.h.

7.6.5.29 next [4/4]

```
struct Target_Node* next
```

Next target in list.

Definition at line 259 of file storage.h.

7.6.5.30 prev

```
struct Ammo_t_Node* prev
```

Previous node in list.

Definition at line 165 of file storage.h.

7.6.5.31 quantity

```
unsigned quantity
```

Available quantity at base.

Definition at line 190 of file storage.h.

7.6.5.32 tail [1/4]

```
Ammo_Stock* tail
```

Sentinel tail node (typePtr = NULL).

Definition at line 201 of file storage.h.

7.6.5.33 tail [2/4]

```
Ammo_t_Node* tail
```

Sentinel tail node (ammoItem = NULL).

Definition at line 178 of file storage.h.

7.6.5.34 tail [3/4]

```
Base_Node* tail
```

Sentinel tail node (baseItem = NULL).

Definition at line 235 of file storage.h.

7.6.5.35 tail [4/4]

```
Target_Node* tail
```

Sentinel tail node (targetItem = NULL).

Definition at line 269 of file storage.h.

7.6.5.36 target_list

```
Target_List* target_list  [extern]
```

```
#include <storage.h>
```

All targets.

Definition at line 14 of file storage.c.

7.6.5.37 targetItem

```
Target* targetItem
```

Pointer to target data.

Definition at line 258 of file storage.h.

7.6.5.38 typePtr

```
Ammo_t* typePtr
```

Pointer to global ammunition type.

Definition at line 189 of file storage.h.

7.6.5.39 weight

```
double weight
```

Mass in kilograms.

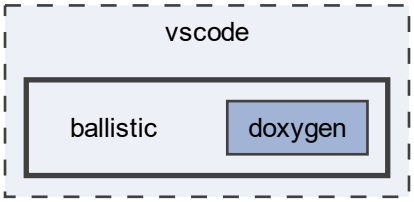
Definition at line 151 of file storage.h.

Chapter 8

Directory Documentation

8.1 C:/vscode/ballistic Directory Reference

Directory dependency graph for ballistic:



Directories

- directory doxygen

8.2 C:/vscode/ballistic/doxygen Directory Reference

Directory dependency graph for doxygen:



Files

- file calculations.c

  Implementation of ballistic calculation functions.
- file calculations.h
- file color.c
- file color.h
- file commands.c
- file commands.h
- file debug.c
- file debug.h
- file debugmalloc.h
- file doxygen_main.h
- file error.c
- file error.h

  Error codes and process states.
- file helpers.c
- file helpers.h
- file load.c
- file load.h
- file main.c
- file main.h

  Main program declarations.
- file storage.c
- file storage.h

8.3 C:/vscode Directory Reference

Directories

- directory ballistic

Chapter 9

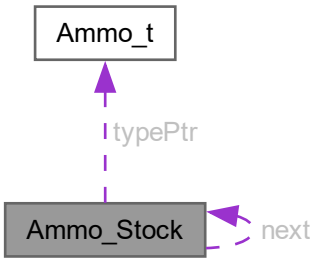
Data Structure Documentation

9.1 Ammo_Stock Struct Reference

Base ammunition stock entry.

`#include <storage.h>`

Collaboration diagram for Ammo_Stock:



Data Fields

- struct Ammo_Stock * next
  Next stock entry in list.
- unsigned quantity
  Available quantity at base.
- Ammo_t * typePtr
  Pointer to global ammunition type.

9.1.1 Detailed Description

Base ammunition stock entry.

Links to global ammunition type and stores quantity.

Definition at line 187 of file storage.h.

The documentation for this struct was generated from the following file:

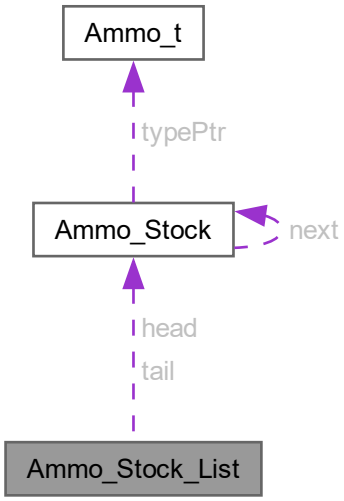
- C:/vscode/ballistic/doxygen/storage.h

9.2 Ammo_Stock_List Struct Reference

Ammunition stock list for a single base (sorted by ammo ID).

`#include <storage.h>`

Collaboration diagram for Ammo_Stock_List:



Data Fields

- Ammo_Stock * head

  Sentinel head node (typePtr = NULL).
- Ammo_Stock * tail

  Sentinel tail node (typePtr = NULL).

9.2.1 Detailed Description

Ammunition stock list for a single base (sorted by ammo ID).

Definition at line 198 of file storage.h.

The documentation for this struct was generated from the following file:

- C:/vscode/ballistic/doxygen/storage.h

9.3 Ammo_t Struct Reference

Ammunition type definition.

`#include <storage.h>`

Data Fields

- double ballistic_coefficient
  G7 ballistic coefficient (kg/m^2).
- double blastRadius
  Explosive effect radius in meters.
- double diameter
  Caliber in millimeters.
- unsigned id
  Unique identifier.
- double muzzleVelocity
  Initial velocity in m/s.
- char * name
  Dynamically allocated name string.
- double weight
  Mass in kilograms.

9.3.1 Detailed Description

Ammunition type definition.

Contains physical and ballistic properties of ammunition types. These are globally defined and referenced by bases' stock lists.

Definition at line 147 of file storage.h.

The documentation for this struct was generated from the following file:

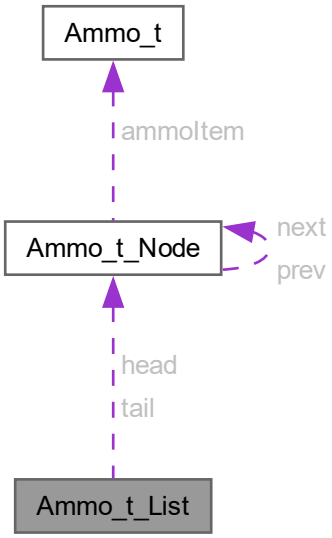
- C:/vscode/ballistic/doxygen/storage.h

9.4 Ammo_t_List Struct Reference

Global ammunition type list (sorted by ID).

`#include <storage.h>`

Collaboration diagram for Ammo_t_List:



Data Fields

- Ammo_t_Node * head
  Sentinel head node (ammoItem = NULL).
- Ammo_t_Node * tail
  Sentinel tail node (ammoItem = NULL).

9.4.1 Detailed Description

Global ammunition type list (sorted by ID).

Sentinel-linked list with head and tail nodes.

Definition at line 175 of file storage.h.

The documentation for this struct was generated from the following file:

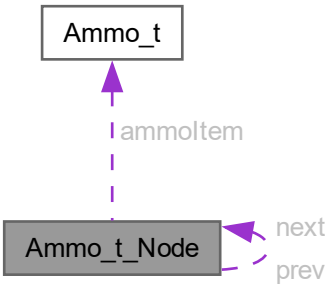
- C:/vscode/ballistic/doxygen/storage.h

9.5 Ammo_t_Node Struct Reference

Node for global ammunition type linked list.

`#include <storage.h>`

Collaboration diagram for Ammo_t_Node:



Data Fields

- Ammo_t * ammoItem
  Pointer to ammunition data.
- struct Ammo_t_Node * next
  Next node in list.
- struct Ammo_t_Node * prev
  Previous node in list.

9.5.1 Detailed Description

Node for global ammunition type linked list.

Definition at line 162 of file storage.h.

The documentation for this struct was generated from the following file:

- C:/vscode/ballistic/doxygen/storage.h

9.6 BallisticResult Struct Reference

Ballistic calculation results.

`#include <calculations.h>`

Data Fields

- double azimuth_deg

  Required azimuth angle in degrees (0-360°).
- double elevation_deg

  Required elevation angle in degrees (5-80° range).
- double impact_velocity

  Estimated impact velocity in m/s.
- double time_of_flight

  Estimated time of flight in seconds.
- int valid

  Flag indicating valid solution (1) or invalid (0).

9.6.1 Detailed Description

Ballistic calculation results.

Contains all computed parameters for a firing solution.

Definition at line 24 of file calculations.h.

9.6.2 Field Documentation

9.6.2.1 azimuth_deg

```
double azimuth_deg
```

Required azimuth angle in degrees (0-360°).

Definition at line 27 of file calculations.h.

9.6.2.2 elevation_deg

```
double elevation_deg
```

Required elevation angle in degrees (5-80° range).

Definition at line 26 of file calculations.h.

9.6.2.3 impact_velocity

```
double impact_velocity
```

Estimated impact velocity in m/s.

Definition at line 29 of file calculations.h.

9.6.2.4 time_of_flight

```
double time_of_flight
```

Estimated time of flight in seconds.

Definition at line 28 of file calculations.h.

9.6.2.5 valid

```
int valid
```

Flag indicating valid solution (1) or invalid (0).

Definition at line 30 of file calculations.h.

The documentation for this struct was generated from the following file:

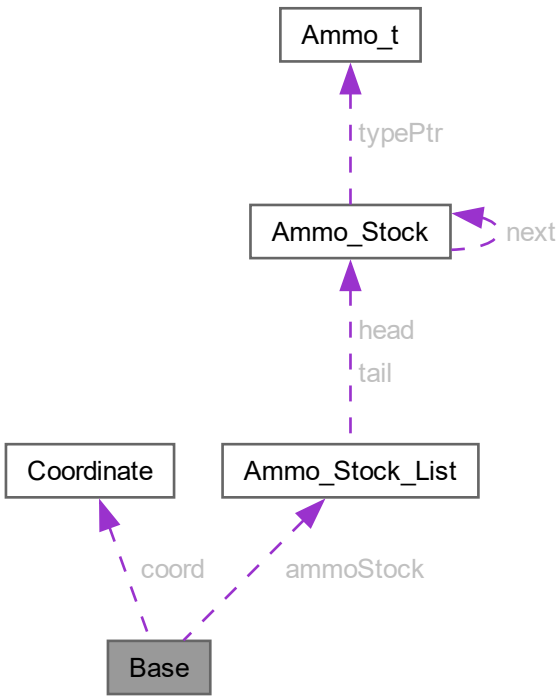
- C:/vscode/ballistic/doxygen/calculations.h

9.7 Base Struct Reference

Military base structure.

```
#include <storage.h>
```

Collaboration diagram for Base:



Data Fields

- Ammo_Stock_List * ammoStock
  Inventory of ammunition types.
- Coordinate coord
  Geographic coordinates.
- unsigned id
  Unique identifier.
- char * label
  Dynamically allocated base name.

9.7.1 Detailed Description

Military base structure.

Contains base identification, location, and ammunition inventory.

Definition at line 210 of file storage.h.

The documentation for this struct was generated from the following file:

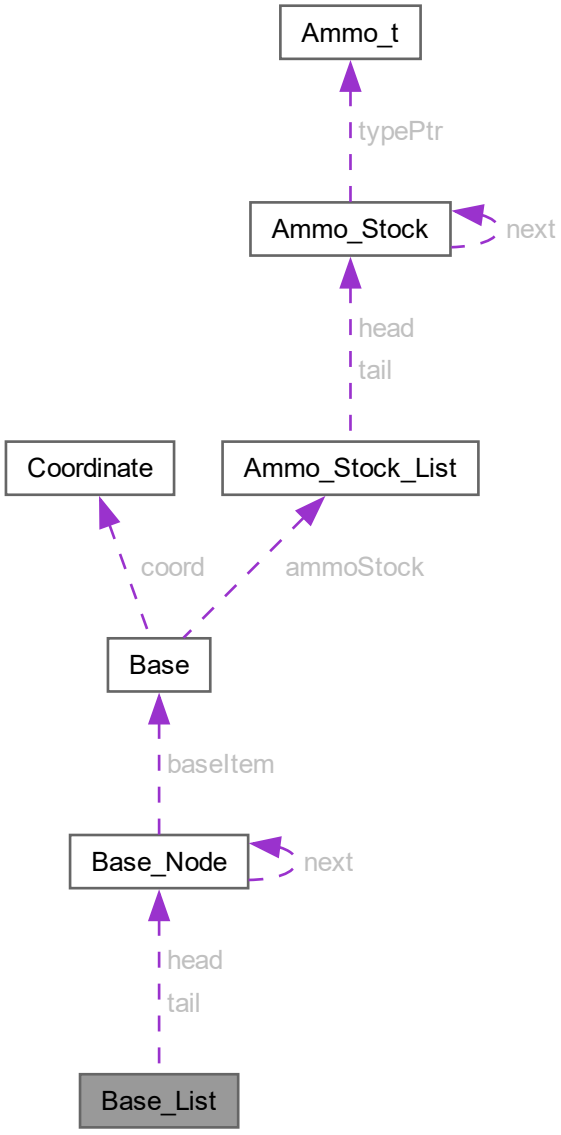
- C:/vscode/ballistic/doxygen/storage.h

9.8 Base_List Struct Reference

Base list (sorted by ID).

```
#include <storage.h>
```

Collaboration diagram for Base_List:



Data Fields

- Base_Node * head

  Sentinel head node (baseItem = NULL).
- Base_Node * tail

  Sentinel tail node (baseItem = NULL).

9.8.1 Detailed Description

Base list (sorted by ID).

Definition at line 232 of file storage.h.

The documentation for this struct was generated from the following file:

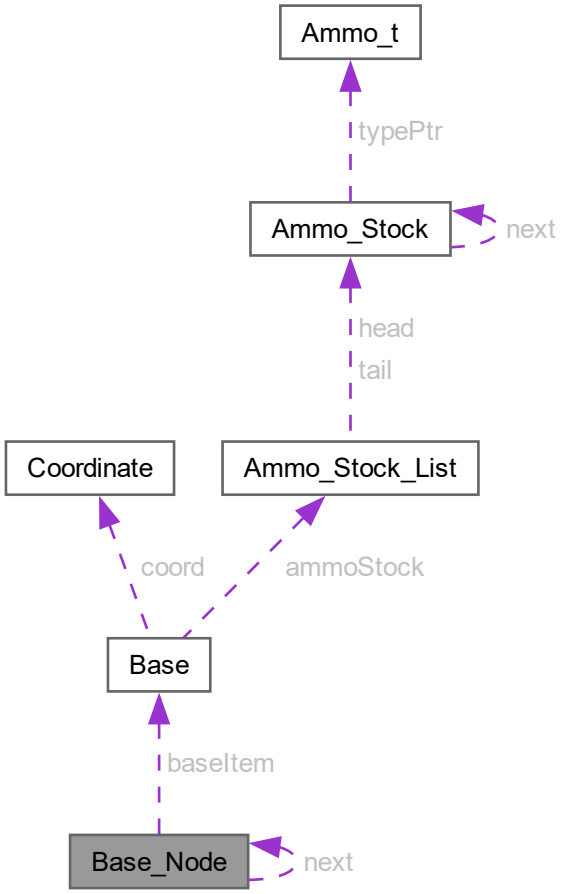
- C:/vscode/ballistic/doxygen/storage.h

9.9 Base_Node Struct Reference

Node for base linked list.

`#include <storage.h>`

Collaboration diagram for Base_Node:



Data Fields

- Base * baseItem
  Pointer to base data.
- struct Base_Node * next
  Next base in list.

9.9.1 Detailed Description

Node for base linked list.

Definition at line 222 of file storage.h.

The documentation for this struct was generated from the following file:

- C:/vscode/ballistic/doxygen/storage.h

9.10 Command Struct Reference

Command definition structure.

`#include <commands.h>`

Data Fields

- uint32_t case_mask
  Bitmask of contexts where command is available.
- const char * description
  Help description.
- void(* handler)(const char *args, Command_case case_flags, PROCESS *running)
  Command handler function.
- const char * name
  Command name (lowercase).
- Command_t type
  Command type identifier.

9.10.1 Detailed Description

Command definition structure.

Definition at line 77 of file commands.h.

9.10.2 Field Documentation

9.10.2.1 case_mask

`uint32_t case_mask`

Bitmask of contexts where command is available.

Definition at line 83 of file commands.h.

9.10.2.2 description

```
const char* description
```

Help description.

Definition at line 82 of file commands.h.

9.10.2.3 handler

```
void(* handler) (const char *args, Command_case case_flags, PROCESS *running)
```

Command handler function.

Definition at line 81 of file commands.h.

9.10.2.4 name

```
const char* name
```

Command name (lowercase).

Definition at line 79 of file commands.h.

9.10.2.5 type

```
Command_t type
```

Command type identifier.

Definition at line 80 of file commands.h.

The documentation for this struct was generated from the following file:

- C:/vscode/ballistic/doxygen/commands.h

9.11 Coordinate Struct Reference

Geographic coordinate structure.

```
#include <storage.h>
```

Data Fields

- double altitude

  Altitude in meters above WGS84 ellipsoid.
- double latitude

  Latitude in degrees (-90 to 90).
- double longitude

  Longitude in degrees (-180 to 180).

9.11.1 Detailed Description

Geographic coordinate structure.

Definition at line 133 of file storage.h.

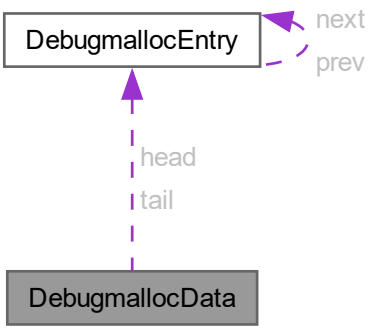
The documentation for this struct was generated from the following file:

- C:/vscode/ballistic/doxygen/storage.h

9.12 DebugmallocData Struct Reference

`#include <debugmalloc.h>`

Collaboration diagram for DebugmallocData:



Data Fields

- long long all_alloc_bytes
- long all_alloc_count
- long long alloc_bytes
- long alloc_count
- DebugmallocEntry head [debugmalloc_tablesize]
- char logfile [256]
- long max_block_size
- DebugmallocEntry tail [debugmalloc_tablesize]

9.12.1 Detailed Description

Definition at line 67 of file debugmalloc.h.

9.12.2 Field Documentation

9.12.2.1 all_alloc_bytes

```
long long all_alloc_bytes
```

Definition at line 73 of file debugmalloc.h.

9.12.2.2 all_alloc_count

```
long all_alloc_count
```

Definition at line 72 of file debugmalloc.h.

9.12.2.3 alloc_bytes

```
long long alloc_bytes
```

Definition at line 71 of file debugmalloc.h.

9.12.2.4 alloc_count

```
long alloc_count
```

Definition at line 70 of file debugmalloc.h.

9.12.2.5 head

```
DebugmallocEntry head[debugmalloc_tablesize]
```

Definition at line 74 of file debugmalloc.h.

9.12.2.6 logfile

```
char logfile[256]
```

Definition at line 68 of file debugmalloc.h.

9.12.2.7 max_block_size

```
long max_block_size
```

Definition at line 69 of file debugmalloc.h.

9.12.2.8 tail

```
DebugmallocEntry tail[debugmalloc_tablesize]
```

Definition at line 74 of file debugmalloc.h.

The documentation for this struct was generated from the following file:

- C:/vscode/ballistic/doxygen/debugmalloc.h

9.13 DebugmallocEntry Struct Reference

```
#include <debugmalloc.h>
```

Collaboration diagram for DebugmallocEntry:



Data Fields

- char expr [128]
- char file [64]
- char func [32]
- unsigned line
- struct DebugmallocEntry * next
- struct DebugmallocEntry * prev
- void * real_mem
- size_t size
- void * user_mem

9.13.1 Detailed Description

Definition at line 52 of file debugmalloc.h.

9.13.2 Field Documentation

9.13.2.1 expr

```
char expr[128]
```

Definition at line 60 of file debugmalloc.h.

9.13.2.2 file

```
char file[64]
```

Definition at line 57 of file debugmalloc.h.

9.13.2.3 func

```
char func[32]
```

Definition at line 59 of file debugmalloc.h.

9.13.2.4 line

```
unsigned line
```

Definition at line 58 of file debugmalloc.h.

9.13.2.5 next

```
struct DebugmallocEntry * next
```

Definition at line 62 of file debugmalloc.h.

9.13.2.6 prev

```
struct DebugmallocEntry* prev
```

Definition at line 62 of file debugmalloc.h.

9.13.2.7 real_mem

```
void* real_mem
```

Definition at line 53 of file debugmalloc.h.

9.13.2.8 size

```
size_t size
```

Definition at line 55 of file debugmalloc.h.

9.13.2.9 user_mem

```
void* user_mem
```

Definition at line 54 of file debugmalloc.h.

The documentation for this struct was generated from the following file:

- C:/vscode/ballistic/doxygen/debugmalloc.h

9.14 State Struct Reference

State vector for ballistic trajectory simulation.

Data Fields

- double vx
- double vy
- double vz

  Velocity components (m/s).
- double x
- double y
- double z

  Position coordinates (meters).

9.14.1 Detailed Description

State vector for ballistic trajectory simulation.

Definition at line 146 of file calculations.c.

9.14.2 Field Documentation

9.14.2.1 vx

```
double vx
```

Definition at line 149 of file calculations.c.

9.14.2.2 vy

```
double vy
```

Definition at line 149 of file calculations.c.

9.14.2.3 vz

```
double vz
```

Velocity components (m/s).

Definition at line 149 of file calculations.c.

9.14.2.4 x

```
double x
```

Definition at line 148 of file calculations.c.

9.14.2.5 y

```
double y
```

Definition at line 148 of file calculations.c.

9.14.2.6 z

```
double z
```

Position coordinates (meters).

Definition at line 148 of file calculations.c.

The documentation for this struct was generated from the following file:

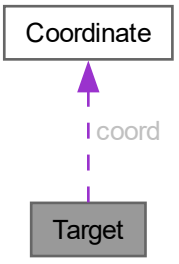
- C:/vscode/ballistic/doxygen/calculations.c

9.15 Target Struct Reference

Target structure.

```
#include <storage.h>
```

Collaboration diagram for Target:



Data Fields

- double area
  Target area in square kilometers.
- Coordinate coord
  Geographic coordinates.
- unsigned id
  Unique identifier.
- char * label
  Dynamically allocated target name.

9.15.1 Detailed Description

Target structure.

Contains target identification, location, and area.

Definition at line 244 of file storage.h.

The documentation for this struct was generated from the following file:

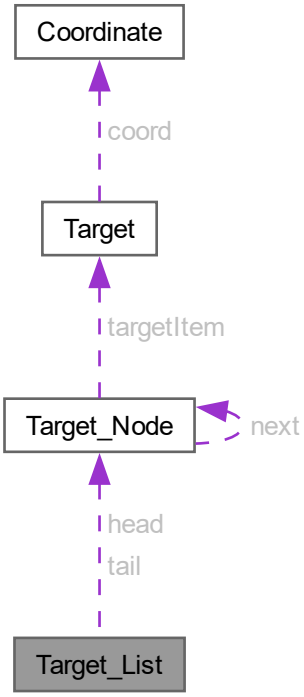
- C:/vscode/ballistic/doxygen/storage.h

9.16 Target_List Struct Reference

Target list (sorted by ID).

`#include <storage.h>`

Collaboration diagram for Target_List:



Data Fields

- Target_Node * head
  Sentinel head node (targetItem = NULL).
- Target_Node * tail
  Sentinel tail node (targetItem = NULL).

9.16.1 Detailed Description

Target list (sorted by ID).

Definition at line 266 of file storage.h.

The documentation for this struct was generated from the following file:

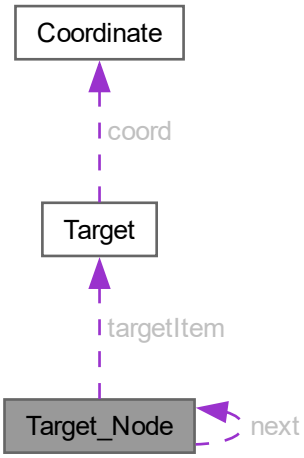
- C:/vscode/ballistic/doxygen/storage.h

9.17 Target_Node Struct Reference

Node for target linked list.

`#include <storage.h>`

Collaboration diagram for Target_Node:



Data Fields

- struct Target_Node * next
  Next target in list.
- Target * targetItem
  Pointer to target data.

9.17.1 Detailed Description

Node for target linked list.

Definition at line 256 of file storage.h.

The documentation for this struct was generated from the following file:

- C:/vscode/ballistic/doxygen/storage.h

9.18 Vec3 Struct Reference

3D vector structure for coordinate calculations

Data Fields

- double x
  X coordinate (meters).
- double y
  Y coordinate (meters).
- double z
  Z coordinate (meters).

9.18.1 Detailed Description

3D vector structure for coordinate calculations

Definition at line 46 of file calculations.c.

9.18.2 Field Documentation

9.18.2.1 x

```
double x
```

X coordinate (meters).

Definition at line 48 of file calculations.c.

9.18.2.2 y

```
double y
```

Y coordinate (meters).

Definition at line 49 of file calculations.c.

9.18.2.3 z

```
double z
```

Z coordinate (meters).

Definition at line 50 of file calculations.c.

The documentation for this struct was generated from the following file:

- C:/vscode/ballistic/doxygen/calculations.c

Chapter 10

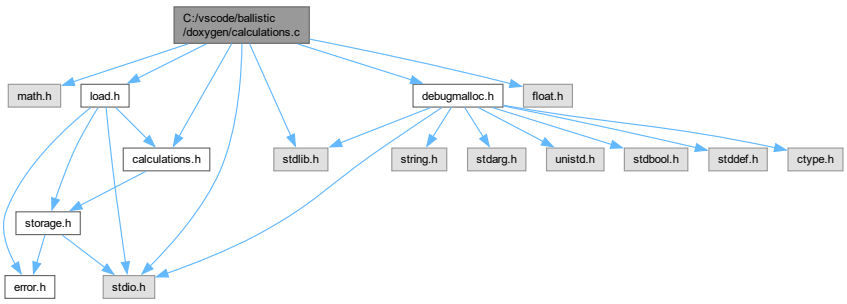
File Documentation

10.1 C:/vscode/ballistic/doxygen/calculations.c File Reference

Implementation of ballistic calculation functions.

```
#include <math.h>
#include <stdlib.h>
#include <stdio.h>
#include <float.h>
#include "calculations.h"
#include "load.h"
#include "debugmalloc.h"
```

Include dependency graph for calculations.c:



Data Structures

- struct State

  State vector for ballistic trajectory simulation.
- struct Vec3

  3D vector structure for coordinate calculations

Macros

- #define _USE_MATH_DEFINES
- #define AIR_DENSITY 1.225

  Air density at sea level (kg/m^3).
- #define DEG2RAD (M_PI / 180.0)

  Conversion factor from degrees to radians.
- #define G 9.80665

  Gravitational acceleration (m/s^2).
- #define RAD2DEG (180.0 / M_PI)

  Conversion factor from radians to degrees.
- #define sq(x)

  Square a value (macro for performance).

Functions

- BallisticResult compute_fire_solution (Base *b, Target *t, Ammo_t *ammo)

  Computes a firing solution for a given base, target, and ammunition.
- int SuggestBestBaseAndAmmo (Base_List *list, Target *tgt, Base **outBase, Ammo_t **outAmmo)

  Suggests the best base and ammunition combination for a target.
- int SuggestBestBaseAndAmmo_RegionAware (Base_List *list, Target *tgt, Base **outBase, Ammo_t **outAmmo)

  Suggests the best base and ammunition combination with regional awareness.

10.1.1 Detailed Description

Implementation of ballistic calculation functions.

This file contains the implementation of functions for calculating ballistic trajectories, coordinate transformations, and optimal base/ammunition selection.

Note

Uses WGS84 ellipsoid model for Earth geometry and G7 drag coefficient model

Implements 4th-order Runge-Kutta integration for trajectory simulation

Distance tolerance: 1 meter, elevation search step: 0.1 degrees

Todo Add caching for frequently computed firing solutions

Implement binary search for elevation angle optimization

Add Coriolis effect for long-range trajectories

Add wind effect modeling

Definition in file calculations.c.

10.1.2 Macro Definition Documentation

10.1.2.1 _USE_MATH_DEFINES

```
#define _USE_MATH_DEFINES
```

Definition at line 17 of file calculations.c.

10.1.2.2 AIR_DENSITY

```
#define AIR_DENSITY 1.225
```

Air density at sea level (kg/m^3).

Definition at line 141 of file calculations.c.

10.1.2.3 DEG2RAD

```
#define DEG2RAD (M_PI / 180.0)
```

Conversion factor from degrees to radians.

Definition at line 28 of file calculations.c.

10.1.2.4 G

```
#define G 9.80665
```

Gravitational acceleration (m/s^2).

Definition at line 140 of file calculations.c.

10.1.2.5 RAD2DEG

```
#define RAD2DEG (180.0 / M_PI)
```

Conversion factor from radians to degrees.

Definition at line 29 of file calculations.c.

10.1.2.6 sq

```
#define sq(
              x)
```

Value:

```
((x) * (x))
```

Square a value (macro for performance).

Parameters

x	Value to square

Returns

Square of the input value

Definition at line 37 of file calculations.c.

10.2 calculations.c

Go to the documentation of this file.

```
00001
00016
00017 #define _USE_MATH_DEFINES
00018 #include <math.h>
00019 #include <stdlib.h>
00020 #include <stdio.h>
00021 #include <float.h>
00022
00023 #include "calculations.h"
00024 #include "load.h"
00025
00026 #include "debugmalloc.h"
00027
00028 #define DEG2RAD (M_PI / 180.0)
00029 #define RAD2DEG (180.0 / M_PI)
00030
00037 #define sq(x) ((x) * (x))
00038
00039 // WGS84 parameters
00040 static const double WGS84_A = 6378137.0;
00041 static const double WGS84_E2 = 6.69437999014e-3;
00042
00046 typedef struct
00047 {
00048     double x;
00049     double y;
00050     double z;
00051 } Vec3;
00052
00064 static Vec3 wgs84_to_ecef(double lat_deg, double lon_deg, double h)
00065 {
00066     double lat = lat_deg * DEG2RAD;
00067     double lon = lon_deg * DEG2RAD;
00068
00069     double sinlat = sin(lat);
00070     double coslat = cos(lat);
00071     double sinlon = sin(lon);
00072     double coslon = cos(lon);
00073
00074     double N = WGS84_A / sqrt(1 - WGS84_E2 * sinlat * sinlat);
00075
00076     Vec3 r;
00077     r.x = (N + h) * coslat * coslon;
00078     r.y = (N + h) * coslat * sinlon;
00079     r.z = (N * (1 - WGS84_E2) + h) * sinlat;
00080     return r;
00081 }
00082
00094 static Vec3 ecef_to_enu(Vec3 p, Vec3 origin, double lat0_deg, double lon0_deg)
00095 {
00096     double lat0 = lat0_deg * DEG2RAD;
00097     double lon0 = lon0_deg * DEG2RAD;
00098
00099     double sinlat = sin(lat0);
00100     double coslat = cos(lat0);
00101     double sinlon = sin(lon0);
00102     double coslon = cos(lon0);
00103
00104     double dx = p.x - origin.x;
00105     double dy = p.y - origin.y;
00106     double dz = p.z - origin.z;
00107
00108     Vec3 enu;
00109     enu.x = -sinlon * dx + coslon * dy;                                  // East
00110     enu.y = -sinlat * coslon * dx - sinlat * sinlon * dy + coslat * dz; // North
00111     enu.z = coslat * coslon * dx + coslat * sinlon * dy + sinlat * dz;  // Up
00112     return enu;
00113 }
00114
00124 static double g7_drag_coefficient(double v)
00125 {
00126     // v is in m/s
00127     double mach = v / 343.0; // Speed of sound at sea level (approx)
00128
00129     if (mach < 0.9)
00130         return 0.5;
00131     if (mach < 1.2)
00132         return 0.8;
00133     if (mach < 2.0)
00134         return 0.5;
00135     if (mach < 3.0)
00136         return 0.3;
```

```
00137     return 0.2;
00138 }
00139
00140 #define G 9.80665
00141 #define AIR_DENSITY 1.225
00142
00146 typedef struct
00147 {
00148     double x, y, z;
00149     double vx, vy, vz;
00150 } State;
00151
00162 static void derivatives(const State *s, State *d, double BC)
00163 {
00164     double v = sqrt(sq(s->vx) + sq(s->vy) + sq(s->vz));
00165     if (v < 1e-6)
00166         v = 1e-6; // avoid divide by zero
00167     double Cd = g7_drag_coefficient(v);
00168     double drag = 0.5 * AIR_DENSITY * Cd * sq(v) / BC;
00169
00170     d->x = s->vx;
00171     d->y = s->vy;
00172     d->z = s->vz;
00173
00174     d->vx = -drag * s->vx / v;
00175     d->vy = -drag * s->vy / v;
00176     d->vz = -G - drag * s->vz / v;
00177 }
00178
00188 static void rk4(State *s, double dt, double BC)
00189 {
00190     State k1, k2, k3, k4, tmp;
00191
00192     derivatives(s, &k1, BC);
00193
00194     tmp = *s;
00195     tmp.x += 0.5 * dt * k1.x;
00196     tmp.y += 0.5 * dt * k1.y;
00197     tmp.z += 0.5 * dt * k1.z;
00198     tmp.vx += 0.5 * dt * k1.vx;
00199     tmp.vy += 0.5 * dt * k1.vy;
00200     tmp.vz += 0.5 * dt * k1.vz;
00201     derivatives(&tmp, &k2, BC);
00202
00203     tmp = *s;
00204     tmp.x += 0.5 * dt * k2.x;
00205     tmp.y += 0.5 * dt * k2.y;
00206     tmp.z += 0.5 * dt * k2.z;
00207     tmp.vx += 0.5 * dt * k2.vx;
00208     tmp.vy += 0.5 * dt * k2.vy;
00209     tmp.vz += 0.5 * dt * k2.vz;
00210     derivatives(&tmp, &k3, BC);
00211
00212     tmp = *s;
00213     tmp.x += dt * k3.x;
00214     tmp.y += dt * k3.y;
00215     tmp.z += dt * k3.z;
00216     tmp.vx += dt * k3.vx;
00217     tmp.vy += dt * k3.vy;
00218     tmp.vz += dt * k3.vz;
00219     derivatives(&tmp, &k4, BC);
00220
00221     s->x += (dt / 6.0) * (k1.x + 2 * k2.x + 2 * k3.x + k4.x);
00222     s->y += (dt / 6.0) * (k1.y + 2 * k2.y + 2 * k3.y + k4.y);
00223     s->z += (dt / 6.0) * (k1.z + 2 * k2.z + 2 * k3.z + k4.z);
00224
00225     s->vx += (dt / 6.0) * (k1.vx + 2 * k2.vx + 2 * k3.vx + k4.vx);
00226     s->vy += (dt / 6.0) * (k1.vy + 2 * k2.vy + 2 * k3.vy + k4.vy);
00227     s->vz += (dt / 6.0) * (k1.vz + 2 * k2.vz + 2 * k3.vz + k4.vz);
00228 }
00229
00245 static BallisticResult solve_ballistic(Vec3 targetENU, double muzzleVel, double BC)
00246 {
00247     BallisticResult r = {0};
00248     double azimuth = atan2(targetENU.x, targetENU.y); // east-north convention
00249     r.azimuth_deg = azimuth * RAD2DEG;
00250
00251     double R = sqrt(targetENU.x * targetENU.x + targetENU.y * targetENU.y);
00252     double DZ = targetENU.z;
00253
00254     double best_error = DBL_MAX;
00255
00256     // Search through elevation angles from 5 to 80 degrees
00257     for (double elev_deg = 5.0; elev_deg <= 80.0; elev_deg += 0.1)
00258     {
00259         double elev = elev_deg * DEG2RAD;
00260
```

```
00261         State s = {0};
00262         s.x = s.y = s.z = 0;
00263         s.vx = muzzleVel * cos(elev) * sin(azimuth);
00264         s.vy = muzzleVel * cos(elev) * cos(azimuth);
00265         s.vz = muzzleVel * sin(elev);
00266 
00267         double t = 0.0;
00268         double dt = 0.02; // 20ms time step
00269 
00270         // Simulate trajectory
00271         for (;;)
00272         {
00273             rk4(&s, dt, BC);
00274             t += dt;
00275 
00276             if (s.z < DZ - 50.0)
00277                 break; // too low too early (miss)
00278             if (t > 200.0)
00279                 break; // safety breaker (max 200 seconds flight time)
00280 
00281             // Check if projectile is near target altitude (1 meter tolerance)
00282             if (s.z < DZ && s.z > DZ - 1.0)
00283             {
00284                 double dist = sqrt(s.x * s.x + s.y * s.y);
00285                 double error = fabs(dist - R);
00286 
00287                 if (error < best_error)
00288                 {
00289                     best_error = error;
00290                     r.elevation_deg = elev_deg;
00291                     r.time_of_flight = t;
00292                     r.impact_velocity = sqrt(s.vx * s.vx + s.vy * s.vy + s.vz * s.vz);
00293                     r.valid = 1;
00294                 }
00295                 break;
00296             }
00297         }
00298     }
00299 
00300     return r;
00301 }
00302 
00318 BallisticResult compute_fire_solution(Base *b, Target *t, Ammo_t *ammo)
00319 {
00320     Vec3 shooterECEF = wgs84_to_ecef(b->coord.latitude, b->coord.longitude, b->coord.altitude);
00321     Vec3 targetECEF = wgs84_to_ecef(t->coord.latitude, t->coord.longitude, t->coord.altitude);
00322 
00323     Vec3 enu_target = ecef_to_enu(
00324         targetECEF,
00325         shooterECEF,
00326         b->coord.latitude,
00327         b->coord.longitude);
00328 
00329     return solve_ballistic(enu_target, ammo->muzzleVelocity, ammo->ballistic_coefficient);
00330 }
00331 
00332 // ============================================================
00333 // Helper functions for base suggestion algorithms
00334 // ============================================================
00335 
00346 static double GeoDistance2D(const Coordinate *a, const Coordinate *b)
00347 {
00348     if (!a || !b)
00349         return 0.0;
00350 
00351     double lat1 = a->latitude * DEG2RAD;
00352     double lat2 = b->latitude * DEG2RAD;
00353     double dlat = lat2 - lat1;
00354     double dlon = (b->longitude - a->longitude) * DEG2RAD;
00355 
00356     double s1 = sin(dlat * 0.5);
00357     double s2 = sin(dlon * 0.5);
00358 
00359     double h = s1 * s1 + cos(lat1) * cos(lat2) * s2 * s2;
00360     double c = 2.0 * atan2(sqrt(h), sqrt(1.0 - h));
00361     return WGS84_A * c; // meters (approx mean Earth radius via WGS84 a)
00362 }
00363 
00374 static double GeoDistance3D(const Coordinate *a, const Coordinate *b)
00375 {
00376     double d2d = GeoDistance2D(a, b);
00377     double dz = (b->altitude - a->altitude);
00378     return sqrt(d2d * d2d + dz * dz);
00379 }
00380 
00388 static int BaseHasAmmo(Base *b, unsigned ammo_id)
00389 {
```

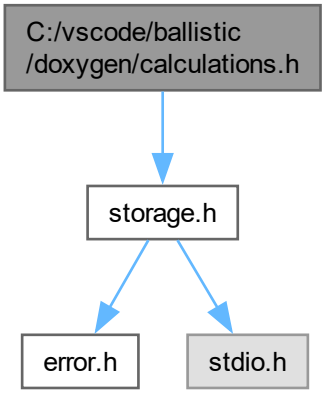
```
00390     if (!b || !b->ammoStock)
00391         return 0;
00392
00393     Ammo_Stock *it = b->ammoStock->head;
00394     while (it)
00395     {
00396         if (it->typePtr && it->typePtr->id == ammo_id && it->quantity > 0)
00397             return 1;
00398         it = it->next;
00399     }
00400     return 0;
00401 }
00402
00423 int SuggestBestBaseAndAmmo(Base_List *list, Target *tgt, Base **outBase, Ammo_t **outAmmo)
00424 {
00425     if (!list || !tgt || !outBase || !outAmmo)
00426         return 0;
00427
00428     Base *bestBase = NULL;
00429     Ammo_t *bestAmmo = NULL;
00430     double bestDist = DBL_MAX;
00431     unsigned bestQty = 0; // tie-breaker: more stock wins
00432
00433     Base_Node *bn = list->head;
00434     while (bn)
00435     {
00436         Base *b = bn->baseItem;
00437         bn = bn->next;
00438
00439         if (!b || !b->ammoStock || !b->ammoStock->head)
00440             continue;
00441
00442         double dist3d = GeoDistance3D(&b->coord, &tgt->coord);
00443
00444         // Early rejection: if already farther than best candidate + 1 meter tolerance, skip
00445         if (dist3d > bestDist + 1.0)
00446             continue;
00447
00448         Ammo_Stock *stock = b->ammoStock->head;
00449         while (stock)
00450         {
00451             Ammo_t *a = stock->typePtr;
00452
00453             if (a && stock->quantity > 0)
00454             {
00455                 BallisticResult sol = compute_fire_solution(b, tgt, a);
00456                 if (sol.valid)
00457                 {
00458                     int better = 0;
00459
00460                     if (dist3d < bestDist - 1.0)
00461                     {
00462                         // Strictly closer (by more than 1 meter) is always better
00463                         better = 1;
00464                     }
00465                     else if (fabs(dist3d - bestDist) <= 1.0)
00466                     {
00467                         // If same distance within 1 meter tolerance, ammunition quantity decides
00468                         if (stock->quantity > bestQty)
00469                             better = 1;
00470                     }
00471
00472                     if (better)
00473                     {
00474                         bestDist = dist3d;
00475                         bestQty = stock->quantity;
00476                         bestBase = b;
00477                         bestAmmo = a;
00478                     }
00479                 }
00480             }
00481             stock = stock->next;
00482         }
00483     }
00484
00485     if (!bestBase)
00486         return 0;
00487
00488     *outBase = bestBase;
00489     *outAmmo = bestAmmo;
00490     return 1;
00491 }
00492
00514 int SuggestBestBaseAndAmmo_RegionAware(Base_List *list, Target *tgt, Base **outBase, Ammo_t **outAmmo)
00515 {
00516     if (!list || !tgt || !outBase || !outAmmo)
00517         return 0;
```

```
00518
00519     unsigned regionStart = (tgt->id / 1000) * 1000;
00520     unsigned regionEnd = regionStart + 999;
00521
00522     Base *bestBase = NULL;
00523     Ammo_t *bestAmmo = NULL;
00524     double bestDist = DBL_MAX;
00525     unsigned bestQty = 0;
00526
00527     int pass;
00528     for (pass = 1; pass <= 2; pass++)
00529     {
00530         Base_Node *bn = list->head;
00531
00532         while (bn)
00533         {
00534             Base *b = bn->baseItem;
00535             bn = bn->next;
00536
00537             if (!b || !b->ammoStock || !b->ammoStock->head)
00538                 continue;
00539
00540             int inRegion = (b->id >= regionStart && b->id <= regionEnd);
00541
00542             if (pass == 1 && !inRegion)
00543                 continue; // first pass: region only
00544             if (pass == 2 && inRegion)
00545                 continue; // second pass: everything except region
00546
00547             double dist3d = GeoDistance3D(&b->coord, &tgt->coord);
00548
00549             if (dist3d > bestDist + 1.0)
00550                 continue;
00551
00552             Ammo_Stock *stock = b->ammoStock->head;
00553             while (stock)
00554             {
00555                 Ammo_t *a = stock->typePtr;
00556
00557                 if (a && stock->quantity > 0)
00558                 {
00559                     BallisticResult sol = compute_fire_solution(b, tgt, a);
00560                     if (sol.valid)
00561                     {
00562                         int better = 0;
00563
00564                         if (dist3d < bestDist - 1.0)
00565                             better = 1;
00566                         else if (fabs(dist3d - bestDist) <= 1.0)
00567                         {
00568                             if (stock->quantity > bestQty)
00569                                 better = 1;
00570                         }
00571
00572                         if (better)
00573                         {
00574                             bestBase = b;
00575                             bestAmmo = a;
00576                             bestDist = dist3d;
00577                             bestQty = stock->quantity;
00578                         }
00579                     }
00580                 }
00581
00582                 stock = stock->next;
00583             }
00584         }
00585
00586         // If pass 1 found something, stop here (prefer regional solutions)
00587         if (bestBase)
00588             break;
00589     }
00590
00591     if (!bestBase)
00592         return 0;
00593
00594     *outBase = bestBase;
00595     *outAmmo = bestAmmo;
00596     return 1;
00597 }
```

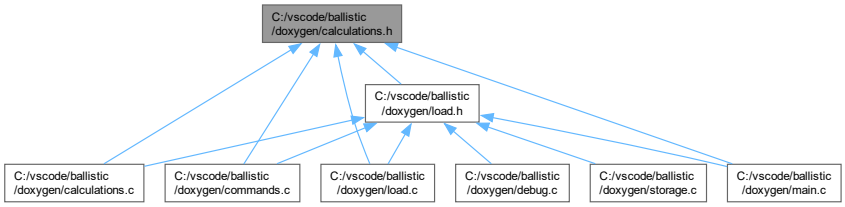
10.3 C:/vscode/ballistic/doxygen/calculations.h File Reference

```
#include "storage.h"
```

Include dependency graph for calculations.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct BallisticResult

  Ballistic calculation results.

Functions

- BallisticResult compute_fire_solution (Base *b, Target *t, Ammo_t *ammo)

  Compute firing solution for base-target-ammunition combination.
- int SuggestBestBaseAndAmmo (Base_List *list, Target *tgt, Base **outBase, Ammo_t **outAmmo)

  Suggest optimal base and ammunition for a target.
- int SuggestBestBaseAndAmmo_RegionAware (Base_List *list, Target *tgt, Base **outBase, Ammo_t **outAmmo)

  Suggest optimal base and ammunition with regional preference.

10.4 calculations.h

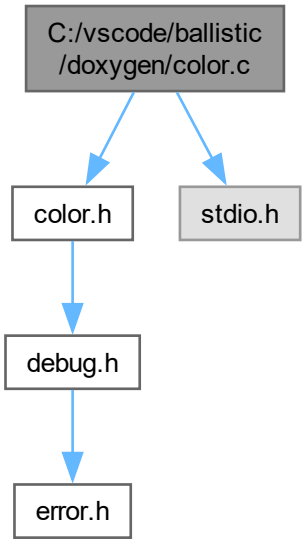
Go to the documentation of this file.

```
00001
00012
00013 #ifndef CALCULATIONS_H
00014 #define CALCULATIONS_H
00015
00016 #include "storage.h"
00017
00024 typedef struct
00025 {
00026     double elevation_deg;
00027     double azimuth_deg;
00028     double time_of_flight;
00029     double impact_velocity;
00030     int valid;
00031 } BallisticResult;
00032
00050 BallisticResult compute_fire_solution(Base *b, Target *t, Ammo_t *ammo);
00051
00074 int SuggestBestBaseAndAmmo(Base_List *list, Target *tgt, Base **outBase, Ammo_t **outAmmo);
00075
00097 int SuggestBestBaseAndAmmo_RegionAware(Base_List *list, Target *tgt, Base **outBase, Ammo_t
      **outAmmo);
00098
00099 #endif /* CALCULATIONS_H */
```

10.5 C:/vscode/ballistic/doxygen/color.c File Reference

```
#include "color.h"
#include "stdio.h"
```

Include dependency graph for color.c:



Functions

- ColorMode get_color_mode (void)

- HRESULT print_colored (const char *color, const char *string, bool always_p)
- void set_color_mode (ColorMode mode)

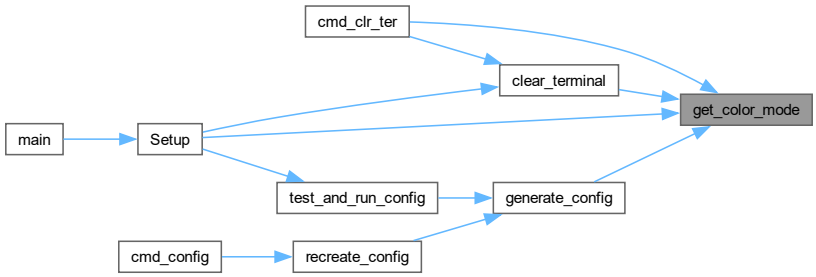
10.5.1 Function Documentation

10.5.1.1 get_color_mode()

```
ColorMode get_color_mode (
            void )
```

Definition at line 6 of file color.c.

Here is the caller graph for this function:

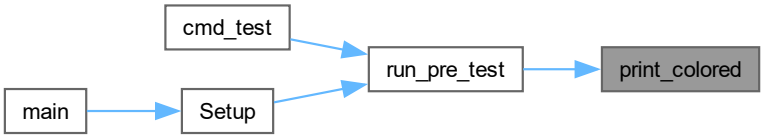


10.5.1.2 print_colored()

```
HRESULT print_colored (
            const char * color,
            const char * string,
            bool always_p)
```

Definition at line 24 of file color.c.

Here is the caller graph for this function:

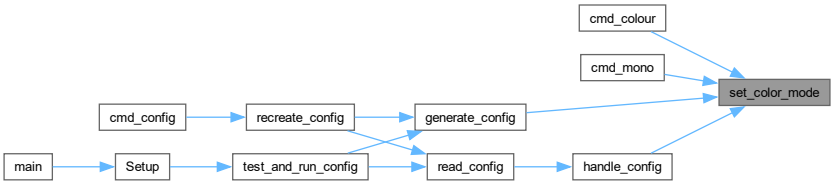


10.5.1.3 set_color_mode()

```
void set_color_mode (
            ColorMode mode)
```

Definition at line 11 of file color.c.

Here is the caller graph for this function:



10.6 color.c

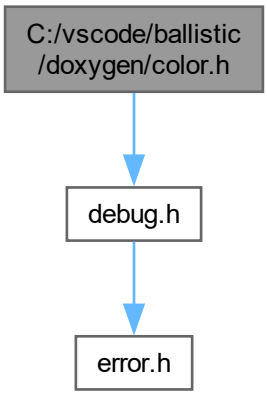
Go to the documentation of this file.

```
00001 #include "color.h"
00002 #include "stdio.h"
00003
00004 static ColorMode current_mode = CM_MONO; // static to make it "private" to this file
00005
00006 ColorMode get_color_mode(void)
00007 {
00008     return current_mode;
00009 }
00010
00011 void set_color_mode(ColorMode mode)
00012 {
00013     switch (mode)
00014     {
00015     case CM_MONO:
00016     case CM_COLOUR:
00017         current_mode = mode;
00018         break;
00019     default:
00020         current_mode = CM_MONO;
00021     }
00022 }
00023
00024 HRESULT print_colored(const char *color, const char *string, bool always_p)
00025 {
00026     if (current_mode == CM_COLOUR)
00027     {
00028         printf("%s%s%s", color, string, ANSI_COLOR_RESET);
00029         return S_OK;
00030     }
00031     else if (always_p)
00032     {
00033         printf("%s", string);
00034     }
00035     return S_FALSE;
00036 }
```

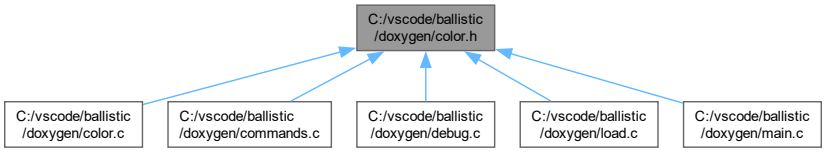
10.7 C:/vscode/ballistic/doxygen/color.h File Reference

```
#include "debug.h"
```

Include dependency graph for color.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define ANSI_COLOR_BLUE "\x1b[34m"
- #define ANSI_COLOR_CYAN "\x1b[36m"
- #define ANSI_COLOR_GREEN "\x1b[32m"
- #define ANSI_COLOR_MAGENTA "\x1b[35m"
- #define ANSI_COLOR_RED "\x1b[31m"
- #define ANSI_COLOR_RESET "\x1b[0m"
- #define ANSI_COLOR_YELLOW "\x1b[33m"

Enumerations

- enum ColorMode { CM_MONO = 0 , CM_COLOUR = 1 }

Functions

- ColorMode get_color_mode (void)
- HRESULT print_colored (const char *color, const char *string, bool always_p)
- void set_color_mode (ColorMode mode)

10.7.1 Macro Definition Documentation

10.7.1.1 ANSI_COLOR_BLUE

```
#define ANSI_COLOR_BLUE "\x1b[34m"
```

Definition at line 9 of file color.h.

10.7.1.2 ANSI_COLOR_CYAN

```
#define ANSI_COLOR_CYAN "\x1b[36m"
```

Definition at line 11 of file color.h.

10.7.1.3 ANSI_COLOR_GREEN

```
#define ANSI_COLOR_GREEN "\x1b[32m"
```

Definition at line 7 of file color.h.

10.7.1.4 ANSI_COLOR_MAGENTA

```
#define ANSI_COLOR_MAGENTA "\x1b[35m"
```

Definition at line 10 of file color.h.

10.7.1.5 ANSI_COLOR_RED

```
#define ANSI_COLOR_RED "\x1b[31m"
```

Definition at line 6 of file color.h.

10.7.1.6 ANSI_COLOR_RESET

```
#define ANSI_COLOR_RESET "\x1b[0m"
```

Definition at line 12 of file color.h.

10.7.1.7 ANSI_COLOR_YELLOW

```
#define ANSI_COLOR_YELLOW "\x1b[33m"
```

Definition at line 8 of file color.h.

10.7.2 Enumeration Type Documentation

10.7.2.1 ColorMode

```
enum ColorMode
```

Enumerator

CM_MONO	
CM_COLOUR	

Definition at line 17 of file color.h.

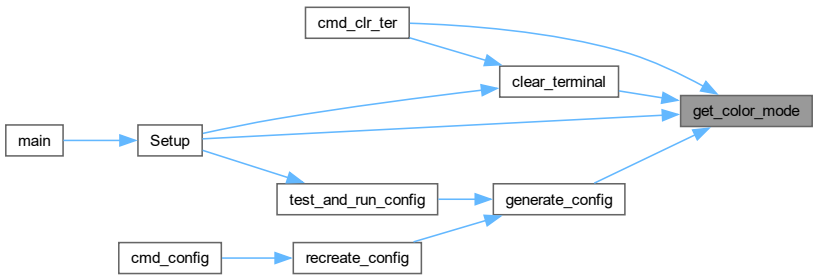
10.7.3 Function Documentation

10.7.3.1 get_color_mode()

```
ColorMode get_color_mode (
            void )
```

Definition at line 6 of file color.c.

Here is the caller graph for this function:

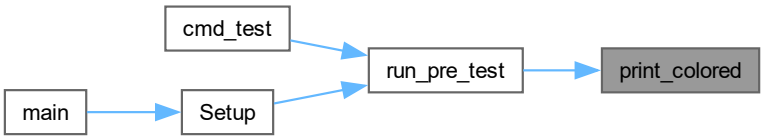


10.7.3.2 print_colored()

```
HRESULT print_colored (
            const char * color,
            const char * string,
            bool always_p)
```

Definition at line 24 of file color.c.

Here is the caller graph for this function:

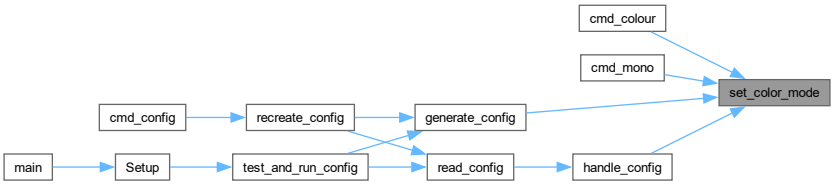


10.7.3.3 set_color_mode()

```
void set_color_mode (
            ColorMode mode)
```

Definition at line 11 of file color.c.

Here is the caller graph for this function:



10.8 color.h

Go to the documentation of this file.

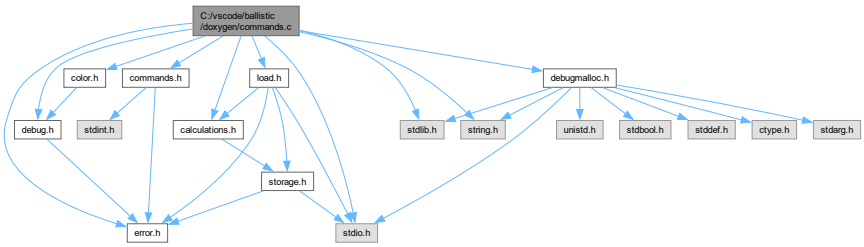
```
00001 #ifndef COLOR_H
00002 #define COLOR_H
00003
00004 #include "debug.h"
00005
00006 #define ANSI_COLOR_RED "\x1b[31m"
00007 #define ANSI_COLOR_GREEN "\x1b[32m"
00008 #define ANSI_COLOR_YELLOW "\x1b[33m"
00009 #define ANSI_COLOR_BLUE "\x1b[34m"
00010 #define ANSI_COLOR_MAGENTA "\x1b[35m"
00011 #define ANSI_COLOR_CYAN "\x1b[36m"
00012 #define ANSI_COLOR_RESET "\x1b[0m"
00013
00014 // reg add HKCU\\Console /v VirtualTerminalLevel /t REG_DWORD /d 1 /f
00015 // make it print out if no colours
00016
00017 typedef enum
00018 {
00019     CM_MONO = 0,
00020     CM_COLOUR = 1
00021 } ColorMode;
00022
00023 ColorMode get_color_mode(void);
00024 void set_color_mode(ColorMode mode);
00025
00026 HRESULT print_colored(const char *color, const char *string, bool always_p);
00027
00028 #endif
```

10.9 C:/vscode/ballistic/doxygen/commands.c File Reference

```
#include "commands.h"
#include "error.h"
#include "debug.h"
#include "calculations.h"
#include "color.h"
#include "load.h"
#include <stdio.h>
#include <stdlib.h>
```

```
#include <string.h>
#include "debugmalloc.h"
```

Include dependency graph for commands.c:



Functions

- void add_seperator (const char *str, char symbol, unsigned sep_length)

  Print formatted separator line.
- void clear_terminal ()

  Clear terminal screen.
- void cmd_ammo (const char *args, Command_case case_flags, PROCESS *running)

  Ammunition operations handler.
- void cmd_base (const char *args, Command_case case_flags, PROCESS *running)

  Base operations handler.
- void cmd_calculate (const char *args, Command_case case_flags, PROCESS *running)

  Calculate trajectory handler.
- void cmd_clr_ter (const char *args, Command_case case_flags, PROCESS *running)

  Clear terminal handler.
- void cmd_colour (const char *args, Command_case case_flags, PROCESS *running)

  Set color mode handler.
- void cmd_config (const char *args, Command_case case_flags, PROCESS *running)

  Configuration handler.
- void cmd_exit (const char *args, Command_case case_flags, PROCESS *running)

  Exit program handler.
- void cmd_help (const char *args, Command_case case_flags, PROCESS *running)

  Help command handler.
- void cmd_mono (const char *args, Command_case case_flags, PROCESS *running)

  Set monochrome mode handler.
- void cmd_target (const char *args, Command_case case_flags, PROCESS *running)

  Target operations handler.
- void cmd_test (const char *args, Command_case case_flags, PROCESS *running)

  Test system handler.
- Command_t GetCommand (Command_case case_flags, PROCESS *running)

  Get command from user with modal context.
- Command_t ParseCommandFromBuffer (char *input, Command_case case_flags, PROCESS *running)

  Parse command from buffer (non-interactive).

Variables

- int command_count = sizeof(commands) / sizeof(commands[0])

  Number of commands in command table.
- Command commands [ ]

  Global command table.

10.10 commands.c

Go to the documentation of this file.

```
00001 #include "commands.h"
00002 #include "error.h"
00003 #include "debug.h"
00004 #include "calculations.h"
00005 #include "color.h"
00006 #include "load.h"
00007
00008 #include <stdio.h>
00009 #include <stdlib.h>
00010 #include <string.h>
00011
00012 #include "debugmalloc.h"
00013
00014 Command commands[] = {
00015     {"back", CMD_BACK, NULL, "step back", CMD_C_LOAD | CMD_C_SELECT | CMD_C_BACK},
00016     {"cancel", CMD_CANCEL, NULL, "Cancel event", CMD_C_CANCEL},
00017     {"select", CMD_SELECT, NULL, "select choice", CMD_C_SELECT},
00018     {"list", CMD_LIST, NULL, "list values", CMD_C_LOAD},
00019     {"add", CMD_ADD, NULL, "add value", CMD_C_LOAD},
00020     {"delete", CMD_DELETE, NULL, "delete value", CMD_C_LOAD},
00021     {"save", CMD_SAVE, NULL, "save changes", CMD_C_EXIT},
00022
00023     {"bases", CMD_BASE, cmd_base, "List bases", CMD_C_GENERIC | CMD_C_DEFAULT | CMD_C_CALC},
00024     {"targets", CMD_TARGET, cmd_target, "List targets", CMD_C_GENERIC | CMD_C_DEFAULT | CMD_C_CALC},
00025     {"ammo", CMD_AMMO, cmd_ammo, "List ammo", CMD_C_GENERIC | CMD_C_DEFAULT | CMD_C_CALC},
00026     {"calc", CMD_CALCULATE, cmd_calculate, "calculate trajectory", CMD_C_GENERIC | CMD_C_DEFAULT},
00027
00028     {"test", CMD_TEST, cmd_test, "Do a manual self-test", CMD_C_GENERIC | CMD_C_DEFAULT |
      CMD_C_SYSTEM},
00029     {"clear", CMD_CLEAR_TERMINAL, cmd_clr_ter, "Clears the terminal", CMD_C_GENERIC | CMD_C_DEFAULT |
      CMD_C_SYSTEM},
00030     {"config", CMD_TEST, cmd_config, "Modify config", CMD_C_GENERIC | CMD_C_DEFAULT | CMD_C_SYSTEM},
00031     {"exit", CMD_EXIT, cmd_exit, "Exit program", CMD_C_GENERIC | CMD_C_DEFAULT | CMD_C_SYSTEM},
00032     {"help", CMD_HELP, cmd_help, "Show commands", CMD_C_ALL},
00033
00034     {"colour", CMD_COLOUR, cmd_colour, "coloured mode command", CMD_C_SETUP},
00035     {"mono", CMD_MONO, cmd_mono, "monochrome mode command", CMD_C_SETUP},
00036
00037     {"yes", CMD_YES, NULL, "yes command", CMD_C_YESNO},
00038     {"no", CMD_NO, NULL, "no command", CMD_C_YESNO}};
00039
00040 /*calculates the number of commands, the division with the first value is needed,
00041 because we devide the whole size, with one entry's size, and by that we get the number of entry's*/
00042 int command_count = sizeof(commands) / sizeof(commands[0]);
00043
00044 //================================================
00045
00046 void cmd_exit(const char *args, Command_case case_flags, PROCESS *running)
00047 {
00048     printf("Exiting...\n");
00049     if (running)
00050         *running = S_END;
00051     else
00052     {
00053         printf("Running pointer error\n");
00054     }
00055 }
00056
00057 void cmd_help(const char *args, Command_case case_flags, PROCESS *running)
00058 {
00059     // clear_terminal();
00060     add_seperator("command list", '-', 30);
00061
00062     for (int i = 0; i < command_count; i++)
00063     {
00064         if (!(commands[i].case_mask & case_flags) || (commands[i].type == CMD_HELP))
00065             continue; // command not available in this mode
```

```
00066         printf("    %-4s\t- %s\n", commands[i].name, commands[i].description);
00067     }
00068     add_seperator(NULL, '-', 30);
00069 }
00070
00071 void cmd_target(const char *args, Command_case case_flags, PROCESS *running)
00072 {
00073     printf("%-39s <[%s]>\n", "What would you like to do?", "back");
00074     printf("\t- %-30s [%s]\n", "show target list", "list");
00075     printf("\t- %-30s [%s]\n", "add target to list", "add");
00076     printf("\t- %-30s [%s]\n", "remove target from list", "delete");
00077
00078     Command_t cmd;
00079     while ((cmd = GetCommand(CMD_C_LOAD | CMD_C_LOOP, NULL)) != CMD_BACK)
00080         switch (cmd)
00081         {
00082         case CMD_LIST:
00083             print_Target_List(target_list);
00084             return;
00085         case CMD_ADD:
00086             char buffer[1024];
00087             fgets(buffer, sizeof(buffer), stdin);
00088             handle_target(buffer, target_list, NULL);
00089             break;
00090         case CMD_DELETE:
00091             unsigned id;
00092             scanf("%u", &id);
00093             delete_Target(target_list, id);
00094             break;
00095
00096         default:
00097             break;
00098         }
00099 }
00100
00101 void cmd_base(const char *args, Command_case case_flags, PROCESS *running)
00102 {
00103     printf("%-39s <[%s]>\n", "What would you like to do?", "back");
00104     printf("\t- %-30s [%s]\n", "show base list", "list");
00105     printf("\t- %-30s [%s]\n", "add base to list", "add");
00106     printf("\t- %-30s [%s]\n", "remove base from list", "delete");
00107     Command_t cmd;
00108     while ((cmd = GetCommand(CMD_C_LOAD | CMD_C_LOOP, NULL)) != CMD_BACK)
00109         switch (cmd)
00110         {
00111         case CMD_LIST:
00112             print_Base_List(base_list, S_DEBUG);
00113             return;
00114         case CMD_ADD:
00115             char buffer[1024];
00116             fgets(buffer, sizeof(buffer), stdin);
00117             handle_base(buffer, base_list, NULL);
00118             break;
00119         case CMD_DELETE:
00120             unsigned id;
00121             scanf("%u", &id);
00122             delete_Base(base_list, id);
00123             break;
00124
00125         default:
00126             break;
00127         }
00128 }
00129 // should accept parameters, from the cmd args, that will be part of a later project
00130 void cmd_ammo(const char *args, Command_case case_flags, PROCESS *running)
00131 {
00132     printf("%-39s <[%s]>\n", "What would you like to do?", "back");
00133     printf("\t- %-30s [%s]\n", "show ammo type list", "list");
00134     printf("\t- %-30s [%s]\n", "add ammo type to list", "add");
00135     printf("\t- %-30s [%s]\n", "remove ammo type from list", "delete");
00136
00137     Command_t cmd;
00138     while ((cmd = GetCommand(CMD_C_LOAD | CMD_C_LOOP, NULL)) != CMD_BACK)
00139         switch (cmd)
00140         {
00141         case CMD_LIST:
00142             print_Ammo_t_List(ammo_t_list);
00143             return;
00144         case CMD_ADD:
00145             char buffer[1024];
00146             fgets(buffer, sizeof(buffer), stdin);
00147             handle_ammo_t(buffer, ammo_t_list, NULL);
00148             break;
00149         case CMD_DELETE:
00150             args;
00151             unsigned id;
00152             scanf("%u", &id);
```

```
00153             delete_Ammo_t(ammo_t_list, id);
00154             break;
00155
00156         default:
00157             break;
00158         }
00159 }
00160
00161 void cmd_test(const char *args, Command_case case_flags, PROCESS *running)
00162 {
00163
00164     run_pre_test(S_DEBUG);
00165 }
00166
00167 void cmd_colour(const char *args, Command_case case_flags, PROCESS *running)
00168 {
00169     set_color_mode(CMD_COLOUR);
00170 }
00171
00172 void cmd_mono(const char *args, Command_case case_flags, PROCESS *running)
00173 {
00174     set_color_mode(CM_MONO);
00175 }
00176
00177 // clear terminal command
00178 void cmd_clr_ter(const char *args, Command_case case_flags, PROCESS *running)
00179 {
00180     if (get_color_mode())
00181         clear_terminal();
00182 }
00183
00184 void cmd_config(const char *args, Command_case case_flags, PROCESS *running)
00185 {
00186     recreate_config("config.txt", S_DEBUG);
00187 }
00188
00189 void cmd_calculate(const char *args, Command_case case_flags, PROCESS *running)
00190 {
00191     EntryPoint:
00192     Command_t cmd;
00193     printf("%-39s <[%s]>\n", "What would you like to do?", "back");
00194     printf("\t- %-30s [%s]\n", "select parameters", "select");
00195     printf("\t- %-30s [%s]\n", "work with bases", "bases");
00196     printf("\t- %-30s [%s]\n", "work with target", "target");
00197     printf("\t- %-30s [%s]\n", "work with ammo", "ammo");
00198     while ((cmd = GetCommand(CMD_C_LOOP | CMD_C_CALC | CMD_C_SELECT, NULL)) != CMD_BACK)
00199     {
00200         switch (cmd)
00201         {
00202         case CMD_SELECT:
00203             printf("Please input the base, target, ammo ID!\n");
00204             printf("If you aren't sure about the ID's, type [back], and check the [list] for the
    desired value!\n");
00205             printf("If you can't find, what you are looking for, then [add] it to the list!\n");
00206             while (HandleParameterSelection() == 0);
00207             break;
00208
00209         default:
00210             break;
00211         }
00212         goto EntryPoint;
00213     }
00214 }
00215 //================================================
00216
00217 Command_t GetCommand(Command_case case_flags, PROCESS *running)
00218 {
00219     char input[128];
00220     int modal = 1;
00221     Command_t command = CMD_INVALID;
00222
00223     while (modal)
00224     {
00225         printf("> ");
00226         if (fgets(input, (sizeof(input) > 128) ? 128 : sizeof(input), stdin))
00227         {
00228             NormalizeInput(input);
00229             char *cmd = strtok(input, " \t\n");
00230             char *args = strtok(NULL, "\n");
00231
00232             if (cmd)
00233             {
00234                 int found = 0;
00235                 for (int i = 0; i < command_count; i++)
00236                 {
00237                     if (!(commands[i].case_mask & case_flags))
00238                         continue; // command not available in this mode
```

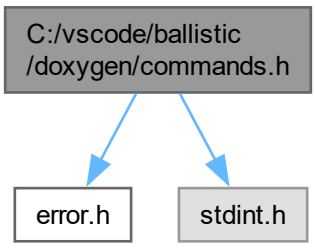
```
00239
00240                     if (strcmp(cmd, commands[i].name) == 0)
00241                     {
00242                         found = 1;
00243
00244                         if (commands[i].type != CMD_HELP) // so help doesn't trigger the finsih
      command
00245                             modal = 0;
00246
00247                         if (case_flags & CMD_C_GENERIC)
00248                             command = commands[i].type;
00249
00250                         if (commands[i].handler)
00251                             commands[i].handler(args, case_flags, running);
00252
00253                         command = commands[i].type;
00254                     }
00255                 }
00256                 if (!found)
00257                 {
00258                     for (int i = 0; i < command_count; i++)
00259                     {
00260                         if (commands[i].type != CMD_HELP) // seraches for the help flag,
00261                             continue;                     // to print out it's current name (only for
      keeping it dynamic)
00262                         printf("Unknown command: [%s]\n If you need help, use the command [%s]\n",
      cmd, commands[i].name);
00263
00264                         if (!(case_flags & CMD_C_LOOP))
00265                             return CMD_INVALID;
00266                     }
00267                 }
00268             }
00269         }
00270     }
00271     return command;
00272 }
00273
00274 Command_t ParseCommandFromBuffer(char *input, Command_case case_flags, PROCESS *running)
00275 {
00276     NormalizeInput(input);
00277
00278     char *cmd  = strtok(input, " \t\n");
00279     char *args = strtok(NULL, "\n");
00280
00281     if (!cmd)
00282         return CMD_INVALID;
00283
00284     int found = 0;
00285
00286     for (int i = 0; i < command_count; i++)
00287     {
00288         if (!(commands[i].case_mask & case_flags))
00289             continue; // command not available in this mode
00290
00291         if (strcmp(cmd, commands[i].name) == 0)
00292         {
00293             found = 1;
00294
00295             if (commands[i].handler)
00296                 commands[i].handler(args, case_flags, running);
00297
00298             return commands[i].type;
00299         }
00300     }
00301
00302     if (!found)
00303     {
00304         // Only show help for commands valid in this mode
00305         for (int i = 0; i < command_count; i++)
00306         {
00307             if (!(commands[i].case_mask & case_flags))
00308                 continue;
00309
00310             if (commands[i].type == CMD_HELP)
00311             {
00312                 printf("Unknown command: [%s]. Use [%s] for help.\n", cmd, commands[i].name);
00313                 break;
00314             }
00315         }
00316     }
00317
00318     return CMD_INVALID;
00319 }
00320
00321
00322
```

```
00323 // later make it enum dependent
00324 void add_seperator(const char *str, char symbol, unsigned sep_length)
00325 {
00326     size_t str_length = (str) ? strlen(str) : 0;
00327     int start = (sep_length - str_length) / 2;
00328     int cnt = 0;
00329
00330     for (cnt = 0; cnt < start; cnt++)
00331     {
00332         printf("%c", symbol);
00333     };
00334     if (str)
00335         printf("%s", str);
00336     for (cnt += str_length; cnt < sep_length; cnt++)
00337     {
00338         printf("%c", symbol);
00339     };
00340     printf("\n");
00341 }
00342
00343 void clear_terminal()
00344 {
00345     if (get_color_mode() == CM_COLOUR)
00346         printf("\033[2J\033[H");
00347 };
```

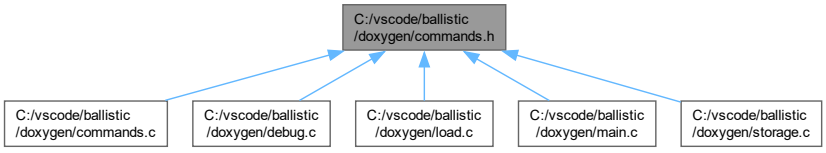
10.11 C:/vscode/ballistic/doxygen/commands.h File Reference

```
#include "error.h"
#include <stdint.h>
```

Include dependency graph for commands.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct Command

  Command definition structure.

Macros

- #define CMD_C_ALL 0xFFFFFFFFu

  All command contexts combined.

Enumerations

- enum Command_case {
  CMD_C_DEFAULT = 1 << 0 , CMD_C_GENERIC = 1 << 1 , CMD_C_SYSTEM = 1 << 2 ,
  CMD_C_YESNO = 1 << 3 ,
  CMD_C_LOAD = 1 << 4 , CMD_C_SETUP = 1 << 5 , CMD_C_EXIT = 1 << 6 , CMD_C_CALC = 1 << 7
  ,
  CMD_C_SELECT = 1 << 8 , CMD_C_BACK = 1 << 9 , CMD_C_CANCEL = 1 << 10 , CMD_C_LOOP =
  1 << 20 }

  Command context flags for modal operation.
- enum Command_t {
  CMD_INVALID , CMD_EXIT , CMD_HELP , CMD_CONTINUE ,
  CMD_YES , CMD_NO , CMD_SAVE , CMD_CANCEL ,
  CMD_SELECT , CMD_BACK , CMD_CALCULATE , CMD_RESET ,
  CMD_TARGET , CMD_BASE , CMD_AMMO , CMD_TEST ,
  CMD_LIST , CMD_ADD , CMD_DELETE , CMD_COLOUR ,
  CMD_MONO , CMD_CLEAR_TERMINAL }

  Command type identifiers.

Functions

Core Command Functions

- void add_seperator (const char *str, char symbol, unsigned sep_length)

  Print formatted separator line.
- void clear_terminal ()

  Clear terminal screen.
- Command_t GetCommand (Command_case case_flags, PROCESS *running)

  Get command from user with modal context.
- void NormalizeInput (char *input)

  Normalize input string to lowercase.
- Command_t ParseCommandFromBuffer (char *input, Command_case case_flags, PROCESS *running)

  Parse command from buffer (non-interactive).

Command Handler Functions

Each handler corresponds to a specific command. Handlers receive arguments string and current context flags.

- void cmd_ammo (const char *args, Command_case case_flags, PROCESS *running)

  Ammunition operations handler.
- void cmd_base (const char *args, Command_case case_flags, PROCESS *running)

  Base operations handler.
- void cmd_calculate (const char *args, Command_case case_flags, PROCESS *running)

  Calculate trajectory handler.
- void cmd_clr_ter (const char *args, Command_case case_flags, PROCESS *running)

  Clear terminal handler.
- void cmd_colour (const char *args, Command_case case_flags, PROCESS *running)

  Set color mode handler.
- void cmd_config (const char *args, Command_case case_flags, PROCESS *running)

  Configuration handler.
- void cmd_exit (const char *args, Command_case case_flags, PROCESS *running)

Exit program handler.

- void cmd_help (const char *args, Command_case case_flags, PROCESS *running)

  Help command handler.
- void cmd_mono (const char *args, Command_case case_flags, PROCESS *running)

  Set monochrome mode handler.
- void cmd_target (const char *args, Command_case case_flags, PROCESS *running)

  Target operations handler.
- void cmd_test (const char *args, Command_case case_flags, PROCESS *running)

  Test system handler.

Variables

- int command_count

  Number of commands in command table.
- Command commands [ ]

  Global command table.

10.12 commands.h

Go to the documentation of this file.

```
00001
00008
00009 #ifndef COMMANDS_H
00010 #define COMMANDS_H
00011
00012 #include "error.h"
00013 #include <stdint.h>
00014
00019 typedef enum
00020 {
00021     CMD_INVALID,
00022     CMD_EXIT,
00023     CMD_HELP,
00024     CMD_CONTINUE,
00025     CMD_YES,
00026     CMD_NO,
00027     CMD_SAVE,
00028     CMD_CANCEL,
00029     CMD_SELECT,
00030     CMD_BACK,
00031     CMD_CALCULATE,
00032     CMD_RESET,
00033     CMD_TARGET,
00034     CMD_BASE,
00035     CMD_AMMO,
00036     CMD_TEST,
00037     CMD_LIST,
00038     CMD_ADD,
00039     CMD_DELETE,
00040     CMD_COLOUR,
00041     CMD_MONO,
00042     CMD_CLEAR_TERMINAL
00043 } Command_t;
00044
00051 typedef enum
00052 {
00053     CMD_C_DEFAULT = 1 << 0,
00054     CMD_C_GENERIC = 1 << 1,
00055     CMD_C_SYSTEM = 1 << 2,
00056     CMD_C_YESNO = 1 << 3,
00057     CMD_C_LOAD = 1 << 4,
00058     CMD_C_SETUP = 1 << 5,
00059     CMD_C_EXIT = 1 << 6,
00060     CMD_C_CALC = 1 << 7,
00061     CMD_C_SELECT = 1 << 8,
00062     CMD_C_BACK = 1 << 9,
00063     CMD_C_CANCEL = 1 << 10,
00064     CMD_C_LOOP = 1 << 20
00065 } Command_case;
00066
00071 #define CMD_C_ALL 0xFFFFFFFFu
```

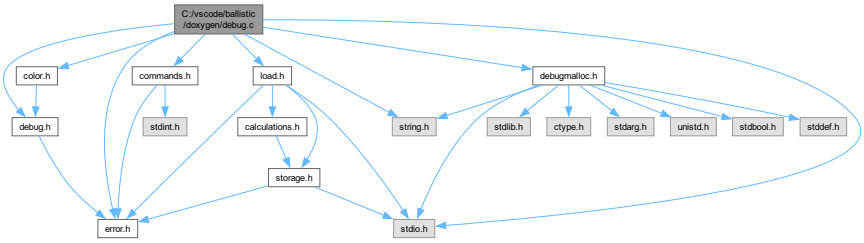
```
00072
00077 typedef struct
00078 {
00079     const char *name;
00080     Command_t type;
00081     void (*handler)(const char *args, Command_case case_flags, PROCESS *running);
00082     const char *description;
00083     uint32_t case_mask;
00084 } Command;
00085
00090
00106 Command_t GetCommand(Command_case case_flags, PROCESS *running);
00107
00120 Command_t ParseCommandFromBuffer(char *input, Command_case case_flags, PROCESS *running);
00121
00127 void NormalizeInput(char *input);
00128
00136 void add_seperator(const char *str, char symbol, unsigned sep_length);
00137
00144 void clear_terminal();
00145
00147
00155
00163 void cmd_exit(const char *args, Command_case case_flags, PROCESS *running);
00164
00172 void cmd_help(const char *args, Command_case case_flags, PROCESS *running);
00173
00181 void cmd_base(const char *args, Command_case case_flags, PROCESS *running);
00182
00190 void cmd_calculate(const char *args, Command_case case_flags, PROCESS *running);
00191
00199 void cmd_ammo(const char *args, Command_case case_flags, PROCESS *running);
00200
00208 void cmd_target(const char *args, Command_case case_flags, PROCESS *running);
00209
00217 void cmd_test(const char *args, Command_case case_flags, PROCESS *running);
00218
00226 void cmd_config(const char *args, Command_case case_flags, PROCESS *running);
00227
00235 void cmd_colour(const char *args, Command_case case_flags, PROCESS *running);
00236
00244 void cmd_mono(const char *args, Command_case case_flags, PROCESS *running);
00245
00253 void cmd_clr_ter(const char *args, Command_case case_flags, PROCESS *running);
00254
00256
00263 extern Command commands[];
00264
00269 extern int command_count;
00270
00271 #endif /* COMMANDS_H */
```

10.13 C:/vscode/ballistic/doxygen/data_structures.dox File Reference

10.14 C:/vscode/ballistic/doxygen/debug.c File Reference

```
#include "debug.h"
#include "error.h"
#include "load.h"
#include "color.h"
#include "commands.h"
#include <stdio.h>
#include <string.h>
#include "debugmalloc.h"
```

Include dependency graph for debug.c:



Functions

- HRESULT run_pre_test (PROCESS process_state)

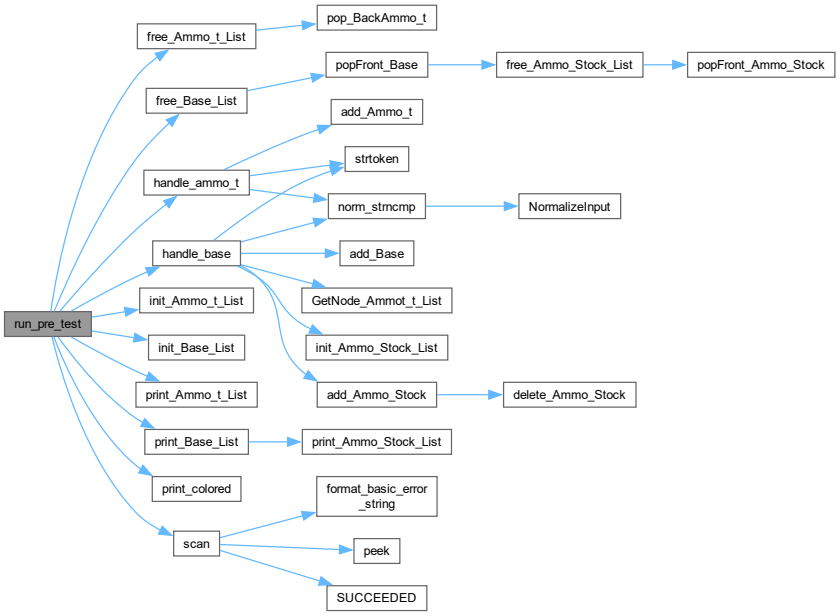
10.14.1 Function Documentation

10.14.1.1 run_pre_test()

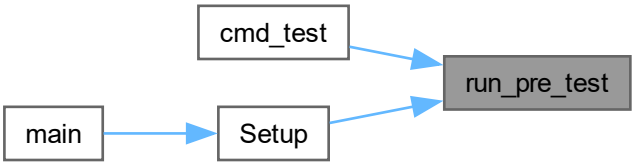
```
HRESULT run_pre_test (
            PROCESS process_state)
```

Definition at line 12 of file debug.c.

Here is the call graph for this function:



Here is the caller graph for this function:



10.15 debug.c

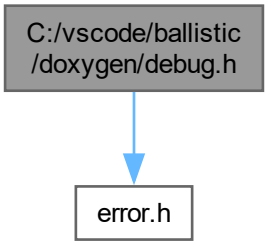
Go to the documentation of this file.

```
00001 #include "debug.h"
00002 #include "error.h"
00003 #include "load.h"
00004 #include "color.h"
00005 #include "commands.h"
00006
00007 #include <stdio.h>
00008 #include <string.h>
00009
00010 #include "debugmalloc.h"
00011
00012 HRESULT run_pre_test(PROCESS process_state)
00013 {
00014     printf("Starting test...\n");
00015
00016     HRESULT hr = S_OK;
00017
00018     char folder_buffer[1024];
00019     strcpy(folder_buffer, folder_path);
00020     strcat(folder_buffer, "ammo_catalouge.txt");
00021     init_Ammo_t_List(&ammo_t_list);
00022     hr = scan(folder_buffer, handle_ammo_t, ammo_t_list, process_state);
00023
00024     strcpy(folder_buffer, folder_path);
00025     strcat(folder_buffer, "bases.txt");
00026
00027     init_Base_List(&base_list);
00028     hr = scan(folder_buffer, handle_base, base_list, process_state);
00029     if (process_state == S_DEBUG)
00030     {
00031         print_Base_List(base_list, process_state);
00032         print_Ammo_t_List(ammo_t_list);
00033
00034         if (print_colored(ANSI_COLOR_RED, "This text should be RED!\n", false) == S_FALSE)
00035         {
00036             printf("This text shouldn't be RED!\n");
00037         }
00038         else
00039             printf("if not, use the following command in the windows terminal, to enable ansi escape
      codes\n\"reg add HKCU\\Console /v VirtualTerminalLevel /t REG_DWORD /d 1 /f\"\n");
00040     }
00041     free_Base_List(&base_list);
00042     free_Ammo_t_List(&ammo_t_list);
00043
00044     printf("---------------------------------\n");
00045     if (hr == S_OK)
00046         print_colored(ANSI_COLOR_GREEN, "Test succeeded!\n", false);
00047     else
00048         print_colored(ANSI_COLOR_RED, "Test failed!\n", true);
00049     printf("---------------------------------\n");
00050
00051     return hr;
00052 }
```

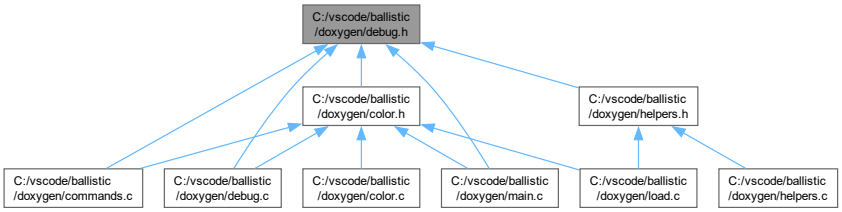
10.16 C:/vscode/ballistic/doxygen/debug.h File Reference

```
#include "error.h"
```

Include dependency graph for debug.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define EMPTY_MESSAGE printf("Nothing to see here! (yet)\n");

Enumerations

- enum bool { false , true }

Functions

- HRESULT run_pre_test (PROCESS process_state)

10.16.1 Macro Definition Documentation

10.16.1.1 EMPTY_MESSAGE

```
#define EMPTY_MESSAGE printf("Nothing to see here!  (yet)\n");
```

Definition at line 6 of file debug.h.

10.16.2 Enumeration Type Documentation

10.16.2.1 bool

```
enum bool
```

Enumerator

false	
true	

Definition at line 8 of file debug.h.

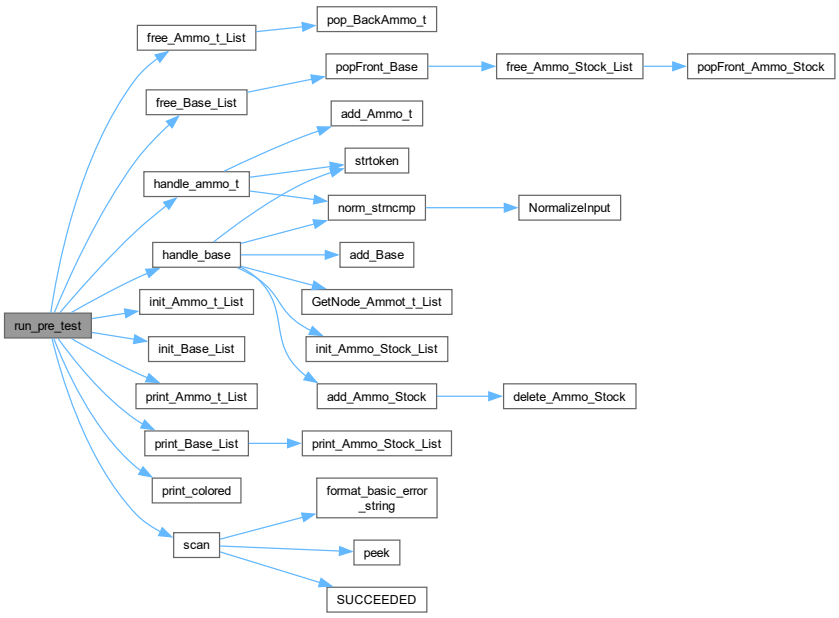
10.16.3 Function Documentation

10.16.3.1 run_pre_test()

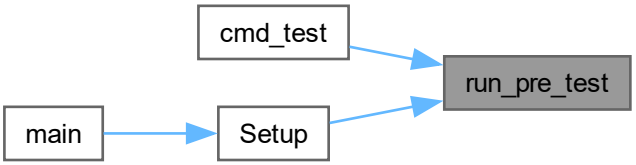
```
HRESULT run_pre_test (
            PROCESS process_state)
```

Definition at line 12 of file debug.c.

Here is the call graph for this function:



Here is the caller graph for this function:



10.17 debug.h

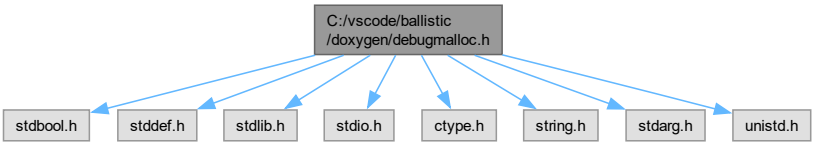
Go to the documentation of this file.

```
00001 #ifndef DEBUG_H
00002 #define DEBUG_H
00003
00004 #include "error.h"
00005
00006 #define EMPTY_MESSAGE printf("Nothing to see here! (yet)\n");
00007
00008 typedef enum
00009 {
00010     false,
00011     true
00012 } bool;
00013
00014 HRESULT run_pre_test(PROCESS process_state);
00015
00016 #endif
```

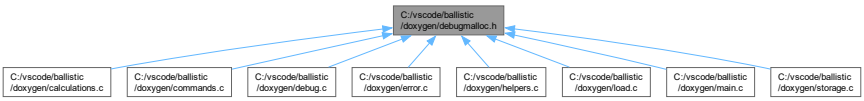
10.18 C:/vscode/ballistic/doxygen/debugmalloc.h File Reference

```
#include <stdbool.h>
#include <stddef.h>
#include <stdlib.h>
#include <stdio.h>
#include <ctype.h>
#include <string.h>
#include <stdarg.h>
#include <unistd.h>
```

Include dependency graph for debugmalloc.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct DebugmallocData
- struct DebugmallocEntry

Macros

- #define calloc(N, S)
- #define free(P)
- #define malloc(S)
- #define realloc(P, S)
- #define strdup(S)
- #define strndup(S, N)

Typedefs

- typedef struct DebugmallocData DebugmallocData
- typedef struct DebugmallocEntry DebugmallocEntry

Enumerations

- enum { debugmalloc_canary_size = 64 , debugmalloc_canary_char = 'K' , debugmalloc_tablesize = 256 , debugmalloc_max_block_size_default = 1048576 }

Functions

- int putenv (char *)

10.18.1 Macro Definition Documentation

10.18.1.1 calloc

```
#define calloc(
              N,
              S)
```

Value:

```
debugmalloc_malloc_full((N)*(S), "calloc", #N ", " #S, __FILE__, __LINE__, true)
```

Definition at line 502 of file debugmalloc.h.

10.18.1.2 free

```
#define free(
              P)
```

Value:

```
debugmalloc_free_full((P), "free", __FILE__, __LINE__)
```

Definition at line 504 of file debugmalloc.h.

10.18.1.3 malloc

```
#define malloc(
              S)
```

Value:

```
debugmalloc_malloc_full((S), "malloc", #S, __FILE__, __LINE__, false)
```

Definition at line 501 of file debugmalloc.h.

10.18.1.4 realloc

```
#define realloc(
              P,
              S)
```

Value:

```
debugmalloc_realloc_full((P), (S), "realloc", #S, __FILE__, __LINE__)
```

Definition at line 503 of file debugmalloc.h.

10.18.1.5 strdup

```
#define strdup(
              S)
```

Value:

```
debugmalloc_strdup((S), "strdup", #S, __FILE__, __LINE__)
```

Definition at line 537 of file debugmalloc.h.

10.18.1.6 strndup

```
#define strndup(
              S,
              N)
```

Value:

```
debugmalloc_strndup((S), (N), "strndup", #S, __FILE__, __LINE__)
```

Definition at line 538 of file debugmalloc.h.

10.18.2 Typedef Documentation

10.18.2.1 DebugmallocData

```
typedef struct DebugmallocData DebugmallocData
```

10.18.2.2 DebugmallocEntry

```
typedef struct DebugmallocEntry DebugmallocEntry
```

10.18.3 Enumeration Type Documentation

10.18.3.1 anonymous enum

```
anonymous enum
```

Enumerator

debugmalloc_canary_size	
debugmalloc_canary_char	
debugmalloc_tablesize	
debugmalloc_max_block_size_default	

Definition at line 13 of file debugmalloc.h.

10.18.4 Function Documentation

10.18.4.1 putenv()

```
int putenv (
             char * )
```

10.19 debugmalloc.h

Go to the documentation of this file.

```
00001 #ifndef DEBUGMALLOC_H
00002 #define DEBUGMALLOC_H
00003
00004 #include <stdbool.h>
00005 #include <stddef.h>
00006 #include <stdlib.h>
00007 #include <stdio.h>
00008 #include <ctype.h>
00009 #include <string.h>
00010 #include <stdarg.h>
00011
00012
00013 enum {
00014     /* size of canary in bytes. should be multiple of largest alignment
00015      * required by any data type (usually 8 or 16) */
```

```
00016     debugmalloc_canary_size = 64,
00017
00018     /* canary byte */
00019     debugmalloc_canary_char = 'K',
00020
00021     /* hash table size for allocated entries */
00022     debugmalloc_tablesize = 256,
00023
00024     /* max block size for allocation, can be modified with debugmalloc_max_block_size() */
00025     debugmalloc_max_block_size_default = 1048576
00026 };
00027
00028
00029 /* make getpid and putenv "crossplatform". deprecated on windows but they work just fine,
00030  * however not declared. */
00031 #ifdef _WIN32
00032     /* windows */
00033     #include <process.h>
00034     #ifdef _MSC_VER
00035         #pragma warning(push)
00036         /* visual studio, getenv/getpid deprecated warning */
00037         #pragma warning(disable: 4996)
00038         #pragma warning(disable: 4127)
00039     #else
00040         /* other windows. the declaration is unfortunately hidden
00041          * in mingw header files by ifdefs. */
00042         int putenv(const char *);
00043     #endif
00044 #else
00045     /* posix */
00046     #include <unistd.h>
00047     int putenv(char *);
00048 #endif
00049
00050
00051 /* linked list entry for allocated blocks */
00052 typedef struct DebugmallocEntry {
00053     void *real_mem;     /* the address of the real allocation */
00054     void *user_mem;     /* address shown to the user */
00055     size_t size;        /* size of block requested by user */
00056
00057     char file[64];      /* malloc called in this file */
00058     unsigned line;      /* malloc called at this line in file */
00059     char func[32];      /* allocation function called (malloc, calloc, realloc) */
00060     char expr[128];     /* expression calculating the size of allocation */
00061
00062     struct DebugmallocEntry *prev, *next;  /* for doubly linked list */
00063 } DebugmallocEntry;
00064
00065
00066 /* debugmalloc singleton, storing all state */
00067 typedef struct DebugmallocData {
00068     char logfile[256];    /* log file name or empty string */
00069     long max_block_size;  /* max size of a single block allocated */
00070     long alloc_count;     /* currently allocated; decreased with free */
00071     long long alloc_bytes;
00072     long all_alloc_count; /* all allocations, never decreased */
00073     long long all_alloc_bytes;
00074     DebugmallocEntry head[debugmalloc_tablesize], tail[debugmalloc_tablesize];  /* head and tail
      elements of allocation lists */
00075 } DebugmallocData;
00076
00077
00078 /* this forward declaration is required by the singleton manager function */
00079 static DebugmallocData * debugmalloc_create(void);
00080
00081
00082 /* creates singleton instance. as this function is static included to different
00083  * translation units, multiple instances of the static variables are created.
00084  * to make sure it is really a singleton, these instances must know each other
00085  * somethow. an environment variable is used for that purpose, ie. the address
00086  * of the singleton allocated is stored by the operating system.
00087  * this implementation is not thread-safe. */
00088 static DebugmallocData * debugmalloc_singleton(void) {
00089     static char envstr[100];
00090     static void *instance = NULL;
00091
00092     /* if we do not know the address of the singleton:
00093      * - maybe we are the one to create it (env variable also does not exist)
00094      * - or it is already created, and stored in the env variable. */
00095     if (instance == NULL) {
00096         char envvarname[100] = "";
00097         sprintf(envvarname, "%s%d", "debugmallocsingleton", (int) getpid());
00098         char *envptr = getenv(envvarname);
00099         if (envptr == NULL) {
00100             /* no env variable: create singleton. */
00101             instance = debugmalloc_create();
```

```
00102             sprintf(envstr, "%s=%p", envvarname, instance);
00103             putenv(envstr);
00104         } else {
00105             /* another copy of this function already created it. */
00106             int ok = sscanf(envptr, "%p", &instance);
00107             if (ok != 1) {
00108                 fprintf(stderr, "debugmalloc: nem lehet ertelmezni: %s!\n", envptr);
00109                 abort();
00110             }
00111         }
00112     }
00113
00114     return (DebugmallocData *) instance;
00115 }
00116
00117
00118 /* better version of strncpy, always terminates string with \0. */
00119 static void debugmalloc_strlcpy(char *dest, char const *src, size_t destsize) {
00120     strncpy(dest, src, destsize-1); /* 03.09.2025 by KZs: destsize changed to destsize-1 according to
      warnings*/
00121     dest[destsize - 1] = '\0';
00122 }
00123
00124
00125 /* set the name of the log file for debugmalloc. empty filename
00126  * means logging to stderr. */
00127 static void debugmalloc_log_file(char const *logfilename) {
00128     if (logfilename == NULL)
00129         logfilename = "";
00130     DebugmallocData *instance = debugmalloc_singleton();
00131     debugmalloc_strlcpy(instance->logfile, logfilename, sizeof(instance->logfile));
00132 }
00133
00134
00135 /* set the maximum size of one block. useful for debugging purposes. */
00136 static void debugmalloc_max_block_size(long max_block_size) {
00137     DebugmallocData *instance = debugmalloc_singleton();
00138     instance->max_block_size = max_block_size;
00139 }
00140
00141
00142
00143 /* printf to the log file, or stderr. */
00144 static void debugmalloc_log(char const *format, ...) {
00145     DebugmallocData *instance = debugmalloc_singleton();
00146     FILE *f = stderr;
00147     if (instance->logfile[0] != '\0') {
00148         f = fopen(instance->logfile, "at");
00149         if (f == NULL) {
00150             f = stderr;
00151             fprintf(stderr, "debugmalloc: nem tudom megnyitni a %s fajlt irasra!\n",
      instance->logfile);
00152             debugmalloc_strlcpy(instance->logfile, "", sizeof(instance->logfile));
00153         }
00154     }
00155
00156     va_list ap;
00157     va_start(ap, format);
00158     vfprintf(f, format, ap);
00159     va_end(ap);
00160
00161     if (f != stderr)
00162         fclose(f);
00163 }
00164
00165
00166 /* initialize a memory block allocated for the user. the start and the end
00167  * of the block is initialized with the canary characters. if 'zero' is
00168  * true, the user memory area is zero-initialized, otherwise it is also
00169  * filled with the canary character to simulate garbage in memory. */
00170 static void debugmalloc_memory_init(DebugmallocEntry *elem, bool zero) {
00171     unsigned char *real_mem = (unsigned char *) elem->real_mem;
00172     unsigned char *user_mem = (unsigned char *) elem->user_mem;
00173     unsigned char *canary1 = real_mem;
00174     unsigned char *canary2 = real_mem + debugmalloc_canary_size + elem->size;
00175     memset(canary1, debugmalloc_canary_char, debugmalloc_canary_size);
00176     memset(canary2, debugmalloc_canary_char, debugmalloc_canary_size);
00177     memset(user_mem, zero ? 0 : debugmalloc_canary_char, elem->size);
00178 }
00179
00180 /* check canary, return true if ok, false if corrupted. */
00181 static bool debugmalloc_canary_ok(DebugmallocEntry const *elem) {
00182     unsigned char *real_mem = (unsigned char *) elem->real_mem;
00183     unsigned char *canary1 = real_mem;
00184     unsigned char *canary2 = real_mem + debugmalloc_canary_size + elem->size;
00185     for (size_t i = 0; i < debugmalloc_canary_size; ++i) {
00186         if (canary1[i] != debugmalloc_canary_char)
```

```
00187             return false;
00188         if (canary2[i] != debugmalloc_canary_char)
00189             return false;
00190     }
00191     return true;
00192 }
00193 
00194 
00195 /* dump memory contents to log file. */
00196 static void debugmalloc_dump_memory(char const *mem, size_t size) {
00197     for (unsigned y = 0; y < (size + 15) / 16; y++) {
00198         char line[80];
00199         int pos = 0;
00200         pos += sprintf(line + pos, "      %04x  ", y * 16);
00201         for (unsigned x = 0; x < 16; x++) {
00202             if (y * 16 + x < size)
00203                 pos += sprintf(line + pos, "%02x ", (unsigned char)mem[y * 16 + x]);
00204             else
00205                 pos += sprintf(line + pos, "   ");
00206         }
00207         pos += sprintf(line + pos, "  ");
00208         for (unsigned x = 0; x < 16; x++) {
00209             if (y * 16 + x < size) {
00210                 unsigned char c = mem[y * 16 + x];
00211                 pos += sprintf(line + pos, "%c", isprint(c) ? c : '.');
00212             }
00213             else {
00214                 pos += sprintf(line + pos, " ");
00215             }
00216         }
00217         debugmalloc_log("%s\n", line);
00218     }
00219 }
00220 
00221 
00222 /* dump data of allocated memory block.
00223  * if the canary is corrupted, it is also written to the log. */
00224 static void debugmalloc_dump_elem(DebugmallocEntry const *elem) {
00225     bool canary_ok = debugmalloc_canary_ok(elem);
00226 
00227     debugmalloc_log("  %p, %u bajt, kanari: %s\n"
00228                     "  %s:%u, %s(%s)\n",
00229                     elem->user_mem, (unsigned) elem->size, canary_ok ? "ok" : "**SERULT**",
00230                     elem->file, elem->line,
00231                     elem->func, elem->expr);
00232 
00233     if (!canary_ok) {
00234         debugmalloc_log("    ELOTTE kanari: \n");
00235         debugmalloc_dump_memory((char const *) elem->real_mem, debugmalloc_canary_size);
00236     }
00237 
00238     debugmalloc_dump_memory((char const *) elem->user_mem, elem->size > 64 ? 64 : elem->size);
00239 
00240     if (!canary_ok) {
00241         debugmalloc_log("    UTANA kanari: \n");
00242         debugmalloc_dump_memory((char const *) elem->real_mem + debugmalloc_canary_size + elem->size,
    debugmalloc_canary_size);
00243     }
00244 }
00245 
00246 
00247 /* dump data of all memory blocks allocated. */
00248 static void debugmalloc_dump(void) {
00249     DebugmallocData *instance = debugmalloc_singleton();
00250     debugmalloc_log("** DEBUGMALLOC DUMP **********************************\n");
00251     int cnt = 0;
00252     for (size_t i = 0; i < debugmalloc_tablesize; i++) {
00253         DebugmallocEntry *head = &instance->head[i];
00254         for (DebugmallocEntry *iter = head->next; iter->next != NULL; iter = iter->next) {
00255             ++cnt;
00256             debugmalloc_log("** %d/%d. rekord:\n", cnt, instance->alloc_count);
00257             debugmalloc_dump_elem(iter);
00258         }
00259     }
00260     debugmalloc_log("** DEBUGMALLOC DUMP VEGE ******************************\n");
00261 }
00262 
00263 
00264 /* called at program exit to dump data if there is a leak,
00265  * ie. allocated block remained. */
00266 static void debugmalloc_atexit_dump(void) {
00267     DebugmallocData *instance = debugmalloc_singleton();
00268 
00269     if (instance->alloc_count > 0) {
00270         debugmalloc_log("\n"
00271                         "******************************************************\n"
00272                         "* MEMORIASZIVARGAS VAN A PROGRAMBAN!!!\n"
```

```
00273                         "*************************************************\n"
00274                         "\n");
00275         debugmalloc_dump();
00276     } else {
00277         debugmalloc_log("*************************************************\n"
00278                         "* Debugmalloc: nincs memoriaszivargas a programban.\n"
00279                         "* Osszes foglalas: %d blokk, %d bajt.\n"
00280                         "*************************************************\n",
00281                         instance->all_alloc_count, instance->all_alloc_bytes);
00282     }
00283 }
00284 
00285 
00286 /* hash function for bucket hash. */
00287 static size_t debugmalloc_hash(void *address) {
00288     /* the last few bits are ignored, as they are usually zero for
00289      * alignment purposes. all tested architectures used 16 byte allocation. */
00290     size_t cut = (size_t)address >> 4;
00291     return cut % debugmalloc_tablesize;
00292 }
00293 
00294 
00295 /* insert element to hash table. */
00296 static void debugmalloc_insert(DebugmallocEntry *entry) {
00297     DebugmallocData *instance = debugmalloc_singleton();
00298     size_t idx = debugmalloc_hash(entry->user_mem);
00299     DebugmallocEntry *head = &instance->head[idx];
00300     entry->prev = head;
00301     entry->next = head->next;
00302     head->next->prev = entry;
00303     head->next = entry;
00304     instance->alloc_count += 1;
00305     instance->alloc_bytes += entry->size;
00306     instance->all_alloc_count += 1;
00307     instance->all_alloc_bytes += entry->size;
00308 }
00309 
00310 
00311 /* remove element from hash table */
00312 static void debugmalloc_remove(DebugmallocEntry *entry) {
00313     DebugmallocData *instance = debugmalloc_singleton();
00314     entry->next->prev = entry->prev;
00315     entry->prev->next = entry->next;
00316     instance->alloc_count -= 1;
00317     instance->alloc_bytes -= entry->size;
00318 }
00319 
00320 
00321 /* find element in hash table, given with the memory address that the user sees.
00322  * @return the linked list entry, or null if not found. */
00323 static DebugmallocEntry *debugmalloc_find(void *mem) {
00324     DebugmallocData *instance = debugmalloc_singleton();
00325     size_t idx = debugmalloc_hash(mem);
00326     DebugmallocEntry *head = &instance->head[idx];
00327     for (DebugmallocEntry *iter = head->next; iter->next != NULL; iter = iter->next)
00328         if (iter->user_mem == mem)
00329             return iter;
00330     return NULL;
00331 }
00332 
00333 
00334 /* allocate memory. this function is called via the macro. */
00335 static void *debugmalloc_malloc_full(size_t size, char const *func, char const *expr, char const
      *file, unsigned line, bool zero) {
00336     /* imitate standard malloc: return null if size is zero */
00337     if (size == 0)
00338         return NULL;
00339 
00340     /* check max size */
00341     DebugmallocData *instance = debugmalloc_singleton();
00342     if (size > (size_t)(instance->max_block_size)) {
00343         debugmalloc_log("debugmalloc: %s @ %s:%u: a blokk merete tul nagy, %u bajt;
      debugmalloc_max_block_size() fuggvennyel novelheto.\n", func, file, line, (unsigned) size);
00344         abort();
00345     }
00346 
00347     /* allocate more memory, make room for canary */
00348     void *real_mem = malloc(size + 2 * debugmalloc_canary_size);
00349     if (real_mem == NULL) {
00350         debugmalloc_log("debugmalloc: %s @ %s:%u: nem sikerult %u meretu memoriat foglalni!\n", func,
      file, line, (unsigned) size);
00351         return NULL;
00352     }
00353 
00354     /* allocate memory for linked list element */
00355     DebugmallocEntry *newentry = (DebugmallocEntry *) malloc(sizeof(DebugmallocEntry));
00356     if (newentry == NULL) {
```

```
00357         free(real_mem);
00358         debugmalloc_log("debugmalloc: %s @ %s:%u: le tudtam foglalni %u memoriat, de utana a sajatnak
      nem, sry\n", func, file, line, (unsigned) size);
00359         abort();
00360     }
00361
00362     /* metadata of allocation: caller function, code line etc. */
00363     debugmalloc_strlcpy(newentry->func, func, sizeof(newentry->func));
00364     debugmalloc_strlcpy(newentry->expr, expr, sizeof(newentry->expr));
00365     debugmalloc_strlcpy(newentry->file, file, sizeof(newentry->file));
00366     newentry->line = line;
00367
00368     /* address of allocated memory chunk */
00369     newentry->real_mem = real_mem;
00370     newentry->user_mem = (unsigned char *) real_mem + debugmalloc_canary_size;
00371     newentry->size = size;
00372     debugmalloc_memory_init(newentry, zero);
00373
00374     /* store in list and return pointer to user area */
00375     debugmalloc_insert(newentry);
00376     return newentry->user_mem;
00377 }
00378
00379
00380 /* free memory and remove list item. before deleting, the chuck is filled with
00381  * the canary byte to make sure that the user will see garbage if the memory
00382  * is accessed after freeing. */
00383 static void debugmalloc_free_inner(DebugmallocEntry *deleted) {
00384     debugmalloc_remove(deleted);
00385
00386     /* fill with garbage, then remove from linked list */
00387     memset(deleted->real_mem, debugmalloc_canary_char, deleted->size + 2 * debugmalloc_canary_size);
00388     free(deleted->real_mem);
00389     free(deleted);
00390 }
00391
00392
00393 /* free memory - called via the macro.
00394  * as all allocations are tracked in the list, this function can terminate the program
00395  * if a block is freed twice or the free function is called with an invalid address. */
00396 static void debugmalloc_free_full(void *mem, char const *func, char const *file, unsigned line) {
00397     /* imitate standard free function: if ptr is null, no operation is performed */
00398     if (mem == NULL)
00399         return;
00400
00401     /* find allocation, abort if not found */
00402     DebugmallocEntry *deleted = debugmalloc_find(mem);
00403     if (deleted == NULL) {
00404         debugmalloc_log("debugmalloc: %s @ %s:%u: olyan teruletet probalsz felszabaditani, ami nincs
      lefoglalva!\n", func, file, line);
00405         abort();
00406     }
00407
00408     /* check canary and then free memory */
00409     if (!debugmalloc_canary_ok(deleted)) {
00410         debugmalloc_log("debugmalloc: %s @ %s:%u: a %p memoriateruletet tulindexelted!\n", func, file,
      line, mem);
00411         debugmalloc_dump_elem(deleted);
00412     }
00413     debugmalloc_free_inner(deleted);
00414 }
00415
00416
00417 /* realloc-like function. */
00418 static void *debugmalloc_realloc_full(void *oldmem, size_t newsize, char const *func, char const
      *expr, char const *file, unsigned line) {
00419     /* imitate standard realloc: equivalent to free if size is null. */
00420     if (newsize == 0) {
00421         debugmalloc_free_full(oldmem, func, file, line);
00422         return NULL;
00423     }
00424     /* imitate standard realloc: equivalent to malloc if first param is NULL */
00425     if (oldmem == NULL)
00426         return debugmalloc_malloc_full(newsize, func, expr, file, line, 0);
00427
00428     /* find old allocation. abort if not found. */
00429     DebugmallocEntry *oldentry = debugmalloc_find(oldmem);
00430     if (oldentry == NULL) {
00431         debugmalloc_log("debugmalloc: %s @ %s:%u: olyan teruletet probalsz atmeretezni, ami nincs
      lefoglalva!\n", func, file, line);
00432         abort();
00433     }
00434
00435     /* create new allocation, copy & free old data */
00436     void *newmem = debugmalloc_malloc_full(newsize, func, expr, file, line, false);
00437     if (newmem == NULL) {
00438         debugmalloc_log("debugmalloc: %s @ %s:%u: nem sikerult uj memoriat foglalni az
```

```
      atmeretezeshez!\n", func, file, line);
00439         /* imitate standard realloc: original block is untouched, but return NULL */
00440         return NULL;
00441     }
00442     size_t smaller = oldentry->size < newsize ? oldentry->size : newsize;
00443     memcpy(newmem, oldmem, smaller);
00444     debugmalloc_free_inner(oldentry);
00445
00446     return newmem;
00447 }
00448
00449
00450 /* initialize debugmalloc singleton. returns the newly allocated instance */
00451 static DebugmallocData * debugmalloc_create(void) {
00452     /* config check */
00453     if (debugmalloc_canary_size % 16 != 0) {
00454         debugmalloc_log("debugmalloc: a kanari merete legyen 16-tal oszthato\n");
00455         abort();
00456     }
00457     if (debugmalloc_canary_char == 0) {
00458         debugmalloc_log("debugmalloc: a kanari legyen 0-tol kulonbozo\n");
00459         abort();
00460     }
00461     /* avoid compiler warning if these functions are not used */
00462     (void) debugmalloc_realloc_full;
00463     (void) debugmalloc_log_file;
00464     (void) debugmalloc_max_block_size;
00465
00466     /* create and initialize instance */
00467     DebugmallocData *instance = (DebugmallocData *) malloc(sizeof(DebugmallocData));
00468     if (instance == NULL) {
00469         debugmalloc_log("debugmalloc: nem sikerult elinditani a memoriakezelest\n");
00470         abort();
00471     }
00472     debugmalloc_strlcpy(instance->logfile, "", sizeof(instance->logfile));
00473     instance->max_block_size = debugmalloc_max_block_size_default;
00474     instance->alloc_count = 0;
00475     instance->alloc_bytes = 0;
00476     instance->all_alloc_count = 0;
00477     instance->all_alloc_bytes = 0;
00478     for (size_t i = 0; i < debugmalloc_tablesize; i++) {
00479         instance->head[i].prev = NULL;
00480         instance->head[i].next = &instance->tail[i];
00481         instance->tail[i].next = NULL;
00482         instance->tail[i].prev = &instance->head[i];
00483     }
00484
00485     atexit(debugmalloc_atexit_dump);
00486     return instance;
00487 }
00488
00489
00490 /* These macro-like functions forward all allocation/free
00491  * calls to debugmalloc. Usage is the same, malloc(size)
00492  * gives the address of a new memory block, free(ptr)
00493  * deallocates etc.
00494  *
00495  * If you use this file, make sure that you include this
00496  * in *ALL* translation units (*.c) of your source. The
00497  * builtin free() function cannot deallocate a memory block
00498  * that was allocated via debugmalloc, yet the name of
00499  * the function is the same! */
00500
00501 #define malloc(S) debugmalloc_malloc_full((S), "malloc", #S, __FILE__, __LINE__, false)
00502 #define calloc(N,S) debugmalloc_malloc_full((N)*(S), "calloc", #N ", " #S, __FILE__, __LINE__, true)
00503 #define realloc(P,S) debugmalloc_realloc_full((P), (S), "realloc", #S, __FILE__, __LINE__)
00504 #define free(P) debugmalloc_free_full((P), "free", __FILE__, __LINE__)
00505
00506 /* To include strdup and strndup functions (since c20)
00507  * in debugmalloc administration mechanism. */
00508
00509 static inline char * debugmalloc_strdup (const char *s, char const *func, char const *expr, char const
      *file, unsigned line)
00510 {
00511   size_t len = strlen (s) + 1;
00512   void *new = debugmalloc_malloc_full (len, func, expr, file, line, 0);
00513
00514   if (new == NULL)
00515     return NULL;
00516
00517   return (char *) memcpy (new, s, len);
00518 }
00519
00520
00521 static inline char * debugmalloc_strndup (const char *s, size_t n, char const *func, char const *expr,
      char const *file, unsigned line)
00522 {
```

```
00523   size_t len = strlen (s);
00524
00525   if (n < len)
00526     len = n;
00527
00528   void *new = debugmalloc_malloc_full (len+1, func, expr, file, line, 0);
00529
00530   if (new == NULL)
00531     return NULL;
00532
00533   ((char*)new)[len] = '\0';
00534   return (char *) memcpy (new, s, len);
00535 }
00536
00537 #define strdup(S) debugmalloc_strdup((S), "strdup", #S, __FILE__, __LINE__)
00538 #define strndup(S,N) debugmalloc_strndup((S), (N), "strndup", #S, __FILE__, __LINE__)
00539
00540 #if defined(_WIN32) && defined(_MSC_VER)
00541     #pragma warning(pop)
00542 #endif
00543
00544 #endif
```

10.20 C:/vscode/ballistic/doxygen/doxygen_main.h File Reference

10.21 doxygen_main.h

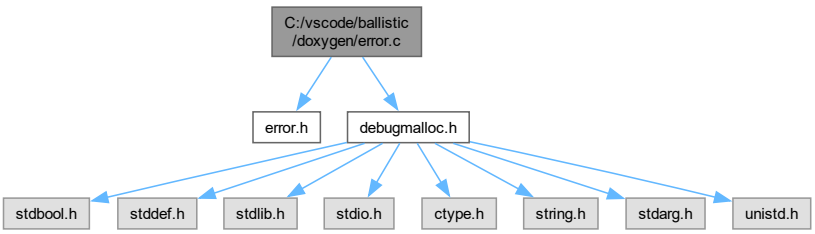
Go to the documentation of this file.

```
00001
```

10.22 C:/vscode/ballistic/doxygen/error.c File Reference

```
#include "error.h"
#include "debugmalloc.h"
```

Include dependency graph for error.c:



Functions

- int FAILED (HRESULT hr)

  Check if result indicates failure.
- int SUCCEEDED (HRESULT hr)

  Check if result indicates success.

10.23 error.c

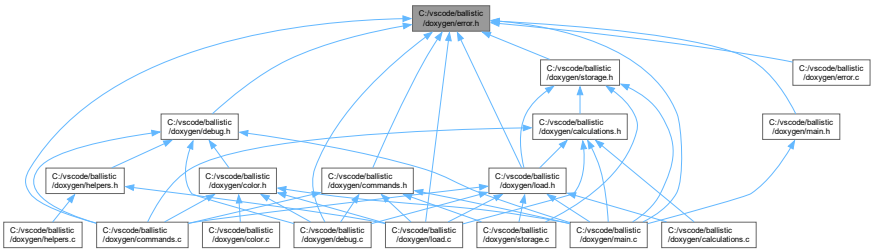
Go to the documentation of this file.

```
00001 #include "error.h"
00002 #include "debugmalloc.h"
00003
00004 int SUCCEEDED(HRESULT hr){
00005     return (hr >= 0) ? 1 : 0;
00006 }
00007
00008 int FAILED(HRESULT hr){
00009     return (hr < 0) ? 1 : 0;
00010 }
```

10.24 C:/vscode/ballistic/doxygen/error.h File Reference

Error codes and process states.

This graph shows which files directly or indirectly include this file:



Enumerations

- enum HRESULT { S_OK = 0 , S_FALSE = 1 , S_PARTIAL = 2 , E_FAIL = -1 }

  Result codes (similar to Windows HRESULT).
- enum PROCESS { S_RUNNING , S_END , S_TERMINATE , S_DEBUG }

  Program process states.

Functions

- int FAILED (HRESULT hr)

  Check if result indicates failure.
- int SUCCEEDED (HRESULT hr)

  Check if result indicates success.

10.24.1 Detailed Description

Error codes and process states.

Definition in file error.h.

10.25 error.h

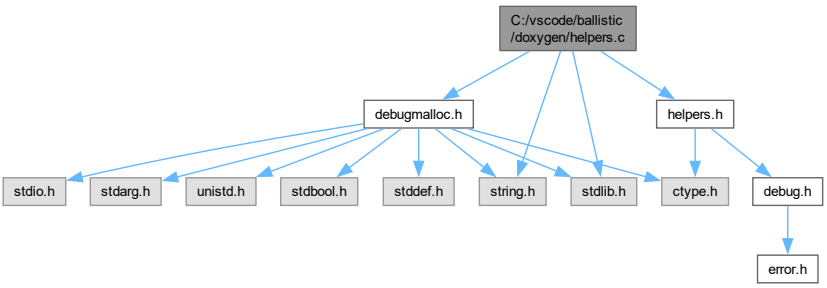
Go to the documentation of this file.

```
00001
00007
00013
00014 #ifndef ERROR_H
00015 #define ERROR_H
00016
00021 typedef enum
00022 {
00023     S_OK = 0,
00024     S_FALSE = 1,
00025     S_PARTIAL = 2,
00026     E_FAIL = -1
00027 } HRESULT;
00028
00033 typedef enum
00034 {
00035     S_RUNNING,
00036     S_END,
00037     S_TERMINATE,
00038     S_DEBUG
00039 } PROCESS;
00040
00047 int SUCCEEDED(HRESULT hr);
00048
00055 int FAILED(HRESULT hr);
00056
00057 #endif /* ERROR_H */
```

10.26 C:/vscode/ballistic/doxygen/helpers.c File Reference

```
#include "helpers.h"
#include <string.h>
#include <stdlib.h>
#include "debugmalloc.h"
```

Include dependency graph for helpers.c:



Functions

- char * ensure_trailing_backslash (char *str)
- HRESULT norm_strncmp (const char *record, const char *str2)
- char * Normalize_path (char *path)
- void NormalizeInput (char *input)

  Normalize input string to lowercase.
- char * remove_quotes (char *str, char symbol)
- char * strtoken (char *str, char **endptr)

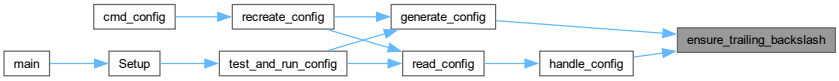
10.26.1 Function Documentation

10.26.1.1 ensure_trailing_backslash()

```
char * ensure_trailing_backslash (
            char * str)
```

Definition at line 8 of file helpers.c.

Here is the caller graph for this function:

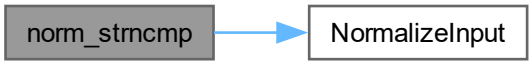


10.26.1.2 norm_strncmp()

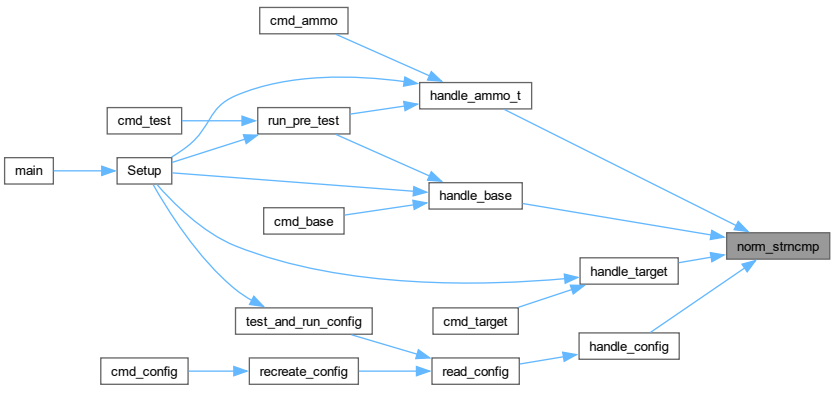
```
HRESULT norm_strncmp (
            const char * record,
            const char * str2)
```

Definition at line 83 of file helpers.c.

Here is the call graph for this function:



Here is the caller graph for this function:

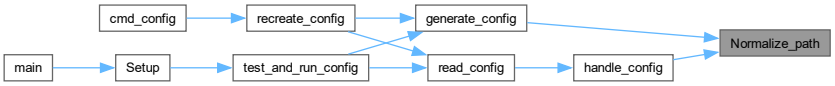


10.26.1.3 Normalize_path()

```
char * Normalize_path (
            char * path)
```

Definition at line 20 of file helpers.c.

Here is the caller graph for this function:

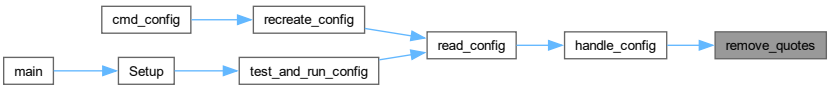


10.26.1.4 remove_quotes()

```
char * remove_quotes (
            char * str,
            char symbol)
```

Definition at line 65 of file helpers.c.

Here is the caller graph for this function:

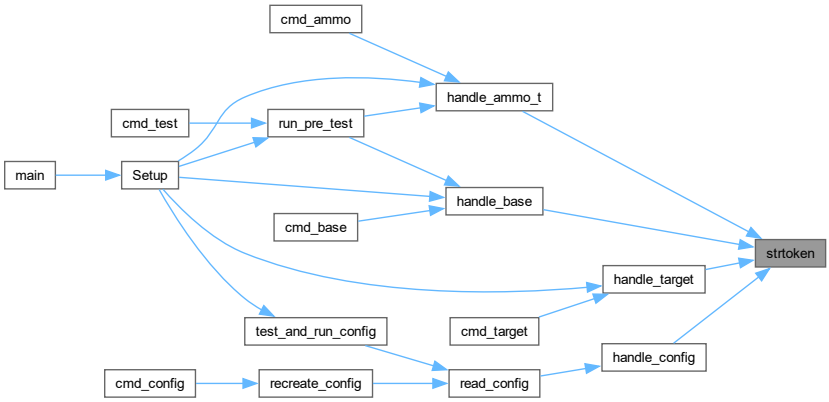


10.26.1.5 strtoken()

```
char * strtoken (
            char * str,
            char ** endptr)
```

Definition at line 49 of file helpers.c.

Here is the caller graph for this function:



10.27 helpers.c

Go to the documentation of this file.

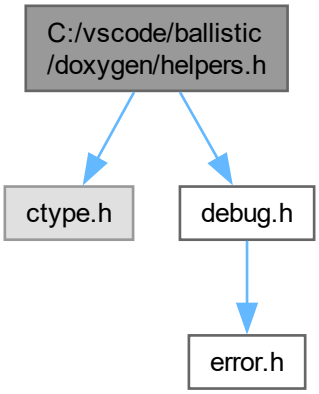
```
00001 #include "helpers.h"
00002
00003 #include <string.h>
00004 #include <stdlib.h>
00005
00006 #include "debugmalloc.h"
00007
00008 char *ensure_trailing_backslash(char *str)
00009 {
00010     size_t len = strlen(str);
00011
00012     if (len == 0 || str[len - 1] != '\\')
00013     {
00014         str[len] = '\\';     // place backslash before null terminator
00015         str[len + 1] = '\0'; // new null terminator
00016     }
00017     return str;
00018 }
00019
00020 char *Normalize_path(char *path)
00021 {
00022     if (!path)
00023         return NULL;
00024
00025     int offset = 0;
00026
00027     for (int i = 0; path[i] != '\0'; i++)
00028     {
00029
00030         for (int j = 0; (path[i] == '/' || path[i] == '\\'); i++)
00031         {
00032             if (j == 0)
00033             {
00034                 j++;
00035                 path[i + offset] = '/';
00036                 continue;
00037             }
00038
00039             offset--;
00040         }
00041
00042         path[i + offset] = path[i];
00043     }
00044
00045     return path;
00046 }
00047
00048 //returns a token from a string, and changes the end ptr
00049 char *strtoken(char *str, char **endptr)
```

```
00050 {
00051     while (*str == ' ' || *str == '\t' || *str == '\n' || *str == ':')
00052         str++;
00053 
00054     char *start = str;
00055 
00056     while (*str && *str != ' ' && *str != '\t' && *str != '\n' && *str != ':')
00057         str++;
00058 
00059     if (endptr)
00060         *endptr = str;
00061 
00062     return start;
00063 }
00064 
00065 char *remove_quotes(char *str, char symbol)
00066 {
00067     int len = strlen(str);
00068     if (len < 2)
00069         return str;
00070 
00071     char first = str[0];
00072     char last = str[len - 1];
00073 
00074     if (last == symbol)
00075         str[len - 1] = '\0'; // remove closing quote
00076     if (first == symbol)
00077         return str + 1; // skip the opening quote
00078 
00079     return str;
00080 }
00081 
00082 
00083 HRESULT norm_strncmp(const char *record, const char *str2)
00084 {
00085     size_t length = strlen(str2);
00086     for (int i = 0; record[i] != '\0' && i < length; i++)
00087     {
00088         char str = record[i];
00089         char str1 = str2[i];
00090         NormalizeInput(&str);
00091         NormalizeInput(&str1);
00092 
00093         if (str != str1)
00094             return S_FALSE;
00095 
00096         if (length - i > 1 && record[i + 1] == '\0')
00097             return S_FALSE;
00098     }
00099 
00100     return S_OK;
00101 }
00102 
00103 void NormalizeInput(char *input)
00104 {
00105     for (int i = 0; i < sizeof(input); i++)
00106         if (input[i] >= 'A' && input[i] <= 'Z')
00107             input[i] += 'a' - 'A';
00108 }
```

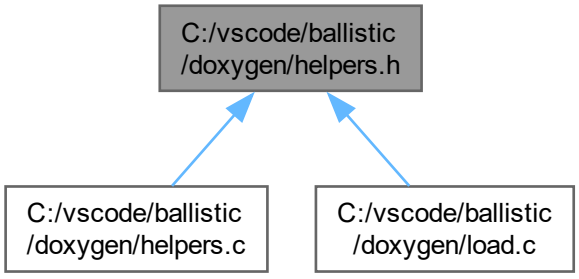
10.28 C:/vscode/ballistic/doxygen/helpers.h File Reference

```
#include <ctype.h>
#include "debug.h"
```

Include dependency graph for helpers.h:



This graph shows which files directly or indirectly include this file:



Functions

- char * ensure_trailing_backslash (char *str)
- HRESULT norm_strncmp (const char *record, const char *str2)
- char * Normalize_path (char *path)
- void NormalizeInput (char *input)

  Normalize input string to lowercase.
- char * remove_quotes (char *str, char symbol)
- char * strtoken (char *s, char **endptr)

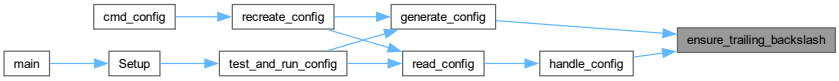
10.28.1 Function Documentation

10.28.1.1 ensure_trailing_backslash()

```
char * ensure_trailing_backslash (
            char * str)
```

Definition at line 8 of file helpers.c.

Here is the caller graph for this function:

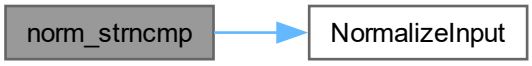


10.28.1.2 norm_strncmp()

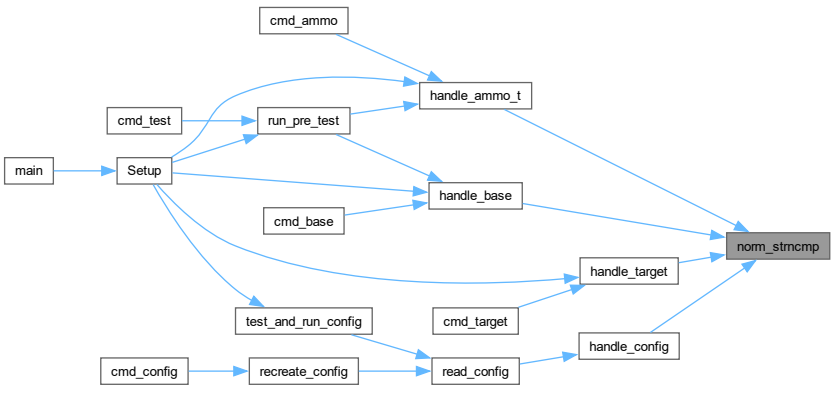
```
HRESULT norm_strncmp (
            const char * record,
            const char * str2)
```

Definition at line 83 of file helpers.c.

Here is the call graph for this function:



Here is the caller graph for this function:

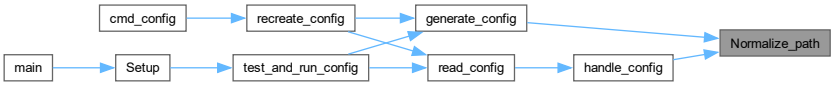


10.28.1.3 Normalize_path()

```
char * Normalize_path (
            char * path)
```

Definition at line 20 of file helpers.c.

Here is the caller graph for this function:

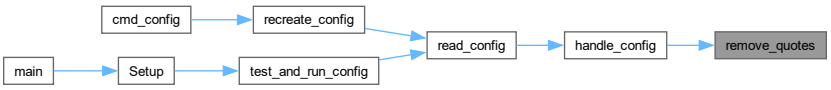


10.28.1.4 remove_quotes()

```
char * remove_quotes (
            char * str,
            char symbol)
```

Definition at line 65 of file helpers.c.

Here is the caller graph for this function:

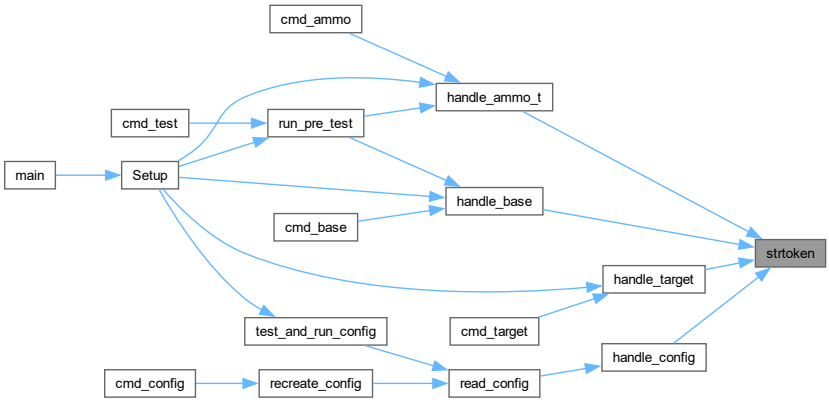


10.28.1.5 strtoken()

```
char * strtoken (
            char * s,
            char ** endptr)
```

Definition at line 49 of file helpers.c.

Here is the caller graph for this function:



10.29 helpers.h

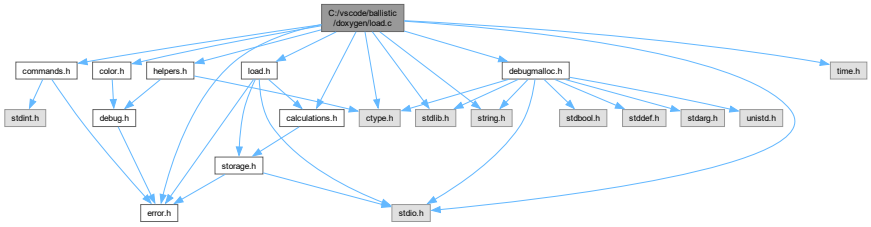
Go to the documentation of this file.

```
00001 #ifndef HELPERS_H
00002 #define HELPERS_H
00003
00004 #include <ctype.h>
00005 #include "debug.h"
00006
00007 char *ensure_trailing_backslash(char *str);
00008 char *Normalize_path(char *path);
00009 char *strtoken(char *s, char **endptr);
00010 char *remove_quotes(char *str, char symbol);
00011 HRESULT norm_strncmp(const char *record, const char *str2);
00012 void NormalizeInput(char *input);
00013
00014 #endif
```

10.30 C:/vscode/ballistic/doxygen/load.c File Reference

```
#include "load.h"
#include "calculations.h"
#include "error.h"
#include "commands.h"
#include "color.h"
#include "helpers.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <time.h>
#include "debugmalloc.h"
```

Include dependency graph for load.c:



Functions

- HRESULT format_basic_error_string (char *error_buffer, int *error_flags)

  Format error string for missing parameters.
- void free_folder_path (void)

  Free the working directory path memory.
- HRESULT generate_config (FILE *configfile, const char *config_file)

  Generate new configuration file interactively.
- HRESULT handle_ammo_t (char *record, void *data, int *error_flags)

  Parse ammunition type record.
- HRESULT handle_base (char *record, void *data, int *error_flags)

  Parse base record.
- HRESULT handle_config (char *record, void *data, int *error_flags)

  Handle configuration file record parsing.
- HRESULT handle_target (char *record, void *data, int *error_flags)

  Parse target record.
- int HandleParameterSelection (void)

  Interactive parameter selection for calculations.
- void LogResult (const Base *base, const Target *tgt, const Ammo_t *ammo, const BallisticResult *res)

  Log calculation result to file.
- int peek (FILE *file)

  Peek next character from file without consuming it.
- HRESULT read_config (const char *config_file)

  Read and apply existing configuration file.
- HRESULT recreate_config (const char *config_file, PROCESS mode)

  Recreate configuration file with new settings.
- HRESULT scan (const char *filename, RecordHandler handler, void *data, PROCESS mode)

  Parse a file and call handler for each record.
- void set_folder_path (const char *new_path)

  Set the working directory path.
- HRESULT test_and_run_config (const char *config_file, PROCESS mode)

  Test configuration and create if missing.

Variables

- char * folder_path = NULL

  Pointer to current working directory path.

10.31 load.c

Go to the documentation of this file.

```
00001 #include "load.h"
00002 #include "calculations.h"
00003 #include "error.h"
00004 #include "commands.h"
00005 #include "color.h"
00006 #include "helpers.h"
00007
00008 #include <stdio.h>
00009 #include <stdlib.h>
00010 #include <string.h>
00011 #include <ctype.h>
00012 #include <time.h>
00013
00014 #include "debugmalloc.h"
00015
00016 // add the save to file option, if it's done, only the calculations left
00017
00018 char *folder_path = NULL;
00019
00020 void set_folder_path(const char *new_path)
00021 {
00022     if (!new_path)
00023         return;
00024
00025     // Free old memory if already allocated
00026     free(folder_path);
00027
00028     // Allocate new memory for the new path
00029     folder_path = malloc(strlen(new_path) + 1);
00030     if (!folder_path)
00031     {
00032         fprintf(stderr, "Memory allocation failed for folder_path\n");
00033         return;
00034     }
00035
00036     strcpy(folder_path, new_path);
00037 }
00038
00039 void free_folder_path(void)
00040 {
00041     free(folder_path);
00042     folder_path = NULL;
00043 }
00044
00045 /*file name to scan  |  handler function  |  external data for the handler  |  scan mode*/
00046 HRESULT scan(const char *filename, RecordHandler handler, void *data, PROCESS mode)
00047 {
00048     FILE *file = fopen(filename, "r"); // "r" open file in read only mode
00049     if (!file)
00050     {
00051         printf("Failed to open or locate [%s] in the working directory!\n", filename);
00052         return S_FALSE;
00053     }
00054
00055     int c;
00056     ParseState state = NORMAL;
00057     char buffer[512];
00058     int index = 0;
00059     HRESULT hr = S_OK; // handle to a result
00060
00061     while ((c = fgetc(file)) != EOF && SUCCEEDED(hr))
00062     {
00063         switch (state)
00064         {
00065         case NORMAL:
00066         {
00067             if (c == '/')
00068             {
00069                 int value = peek(file);
00070                 if (value == '/')
00071                     state = LINE_COMMENT;
00072                 else if (value == '*')
00073                     state = BLOCK_COMMENT;
00074                 else
00075                 { // invalid line
00076                     fprintf(stderr, "Warning: invalid syntax, skipping line\n");
00077                     while ((c = fgetc(file)) != EOF && c != '\n')
00078                         ;
00079                 }
00080             }
00081             else if (!isspace(c))
00082             {
```

```
00083                 // start buffering a record
00084                 ungetc(c, file);
00085                 state = RECORD;
00086             }
00087 
00088             break;
00089         }
00090         case LINE_COMMENT:
00091         {
00092             if (mode == S_DEBUG)
00093                 printf("[Line comment] : \n");
00094             while ((c = fgetc(file)) != EOF)
00095             {
00096                 char idk = (char)c;
00097                 if (mode == S_DEBUG)
00098                     putchar((unsigned char)c);
00099 
00100                 if (c == '\n')
00101                     break;
00102             }
00103             state = NORMAL;
00104             break;
00105         }
00106         case BLOCK_COMMENT:
00107         {
00108             if (mode == S_DEBUG)
00109                 printf("[Block comment] : \n");
00110             while ((c = fgetc(file)) != EOF)
00111                 if (c == '*' && peek(file) == '/')
00112                     break;
00113                 else if (mode == S_DEBUG)
00114                     putchar(c);
00115 
00116             fgetc(file);
00117             state = NORMAL;
00118             break;
00119         }
00120         case RECORD:
00121         {
00122             if (c == ';')
00123             {
00124                 // terminate buffer
00125                 buffer[index] = '\0';
00126                 int error_flags[5] = {0};
00127                 // a 256 sized array gets passed to the handler, so you can safely step it's ptr
00128                 if (handler)
00129                     hr = handler(buffer, data, error_flags); // pass the buffer and the user input
    address to the function
00130 
00131                 else
00132                 {
00133                     printf("No input handler!\n");
00134                     hr = S_FALSE;
00135                 }
00136 
00137                 if (hr == S_PARTIAL)
00138                 {
00139                     char error[128] = {0};
00140                     if (format_basic_error_string(error, error_flags) == S_FALSE)
00141                         printf("[Error] : Error buffer overflow!\n");
00142                     printf("[Warning] : Missing parameter in the file [%s] : \nat the line [%s] : \n
    suggested parameters to add [%s]\n", filename, buffer, error);
00143                 }
00144                 index = 0;
00145                 state = NORMAL;
00146             }
00147             else if (index < sizeof(buffer) - 1) // update the buffer loader to not store useless
    spaces, only store 1
00148             {
00149                 if (isspace(c))
00150                     state = NORMAL;
00151                 buffer[index++] = (char)c;
00152             }
00153             else
00154             {
00155                 printf("[Error] buffer overflow!");
00156                 hr = S_FALSE;
00157             }
00158 
00159             break;
00160         }
00161         }
00162     }
00163 
00164     if (fclose(file) == EOF)
00165         hr = E_FAIL;
00166     return hr;
```

```
00167 }
00168
00169 int peek(FILE *file)
00170 {
00171     int c = fgetc(file);
00172     if (c != EOF)
00173         ungetc(c, file); // put it back
00174     return c;
00175 }
00176
00177 HRESULT format_basic_error_string(char *error_buffer, int *error_flags)
00178 {
00179     int er_cnt = -1;
00180
00181     for (int i = 0; i < 5; i++)
00182     {
00183         if (error_flags[i] != 1)
00184         {
00185             er_cnt++;
00186             switch (i)
00187             {
00188             case F_ID:
00189             {
00190                 const char *id = "I";
00191                 if (er_cnt > 0)
00192                     id = ", I";
00193                 strncat(error_buffer, id, sizeof(error_buffer) - strlen(error_buffer) - 1);
00194                 continue;
00195             }
00196             case F_LABEL:
00197             {
00198                 const char *id = "L";
00199                 if (er_cnt > 0)
00200                     id = ", L";
00201                 strncat(error_buffer, id, sizeof(error_buffer) - strlen(error_buffer) - 1);
00202                 continue;
00203             }
00204             case F_LATITUDE:
00205             {
00206                 const char *id = "CLA";
00207                 if (er_cnt > 0)
00208                     id = ", CLA";
00209                 strncat(error_buffer, id, sizeof(error_buffer) - strlen(error_buffer) - 1);
00210                 continue;
00211             }
00212             case F_LONGITUDE:
00213             {
00214                 const char *id = "CLO";
00215                 if (er_cnt > 0)
00216                     id = ", CLO";
00217                 strncat(error_buffer, id, sizeof(error_buffer) - strlen(error_buffer) - 1);
00218                 continue;
00219             }
00220             case F_ALTITUDE:
00221             {
00222                 const char *id = "CA";
00223                 if (er_cnt > 0)
00224                     id = ", CA";
00225                 strncat(error_buffer, id, sizeof(error_buffer) - strlen(error_buffer) - 1);
00226                 continue;
00227             }
00228             }
00229             return S_FALSE;
00230         }
00231     }
00232     strncat(error_buffer, "\0", sizeof(error_buffer) - strlen(error_buffer));
00233     return S_OK;
00234 }
00235
00236 // config============================================
00237
00238 HRESULT test_and_run_config(const char *config_file, PROCESS mode) // check if it's the first time,
      that the program is being ran, or if there is any missing info
00239 {
00240     HRESULT hr = S_OK;
00241     FILE *configfile = fopen(config_file, "r"); // "r" open file in read only mode
00242     if (!configfile)
00243     {
00244         printf("Failed to open or locate [%s] in the working directory!\n", config_file);
00245         printf("Do you want to create a new one?\n");
00246
00247         switch (GetCommand(CMD_C_LOOP | CMD_C_YESNO, NULL))
00248         {
00249         case CMD_YES:
00250         {
00251             hr = generate_config(configfile, config_file);
00252             break;
```

```
00253         }
00254         case CMD_NO:
00255         {
00256             hr = E_FAIL;
00257             break;
00258         }
00259         }
00260         return hr;
00261     }
00262     fclose(configfile);
00263 
00264     hr = read_config(config_file);
00265     if (SUCCEEDED(hr))
00266     {
00267         if (mode == S_DEBUG)
00268             printf("Working directory have been set to: [%s]\n", folder_path);
00269         printf("-----------------------------------\n");
00270         printf("Config loaded succesfully!\n");
00271         printf("-----------------------------------\n");
00272         printf("\n");
00273     }
00274     return hr;
00275 }
00276 
00277 HRESULT recreate_config(const char *config_file, PROCESS mode)
00278 {
00279     HRESULT hr = S_OK;
00280     FILE *configfile = NULL; // "r" open file in read only mode
00281 
00282     return generate_config(configfile, config_file);
00283 
00284     hr = read_config(config_file);
00285     if (SUCCEEDED(hr))
00286     {
00287         if (mode == S_DEBUG)
00288             printf("Working directory have been set to: [%s]\n", folder_path);
00289         printf("-----------------------------------\n");
00290         printf("Config updated successfully!\n");
00291         printf("-----------------------------------\n");
00292         printf("\n");
00293     }
00294     return hr;
00295 }
00296 
00297 HRESULT generate_config(FILE *configfile, const char *config_file)
00298 {
00299     printf("Enter a folder path:\n");
00300 
00301     char temp[1024]; // temporary buffer for user input
00302     if (fgets(temp, sizeof(temp), stdin))
00303     {
00304         temp[strcspn(temp, "\n")] = '\0'; // remove trailing newline
00305 
00306         ensure_trailing_backslash(temp);
00307         Normalize_path(temp);
00308 
00309         set_folder_path(temp);
00310     }
00311 
00312     configfile = fopen(config_file, "w"); // create a new config file
00313     if (!config_file)
00314         return S_FALSE;
00315 
00316     printf("Enter the colour mode:\n");
00317 
00318     switch (GetCommand(CMD_C_LOOP | CMD_C_SETUP, NULL))
00319     {
00320     case CMD_COLOUR:
00321         set_color_mode(CM_COLOUR);
00322         break;
00323     case CMD_MONO:
00324         set_color_mode(CM_MONO);
00325         break;
00326     }
00327 
00328     fprintf(configfile, "/*\n=====================================================\n");
00329     fprintf(configfile, "--------Parameters-----------------------------\n");
00330     fprintf(configfile, "working directory :\n");
00331     fprintf(configfile, "color mode : [0=monochrome, 1=coloured]\n");
00332     fprintf(configfile, "=====================================================\n");
00333     fprintf(configfile, "-the id gets normalized\n");
00334     fprintf(configfile, "-end every line with ;\n");
00335     fprintf(configfile, "=====================================================\n*/\n");
00336     fprintf(configfile, "\n");
00337     fprintf(configfile, "DIR: \"%s\";\n", folder_path);
00338     fprintf(configfile, "COLOUR_M: %d;\n", get_color_mode());
00339 
```

```
00340     if (fclose(configfile) != 0)
00341     {
00342         printf("\n========================\n");
00343         printf("[ERROR] Config file creation error");
00344         printf("\n========================\n");
00345         return S_TERMINATE;
00346     }
00347     return S_OK;
00348 }
00349
00350 HRESULT read_config(const char *config_file)
00351 {
00352
00353     FILE *configfile = fopen(config_file, "r");
00354     if (!configfile)
00355     {
00356         printf("Failed to open or locate [%s] in the working directory!\n", config_file);
00357         return S_FALSE;
00358     }
00359
00360     scan(config_file, handle_config, NULL, S_RUNNING);
00361
00362     if (fclose(configfile) == EOF)
00363
00364         return E_FAIL;
00365
00366     return S_OK;
00367 }
00368
00369 HRESULT handle_config(char *record, void *data, int *error_flags)
00370 {
00371     char *end = NULL;
00372     /*searches for the normlaized DIR token, then if it is, searches for the value after the ':'
    charcter*/
00373     if (norm_strncmp(strtoken(record, &end), "DIR") == S_OK && (record = strchr(end, ':')) != NULL)
00374     {
00375         set_folder_path(Normalize_path(ensure_trailing_backslash(remove_quotes((char
    *)strtoken(record, &record), '\"'))));
00376         return S_OK;
00377     }
00378
00379     /*searches for the normlaized COLOUR_M token, then if it is, searches for the value after  the ':'
    charcter*/
00380     if (norm_strncmp(strtoken(record, &end), "COLOUR_M") == S_OK && (record = strchr(end, ':')) !=
    NULL)
00381     {
00382         set_color_mode((int)strtof(strtoken(record, &end), &record));
00383         return S_OK;
00384     }
00385     return S_OK;
00386 }
00387
00388 HRESULT handle_ammo_t(char *record, void *data, int *error_flags)
00389 {
00390     // record in the scan case is only on the stack, so I can modify the pointer without a problem
00391     Ammo_t ammo;
00392
00393     // hard cap, so if the user gives some invalid parameters, the program won't get stuck
00394     for (int i = 0; i < 15 && strlen(record) > 0; i++)
00395     {
00396
00397         char *end = NULL;
00398         /*id*/
00399         if (norm_strncmp(record, "I") == S_OK)
00400         {
00401             record = strtoken(record, &end);
00402             ammo.id = (unsigned)strtof(++record, &end);
00403             record = strtoken(end, &end);
00404         }
00405         /*name*/
00406         if (norm_strncmp(record, "N") == S_OK)
00407         {
00408             record = strtoken(record, &end);
00409             record++;
00410             ammo.name = (char *)malloc(end - record + 1); // +1 for null terminator
00411             strncpy(ammo.name, record, end - record);
00412             ammo.name[end - record] = '\0';
00413
00414             record = strtoken(end, &end);
00415         }
00416         /*weight*/
00417         if (norm_strncmp(record, "W") == S_OK)
00418         {
00419             record = strtoken(record, &end);
00420             ammo.weight = strtod(++record, &end);
00421             record = strtoken(end, &end);
00422         }
```

```
00423         /*diameter*/
00424         if (norm_strncmp(record, "D") == S_OK)
00425         {
00426             record = strtoken(record, &end);
00427             ammo.diameter = strtod(++record, &end);
00428             record = strtoken(end, &end);
00429         }
00430         /*blast radius*/
00431         if (norm_strncmp(record, "BR") == S_OK)
00432         {
00433             record = strtoken(record, &end);
00434             record++;
00435             ammo.blastRadius = strtod(++record, &end);
00436             record = strtoken(end, &end);
00437         }
00438 
00439         /*blast radius*/
00440         if (norm_strncmp(record, "MV") == S_OK)
00441         {
00442             record = strtoken(record, &end);
00443             record++;
00444             ammo.muzzleVelocity = strtod(++record, &end);
00445             record = strtoken(end, &end);
00446         }
00447 
00448         /*ballistic coefficent*/
00449         if (norm_strncmp(record, "BC") == S_OK)
00450         {
00451             record = strtoken(record, &end);
00452             record++;
00453             ammo.ballistic_coefficient = strtod(++record, &end);
00454             record = strtoken(end, &end);
00455         }
00456     }
00457 
00458     add_Ammo_t(data, ammo);
00459 
00460     free(ammo.name);
00461 
00462     return S_OK;
00463 }
00464 
00465 HRESULT handle_base(char *record, void *data, int *error_flags)
00466 {
00467     // record in the scan case is only on the stack, so I can modify the pointer without a problem
00468     Base base;
00469 
00470     // hard cap, so if the user gives some invalid parameters, the program won't get stuck
00471     for (int i = 0; i < 15 && strlen(record) > 0; i++)
00472     {
00473 
00474         char *end = NULL;
00475         // id
00476         if (norm_strncmp(record, "I") == S_OK)
00477         {
00478             record = strtoken(record, &end);
00479             base.id = (unsigned)strtof(++record, &end);
00480             record = strtoken(end, &end);
00481         }
00482         // name
00483         if (norm_strncmp(record, "L") == S_OK)
00484         {
00485             record = strtoken(record, &end);
00486             record++;
00487             base.label = (char *)malloc(end - record + 1); // +1 for null terminator
00488             strncpy(base.label, record, end - record);
00489             base.label[end - record] = '\0';
00490 
00491             record = strtoken(end, &end);
00492         }
00493         // latitude
00494         if (norm_strncmp(record, "CLA") == S_OK)
00495         {
00496             record = strtoken(record, &end);
00497             record += 3;
00498             base.coord.latitude = strtod(record, &end);
00499             record = strtoken(end, &end);
00500         }
00501         // longitude
00502         if (norm_strncmp(record, "CLO") == S_OK)
00503         {
00504             record = strtoken(record, &end);
00505             record += 3;
00506             base.coord.longitude = strtod(record, &end);
00507             record = strtoken(end, &end);
00508         }
00509         // altitude
```

```
00510         if (norm_strncmp(record, "CA") == S_OK)
00511         {
00512             record = strtoken(record, &end);
00513             record += 2;
00514             base.coord.altitude = strtod(record, &end);
00515             record = strtoken(end, &end);
00516         }
00517         // ammo list
00518         if (norm_strncmp(record, "AMS") == S_OK)
00519         {
00520             record += 3;
00521             record = strtoken(record, &end);
00522             record = strtoken(++record, &end);
00523 
00524             base.ammoStock = NULL;
00525             init_Ammo_Stock_List(&base.ammoStock);
00526 
00527             while (strlen(record) > 1)
00528             {
00529                 // add ammo type, to combed list
00530                 unsigned id = (unsigned)strtof(record, &end);
00531                 record = strtoken(end, &end);
00532 
00533                 unsigned ammount = (unsigned)strtof(record, &end);
00534                 record = strtoken(end, &end);
00535                 record++;
00536 
00537                 add_Ammo_Stock(base.ammoStock, GetNode_Ammot_t_List(ammo_t_list, id), ammount);
00538             }
00539         }
00540     }
00541 
00542     add_Base(data, base);
00543 
00544     free(base.label);
00545 
00546     return S_OK;
00547 }
00548 
00549 HRESULT handle_target(char *record, void *data, int *error_flags)
00550 {
00551     // record in the scan case is only on the stack, so I can modify the pointer without a problem
00552     Target target;
00553 
00554     // hard cap, so if the user gives some invalid parameters, the program won't get stuck
00555     for (int i = 0; i < 15 && strlen(record) > 0; i++)
00556     {
00557 
00558         char *end = NULL;
00559         // id
00560         if (norm_strncmp(record, "I") == S_OK)
00561         {
00562             record = strtoken(record, &end);
00563             target.id = (unsigned)strtof(++record, &end);
00564             record = strtoken(end, &end);
00565         }
00566         // name
00567         if (norm_strncmp(record, "L") == S_OK)
00568         {
00569             record = strtoken(record, &end);
00570             record++;
00571             target.label = (char *)malloc(end - record + 1); // +1 for null terminator
00572             strncpy(target.label, record, end - record);
00573             target.label[end - record] = '\0';
00574 
00575             record = strtoken(end, &end);
00576         }
00577         // latitude
00578         if (norm_strncmp(record, "CLA") == S_OK)
00579         {
00580             record = strtoken(record, &end);
00581             record += 3;
00582             target.coord.latitude = strtod(record, &end);
00583             record = strtoken(end, &end);
00584         }
00585         // longitude
00586         if (norm_strncmp(record, "CLO") == S_OK)
00587         {
00588             record = strtoken(record, &end);
00589             record += 3;
00590             target.coord.longitude = strtod(record, &end);
00591             record = strtoken(end, &end);
00592         }
00593         // altitude
00594         if (norm_strncmp(record, "CA") == S_OK)
00595         {
00596             record = strtoken(record, &end);
```

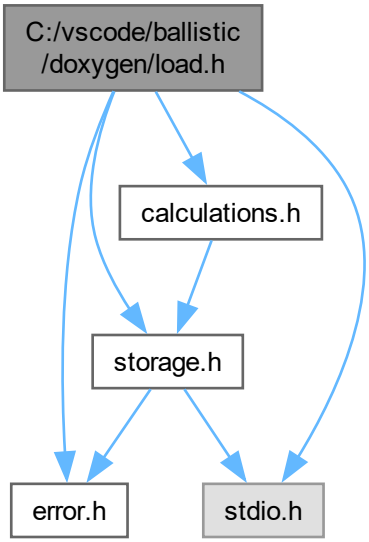
```
00597             record += 2;
00598             target.coord.altitude = strtod(record, &end);
00599             record = strtoken(end, &end);
00600         }
00601         // area
00602         if (norm_strncmp(record, "A") == S_OK)
00603         {
00604             record = strtoken(record, &end);
00605             target.area = strtod(++record, &end);
00606             record = strtoken(end, &end);
00607         }
00608     }
00609
00610     add_Target(data, target);
00611
00612     free(target.label);
00613
00614     return S_OK;
00615 }
00616
00617 void LogResult(const Base *base, const Target *tgt, const Ammo_t *ammo, const BallisticResult *res)
00618 {
00619     if (!base || !tgt || !ammo || !res)
00620     {
00621         printf("LogResult: invalid argument.\n");
00622         return;
00623     }
00624
00625     char folder_buffer[1024];
00626     strcpy(folder_buffer, folder_path);
00627     strcat(folder_buffer, "log.txt");
00628     FILE *fp = fopen(folder_buffer, "a");
00629     if (!fp)
00630     {
00631         printf("Failed to open log.txt for writing.\n");
00632         return;
00633     }
00634
00635     // get the time for log
00636     time_t now = time(NULL);
00637     char *ts = ctime(&now);
00638     if (ts)
00639         ts[strcspn(ts, "\n")] = 0; // strip newline
00640
00641     fprintf(fp,
00642             "=== ENTRY @ %s ===\n"
00643             "Base:\n"
00644             "  ID: %u\n"
00645             "  Label: %s\n"
00646             "  Coord: latitude %.6f, longitude %.6f, altitude %.2f m\n"
00647             "\n"
00648             "Target:\n"
00649             "  ID: %u\n"
00650             "  Label: %s\n"
00651             "  Coord: latitude %.6f, longitude %.6f, altitude %.2f m\n"
00652             "\n"
00653             "Ammo:\n"
00654             "  ID: %u\n"
00655             "  Name: %s\n"
00656             "  Weight: %.3f kg\n"
00657             "  Diameter: %.3f mm\n"
00658             "  BlastRadius: %.3f m\n"
00659             "  MuzzleVel: %.3f m/s\n"
00660             "  BC(G7): %.5f\n"
00661             "\n"
00662             "Solution:\n"
00663             "  Elevation: %.4f deg\n"
00664             "  Azimuth:   %.4f deg\n"
00665             "  TOF:       %.4f s\n"
00666             "  ImpactVel: %.4f m/s\n"
00667             "  Valid:     %s\n"
00668             "------------------------------\n\n",
00669
00670             ts,
00671             base->id, base->label,
00672             base->coord.latitude, base->coord.longitude, base->coord.altitude,
00673
00674             tgt->id, tgt->label,
00675             tgt->coord.latitude, tgt->coord.longitude, tgt->coord.altitude,
00676
00677             ammo->id, ammo->name,
00678             ammo->weight,
00679             ammo->diameter,
00680             ammo->blastRadius,
00681             ammo->muzzleVelocity,
00682             ammo->ballistic_coefficient,
00683
```

```
00684             res->elevation_deg,
00685             res->azimuth_deg,
00686             res->time_of_flight,
00687             res->impact_velocity,
00688             res->valid ? "YES" : "NO");
00689
00690     fclose(fp);
00691 }
00692
00693 int HandleParameterSelection(void)
00694 {
00695     unsigned baseID, targetID, ammoID;
00696     char input[128] = {0};
00697     printf("------------------------------\n");
00698     printf("Enter parameters: <base_id> <target_id> <ammo_id>\n");
00699     printf("Type [back] to return or to view/expand the available [list]!\n");
00700     printf("------------------------------\n\n");
00701
00702     if (!fgets(input, sizeof(input), stdin))
00703         return -1;
00704
00705     input[strcspn(input, "\r\n")] = 0; // remove newline
00706
00707     if (isalpha((unsigned char)input[0]))
00708     {
00709         Command_t cmd = ParseCommandFromBuffer(input, CMD_C_BACK, NULL);
00710         if (cmd == CMD_BACK)
00711             return -1;
00712         if (cmd != CMD_INVALID)
00713             return 0;
00714     }
00715
00716     if (sscanf(input, "%u %u %u", &baseID, &targetID, &ammoID) != 3)
00717     {
00718         printf("Invalid input. Expected format: <base_id> <target_id> <ammo_id>\n");
00719         return -1;
00720     }
00721
00722     Base *base = FindBaseByID(base_list, baseID);
00723     Target *tgt = FindTargetByID(target_list, targetID);
00724     Ammo_t *ammo = FindAmmoByID(ammo_t_list, ammoID);
00725
00726     if (!tgt)
00727     {
00728         printf("Target ID %u does not exist.\n", targetID);
00729         return 0;
00730     }
00731
00732     // If base or ammo is invalid or cannot hit target, suggest automatically
00733     if (!base || !ammo || !compute_fire_solution(base, tgt, ammo).valid)
00734     {
00735         printf("Finding best available base and ammo...\n");
00736         printf("This may take a while...\n");
00737         if (!SuggestBestBaseAndAmmo_RegionAware(base_list, tgt, &base, &ammo))
00738         {
00739             printf("No suitable base/ammo combination found.\n");
00740             return 0;
00741         }
00742
00743         printf("Suggested base and ammo:\n");
00744         printf("  Base ID: %u  Label: %s  Coordinates: lat %.6f, lon %.6f, alt %.2f m\n",
00745                base->id, base->label, base->coord.latitude, base->coord.longitude,
      base->coord.altitude);
00746         printf("  Ammo ID: %u  Label: %s\n", ammo->id, ammo->name);
00747
00748         printf("Would you like to accept these changes? [yes/no]\n");
00749         if (GetCommand(CMD_C_LOOP | CMD_C_YESNO, NULL) == CMD_YES)
00750         {
00751             goto Recalculate;
00752         }
00753
00754         return 0;
00755     }
00756
00757 Recalculate:
00758     BallisticResult b_result = compute_fire_solution(base, tgt, ammo);
00759     if (!b_result.valid)
00760     {
00761         printf("Even the suggested base cannot reach the target.\n");
00762         return 0;
00763     }
00764
00765     LogResult(base, tgt, ammo, &b_result);
00766     printf("Parameters accepted and logged.\n");
00767     return 0;
00768 }
```

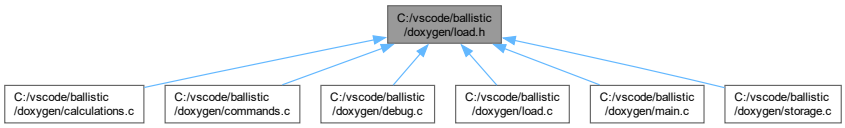
10.32 C:/vscode/ballistic/doxygen/load.h File Reference

```
#include "storage.h"
#include "error.h"
#include "calculations.h"
#include <stdio.h>
```

Include dependency graph for load.h:



This graph shows which files directly or indirectly include this file:



Typedefs

- typedef HRESULT(∗ RecordHandler) (char ∗record, void ∗data, int ∗error_flags)

  Record handler function type for parser.

Enumerations

- enum ParseState { NORMAL , LINE_COMMENT , BLOCK_COMMENT , RECORD }

  Parser state machine states.

Functions

- HRESULT format_basic_error_string (char *error_buffer, int *error_flags)

  Format error string for missing parameters.
- void free_folder_path (void)

  Free the working directory path memory.
- int HandleParameterSelection (void)

  Interactive parameter selection for calculations.
- void LogResult (const Base *base, const Target *tgt, const Ammo_t *ammo, const BallisticResult *res)

  Log calculation result to file.
- int peek (FILE *file)

  Peek next character from file without consuming it.
- HRESULT scan (const char *filename, RecordHandler handler, void *data, PROCESS mode)

  Parse a file and call handler for each record.
- void set_folder_path (const char *new_path)

  Set the working directory path.

Configuration Functions

- HRESULT generate_config (FILE *configfile, const char *config_file)

  Generate new configuration file interactively.
- HRESULT handle_config (char *record, void *data, int *error_flags)

  Handle configuration file record parsing.
- HRESULT read_config (const char *config_file)

  Read and apply existing configuration file.
- HRESULT recreate_config (const char *config_file, PROCESS mode)

  Recreate configuration file with new settings.
- HRESULT test_and_run_config (const char *config_file, PROCESS mode)

  Test configuration and create if missing.

Data File Handlers

- HRESULT handle_ammo_t (char *record, void *data, int *error_flags)

  Parse ammunition type record.
- HRESULT handle_base (char *record, void *data, int *error_flags)

  Parse base record.
- HRESULT handle_target (char *record, void *data, int *error_flags)

  Parse target record.

Variables

- char * folder_path

  Pointer to current working directory path.

10.33 load.h

Go to the documentation of this file.

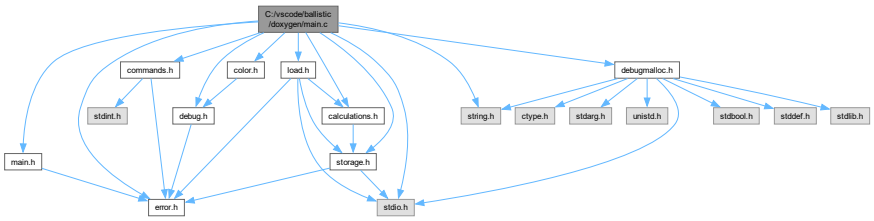
```
00001
00008
00009 #ifndef LOAD_H
00010 #define LOAD_H
00011
00012 #include "storage.h"
00013 #include "error.h"
00014 #include "calculations.h"
00015 #include <stdio.h>
00016
00021 typedef enum
00022 {
00023     NORMAL,
00024     LINE_COMMENT,
00025     BLOCK_COMMENT,
00026     RECORD
00027 } ParseState;
00028
00035 extern char *folder_path;
00036
00042 void set_folder_path(const char *new_path);
00043
00048 void free_folder_path(void);
00049
00060 typedef HRESULT (*RecordHandler)(char *record, void *data, int *error_flags);
00061
00073 HRESULT scan(const char *filename, RecordHandler handler, void *data, PROCESS mode);
00074
00081 int peek(FILE *file);
00082
00090 HRESULT format_basic_error_string(char *error_buffer, int *error_flags);
00091
00096
00104 HRESULT generate_config(FILE *configfile, const char *config_file);
00105
00113 HRESULT recreate_config(const char *config_file, PROCESS mode);
00114
00121 HRESULT read_config(const char *config_file);
00122
00130 HRESULT test_and_run_config(const char *config_file, PROCESS mode);
00131
00140 HRESULT handle_config(char *record, void *data, int *error_flags);
00141
00143
00148
00157 HRESULT handle_ammo_t(char *record, void *data, int *error_flags);
00158
00167 HRESULT handle_base(char *record, void *data, int *error_flags);
00168
00177 HRESULT handle_target(char *record, void *data, int *error_flags);
00178
00180
00189 int HandleParameterSelection(void);
00190
00201 void LogResult(const Base *base, const Target *tgt, const Ammo_t *ammo, const BallisticResult *res);
00202
00203 #endif /* LOAD_H */
```

10.34 C:/vscode/ballistic/doxygen/main.c File Reference

```
#include <stdio.h>
#include <string.h>
#include "main.h"
#include "debug.h"
#include "commands.h"
#include "calculations.h"
#include "load.h"
#include "error.h"
#include "storage.h"
#include "color.h"
```

```
#include "debugmalloc.h"
```

Include dependency graph for main.c:



Functions

- void CleanUp ()
- int main (void)
- PROCESS Setup ()

  Program initialization and setup.

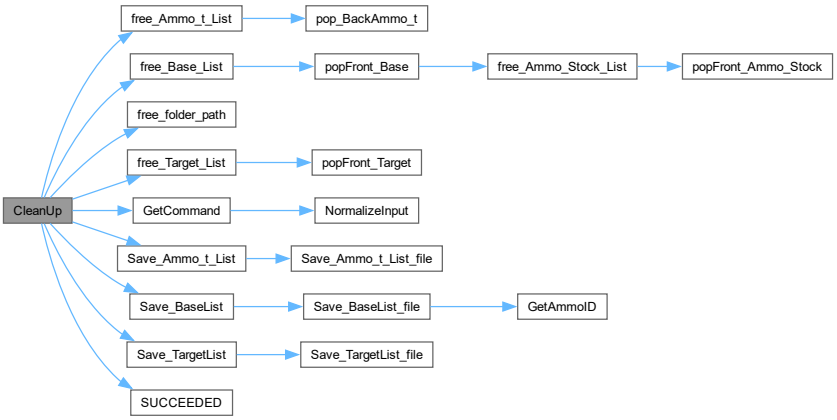
10.34.1 Function Documentation

10.34.1.1 CleanUp()

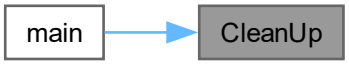
```
void CleanUp ()
```

Definition at line 67 of file main.c.

Here is the call graph for this function:



Here is the caller graph for this function:

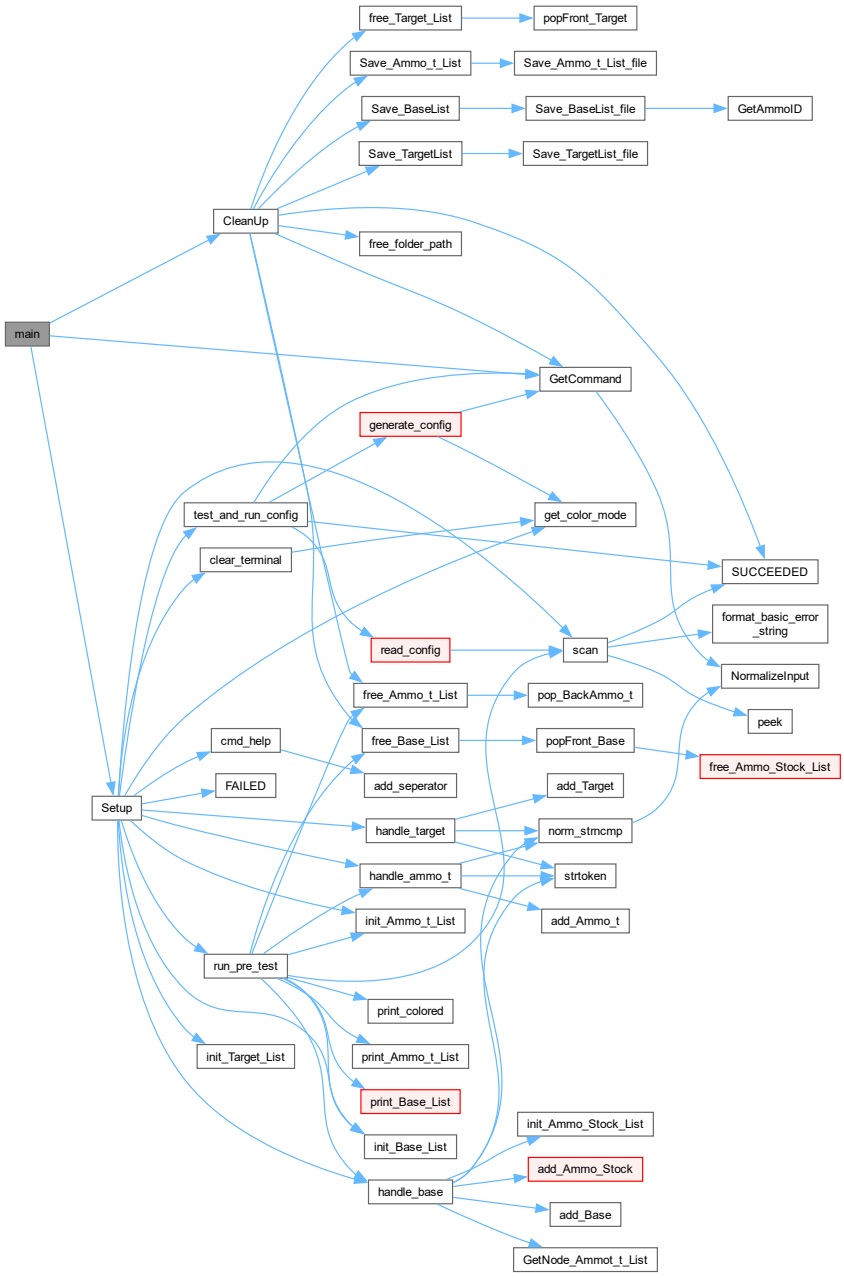


10.34.1.2 main()

```
int main (
            void )
```

Definition at line 107 of file main.c.

Here is the call graph for this function:



10.35 main.c

Go to the documentation of this file.

```
00001 #include <stdio.h>
00002 #include <string.h>
00003
00004 #include "main.h"
00005 #include "debug.h"
00006 #include "commands.h"
00007 #include "calculations.h"
00008 #include "load.h"
00009 #include "error.h"
```

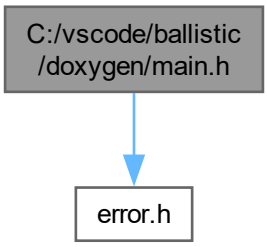
```
00010 #include "storage.h"
00011 #include "color.h"
00012
00013 #include "debugmalloc.h"
00014
00015 //========================================
00016 // Created by.:
00017 // Starting date: 2025.10.17
00018 // Finishing date:
00019 //========================================
00020
00021 PROCESS Setup()
00022 {
00023     printf("Starting setup...\n");
00024     if (FAILED(test_and_run_config("config.txt", S_RUNNING)))
00025         return S_TERMINATE;
00026
00027     if (FAILED(run_pre_test(S_RUNNING)))
00028         return S_TERMINATE;
00029
00030     printf("Setup finished!\n");
00031
00032     char folder_buffer[1024];
00033     strcpy(folder_buffer, folder_path);
00034     strcat(folder_buffer, "ammo_catalouge.txt");
00035     init_Ammo_t_List(&ammo_t_list);
00036     scan(folder_buffer, handle_ammo_t, ammo_t_list, S_RUNNING);
00037     // print_Ammo_t_List(ammo_t_list);
00038
00039     strcpy(folder_buffer, folder_path);
00040     strcat(folder_buffer, "bases.txt");
00041     init_Base_List(&base_list);
00042     scan(folder_buffer, handle_base, base_list, S_RUNNING);
00043     // print_Base_List(base_list, S_RUNNING);
00044
00045     strcpy(folder_buffer, folder_path);
00046     strcat(folder_buffer, "targets.txt");
00047     init_Target_List(&target_list);
00048     scan(folder_buffer, handle_target, target_list, S_RUNNING);
00049     // print_Target_List(target_list);
00050
00051     if (get_color_mode())
00052     {
00053         clear_terminal();
00054     }
00055
00056     printf("\n");
00057     printf("==================================\n");
00058     printf("Ballistic trajectory calculator\n");
00059     printf("Created by.: Adam Jozsef Horvath\n");
00060     printf("==================================\n");
00061     printf("\n");
00062     printf("Type [exit] to leave or type [help] for the commands!\n");
00063     cmd_help(NULL, CMD_HELP, NULL);
00064     return S_RUNNING;
00065 }
00066
00067 void CleanUp()
00068 {
00069     printf("Do you want to save before leaving? [YES/NO]\n");
00070     if (GetCommand(CMD_C_YESNO | CMD_C_LOOP, NULL) == CMD_YES)
00071     {
00072         char folder_buffer[1024];
00073         if (base_list)
00074         {
00075             strcpy(folder_buffer, folder_path);
00076             strcat(folder_buffer, "bases.txt");
00077             if (SUCCEEDED(Save_BaseList(base_list, folder_buffer)))
00078                 printf("Base list saved succesfully!\n");
00079         }
00080
00081         if (target_list)
00082         {
00083             strcpy(folder_buffer, folder_path);
00084             strcat(folder_buffer, "targets.txt");
00085             if (SUCCEEDED(Save_TargetList(target_list, folder_buffer)))
00086                 printf("Target list saved succesfully!\n");
00087         }
00088
00089         if (ammo_t_list)
00090         {
00091             strcpy(folder_buffer, folder_path);
00092             strcat(folder_buffer, "ammo_catalouge.txt");
00093             if (SUCCEEDED(Save_Ammo_t_List(ammo_t_list, folder_buffer)))
00094                 printf("Ammo Type list saved succesfully!\n");
00095         }
00096     }
```

```
00097     if (base_list)
00098         free_Base_List(&base_list);
00099     if (target_list)
00100         free_Target_List(&target_list);
00101     if (ammo_t_list)
00102         free_Ammo_t_List(&ammo_t_list);
00103 
00104     free_folder_path();
00105 }
00106 
00107 int main(void)
00108 {
00109     PROCESS state = Setup();
00110 
00111     while (state == S_RUNNING)
00112     {
00113         GetCommand(CMD_C_GENERIC | CMD_C_SYSTEM, &state);
00114     }
00115 
00116     CleanUp();
00117 
00118     if (state == S_END)
00119     {
00120         printf("Program exited succesfully!\n");
00121         return 0;
00122     }
00123     else if (state == S_TERMINATE)
00124     {
00125         printf("Program terminated!\n");
00126         // getchar(); maybe use it, so the terminal is left open till some keypress
00127         return -1;
00128     }
00129 
00130     return 0;
00131 }
```

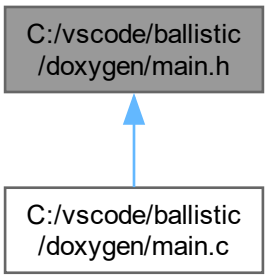
10.36 C:/vscode/ballistic/doxygen/main.h File Reference

Main program declarations.

`#include "error.h"`
Include dependency graph for main.h:



This graph shows which files directly or indirectly include this file:



Functions

- PROCESS Setup ()

  Program initialization and setup.

10.36.1 Detailed Description

Main program declarations.

Definition in file main.h.

10.37 main.h

Go to the documentation of this file.

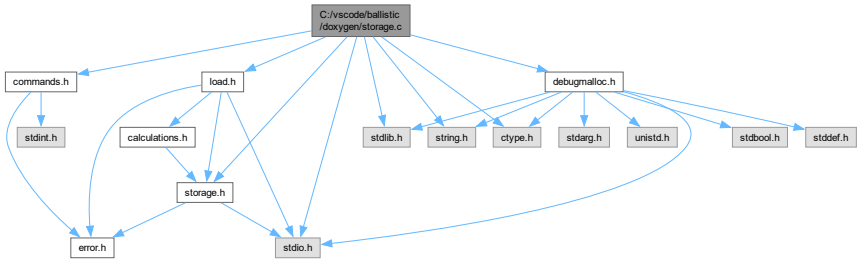
```
00001
00007
00008 #ifndef MAIN_H
00009 #define MAIN_H
00010
00011 #include "error.h"
00012
00026 PROCESS Setup();
00027
00028 #endif /* MAIN_H */
```

10.38 C:/vscode/ballistic/doxygen/storage.c File Reference

```
#include "storage.h"
#include "commands.h"
#include "load.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
```

```
#include "debugmalloc.h"
```

Include dependency graph for storage.c:



Functions

- Ammo_Stock * add_Ammo_Stock (Ammo_Stock_List *list, Ammo_t *ammoRef, unsigned quantity)

  Add ammunition to base stock (sorted by ammo ID).
- Ammo_t_Node * add_Ammo_t (Ammo_t_List *list, Ammo_t ammo)

  Add ammunition type to list (sorted by ID).
- Base_Node * add_Base (Base_List *list, Base base)

  Add base to list (sorted by ID).
- Target_Node * add_Target (Target_List *list, Target target)

  Add target to list (sorted by ID).
- int delete_Ammo_Stock (Ammo_Stock_List *list, Ammo_t *ammoRef)

  Delete ammunition from stock by reference.
- int delete_Ammo_t (Ammo_t_List *list, unsigned id)

  Delete ammunition type by ID.
- int delete_Base (Base_List *list, unsigned id)

  Delete base by ID.
- int delete_Target (Target_List *list, unsigned id)

  Delete target by ID.
- Ammo_t * FindAmmoByID (Ammo_t_List *list, unsigned id)

  Find ammunition by ID.
- Base * FindBaseByID (Base_List *list, unsigned id)

  Find base by ID.
- Target * FindTargetByID (Target_List *list, unsigned id)

  Find target by ID.
- HRESULT free_Ammo_Stock_List (Ammo_Stock_List **list)

  Free all memory associated with ammunition stock list.
- HRESULT free_Ammo_t_List (Ammo_t_List **list)

  Free all memory associated with ammunition list.
- HRESULT free_Base_List (Base_List **list)

  Free all memory associated with base list.
- HRESULT free_Target_List (Target_List **list)

  Free all memory associated with target list.
- unsigned GetAmmoID (const Ammo_t *a)

  Get ammunition ID (safe wrapper).
- Ammo_t * GetNode_Ammot_t_List (Ammo_t_List *list, unsigned id)

  Get ammunition type by ID from ammunition list.

- HRESULT init_Ammo_Stock_List (Ammo_Stock_List **list)
  Initialize an empty ammunition stock list.
- HRESULT init_Ammo_t_List (Ammo_t_List **list)
  Initialize an empty ammunition type list.
- HRESULT init_Base_List (Base_List **list)
  Initialize an empty base list.
- HRESULT init_Target_List (Target_List **list)
  Initialize an empty target list.
- Ammo_t_Node pop_BackAmmo_t (Ammo_t_List *list)
- Ammo_t_Node pop_FrontAmmo_t (Ammo_t_List *list)
- Ammo_Stock popBack_Ammo_Stock (Ammo_Stock_List *list)
  Remove and return last ammunition from stock.
- Base_Node popBack_Base (Base_List *list)
  Remove and return last base from list.
- Target_Node popBack_Target (Target_List *list)
  Remove and return last target from list.
- Ammo_Stock popFront_Ammo_Stock (Ammo_Stock_List *list)
  Remove and return first ammunition from stock.
- Base_Node popFront_Base (Base_List *list)
  Remove and return first base from list.
- Target_Node popFront_Target (Target_List *list)
  Remove and return first target from list.
- void print_Ammo_Stock_List (Ammo_Stock_List *list)
  Print ammunition stock to stdout.
- void print_Ammo_t_List (Ammo_t_List *list)
  Print all ammunition types to stdout.
- void print_Base_List (Base_List *list, PROCESS mode)
  Print all bases to stdout.
- void print_Target_List (Target_List *list)
  Print all targets to stdout.
- HRESULT Save_Ammo_t_List (Ammo_t_List *list, char *path)
  Save ammunition list to file path.
- void Save_Ammo_t_List_file (FILE *fp)
  Save ammunition list to open file.
- HRESULT Save_BaseList (Base_List *list, char *path)
  Save base list to file path.
- void Save_BaseList_file (FILE *fp)
  Save base list to open file.
- HRESULT Save_TargetList (Target_List *list, char *path)
  Save target list to file path.
- void Save_TargetList_file (FILE *fp)
  Save target list to open file.

Variables

- Ammo_t_List * ammo_t_list = NULL
  Global ammunition type database.
- Base_List * base_list = NULL
  All military bases.
- Target_List * target_list = NULL
  All targets.

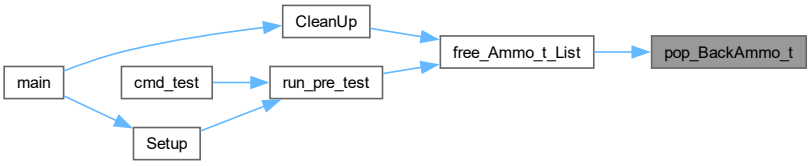
10.38.1 Function Documentation

10.38.1.1 pop_BackAmmo_t()

```
Ammo_t_Node pop_BackAmmo_t (
            Ammo_t_List * list)
```

Definition at line 123 of file storage.c.

Here is the caller graph for this function:



10.38.1.2 pop_FrontAmmo_t()

```
Ammo_t_Node pop_FrontAmmo_t (
            Ammo_t_List * list)
```

Definition at line 108 of file storage.c.

10.39 storage.c

Go to the documentation of this file.

```
00001 #include "storage.h"
00002 #include "commands.h"
00003 #include "load.h"
00004
00005 #include <stdio.h>
00006 #include <stdlib.h>
00007 #include <string.h>
00008 #include <ctype.h>
00009
00010 #include "debugmalloc.h"
00011
00012 Ammo_t_List *ammo_t_list = NULL;
00013 Base_List *base_list = NULL;
00014 Target_List *target_list = NULL;
00015
00016 // ammo list functions=====================================================
00017 HRESULT init_Ammo_t_List(Ammo_t_List **list)
00018 {
00019     if (*list != NULL)
00020         return S_FALSE;
00021     Ammo_t_Node *head = (Ammo_t_Node *)malloc(sizeof(Ammo_t_Node));
00022     Ammo_t_Node *tail = (Ammo_t_Node *)malloc(sizeof(Ammo_t_Node));
00023
00024     head->ammoItem = NULL;
00025     head->prev = NULL;
00026     head->next = tail;
00027     tail->ammoItem = NULL;
00028     tail->next = NULL;
00029     tail->prev = head;
00030
```

```
00031     *list = (Ammo_t_List *)malloc(sizeof(Ammo_t_List));
00032     (*list)->head = head;
00033     (*list)->tail = tail;
00034     return S_OK;
00035 }
00036 // add's an ammo type to a list, in an id order, if there are multiple one's with the same id, the
      places the newest at front
00037 Ammo_t_Node *add_Ammo_t(Ammo_t_List *list, Ammo_t ammo)
00038 {
00039     if (!list)
00040         return NULL;
00041
00042     Ammo_t *new_ammo = (Ammo_t *)malloc(sizeof(Ammo_t));
00043     new_ammo->ballistic_coefficient = ammo.ballistic_coefficient;
00044     new_ammo->blastRadius = ammo.blastRadius;
00045     new_ammo->muzzleVelocity = ammo.muzzleVelocity;
00046     new_ammo->diameter = ammo.diameter;
00047     new_ammo->id = ammo.id;
00048
00049     size_t len = strlen(ammo.name) + 1;
00050     new_ammo->name = (char *)malloc(len * sizeof(char));
00051     if (!new_ammo->name)
00052     {
00053         free(new_ammo);
00054         return NULL;
00055     }
00056     memcpy(new_ammo->name, ammo.name, len);
00057
00058     new_ammo->weight = ammo.weight;
00059
00060     // searches for the non NULL id order, and "moves" the target there
00061     Ammo_t_Node *target = list->head->next;
00062     for (; (target != list->tail); target = target->next)
00063         if ((target->ammoItem && new_ammo))
00064             if (target->ammoItem->id > new_ammo->id)
00065                 break;
00066
00067     // insterts the new item before the target
00068     Ammo_t_Node *new = (Ammo_t_Node *)malloc(sizeof(Ammo_t_Node));
00069
00070     new->ammoItem = new_ammo;
00071     new->next = target;
00072     new->prev = target->prev;
00073     target->prev->next = new;
00074     target->prev = new;
00075
00076     return new;
00077 }
00078
00079 int delete_Ammo_t(Ammo_t_List *list, unsigned id)
00080 {
00081     if (!list)
00082         return -1;
00083
00084     Ammo_t_Node *target = list->head->next;
00085     int no_id = 1;
00086     while (target != list->tail)
00087         if (target->ammoItem && target->ammoItem->id == id)
00088         {
00089             target->next->prev = target->prev;
00090             target->prev->next = target->next;
00091
00092             Ammo_t_Node *tmp = target->next;
00093
00094             free(target->ammoItem->name); // free the string
00095             free(target->ammoItem);       // free the struct
00096             free(target);
00097
00098             target = tmp;
00099
00100             no_id = 0;
00101         }
00102         else
00103             target = target->next;
00104
00105     return (no_id) ? -1 : id;
00106 }
00107
00108 Ammo_t_Node pop_FrontAmmo_t(Ammo_t_List *list)
00109 {
00110     Ammo_t_Node value = {NULL, NULL, NULL};
00111     if (!list)
00112         return value;
00113     Ammo_t_Node *target = list->head->next;
00114     list->head->next = target->next;
00115     target->next->prev = list->head;
00116     value = *target;
```

```
00117     free(target->ammoItem->name); // free the string
00118     free(target->ammoItem);       // free the struct
00119     free(target);                 // free the node
00120     return value;
00121 }
00122
00123 Ammo_t_Node pop_BackAmmo_t(Ammo_t_List *list)
00124 {
00125     Ammo_t_Node value = {NULL, NULL, NULL};
00126     if (!list)
00127         return value;
00128     Ammo_t_Node *target = list->tail->prev;
00129     list->tail->prev = target->prev;
00130     target->prev->next = list->tail;
00131     value = *target;
00132     free(target->ammoItem->name); // free the string
00133     free(target->ammoItem);       // free the struct
00134     free(target);                 // free the node
00135     return value;
00136 }
00137
00138 Ammo_t *GetNode_Ammot_t_List(Ammo_t_List *list, unsigned id)
00139 {
00140     if (!list || (list->head->next == list->tail))
00141         return NULL;
00142
00143     int dir = (abs(list->head->next->ammoItem->id - id) <= abs(list->tail->prev->ammoItem->id - id)) ?
    0 : 1; // 0 up, 1 down
00144
00145     Ammo_t_Node *target = (dir) ? list->tail : list->head;
00146
00147     while (target != list->tail)
00148         if (target->ammoItem && target->ammoItem->id == id)
00149         {
00150
00151             return target->ammoItem;
00152         }
00153         else
00154             target = target->next;
00155
00156     while (target != list->head)
00157         if (target->ammoItem && target->ammoItem->id == id)
00158         {
00159
00160             return target->ammoItem;
00161         }
00162         else
00163             target = target->prev;
00164
00165     return NULL;
00166 }
00167
00168 void print_Ammo_t_List(Ammo_t_List *list)
00169 {
00170     if (!list)
00171         return;
00172
00173     Ammo_t_Node *target = list->head->next;
00174
00175     for (; target != list->tail; target = target->next)
00176         if (target->ammoItem)
00177             printf("id: %5u | label: %20s | weight: %10.2f kg | caliber: %10.2f mm | blast radius
    %10.2f m | muzzle velocity %10.3f m/s | ballistic coefficient %5.3f\n",
00178                    target->ammoItem->id, target->ammoItem->name, target->ammoItem->weight,
    target->ammoItem->diameter, target->ammoItem->blastRadius, target->ammoItem->muzzleVelocity,
    target->ammoItem->ballistic_coefficient);
00179         else
00180             printf("NULL ammo parameter\n");
00181 }
00182 // lifo
00183 HRESULT free_Ammo_t_List(Ammo_t_List **list)
00184 {
00185     if (!*list)
00186         return S_FALSE;
00187     while ((*list)->head->next != (*list)->tail)
00188         pop_BackAmmo_t(*list);
00189
00190     free((*list)->head);
00191     free((*list)->tail);
00192     free(*list);
00193
00194     *list = NULL;
00195
00196     return S_OK;
00197 }
00198
00199 Ammo_t *FindAmmoByID(Ammo_t_List *list, unsigned id)
```

```
00200 {
00201     if (!list || (list->head->next == list->tail))
00202         return NULL;
00203 
00204     int dir = (abs(list->head->next->ammoItem->id - id) <= abs(list->tail->prev->ammoItem->id - id)) ?
      0 : 1; // 0 up, 1 down
00205 
00206     Ammo_t_Node *target = (dir) ? list->tail : list->head;
00207 
00208     while (target != list->tail)
00209         if (target->ammoItem && target->ammoItem->id == id)
00210         {
00211 
00212             return target->ammoItem;
00213         }
00214         else
00215             target = target->next;
00216 
00217     while (target != list->head)
00218         if (target->ammoItem && target->ammoItem->id == id)
00219         {
00220 
00221             return target->ammoItem;
00222         }
00223         else
00224             target = target->prev;
00225 
00226     return NULL;
00227 }
00228 
00229 // base list functions=======================================================
00230 HRESULT init_Base_List(Base_List **list)
00231 {
00232     if (*list != NULL)
00233         return S_FALSE;
00234     Base_Node *head = (Base_Node *)malloc(sizeof(Base_Node));
00235     Base_Node *tail = (Base_Node *)malloc(sizeof(Base_Node));
00236 
00237     head->baseItem = NULL;
00238     head->next = tail;
00239     tail->baseItem = NULL;
00240     tail->next = NULL;
00241 
00242     *list = (Base_List *)malloc(sizeof(Base_List));
00243     (*list)->head = head;
00244     (*list)->tail = tail;
00245     return S_OK;
00246 }
00247 // add's an ammo type to a list, in an id order, if there are multiple one's with the same id, the
      places the newest at front
00248 Base_Node *add_Base(Base_List *list, Base base)
00249 {
00250     if (!list)
00251         return NULL;
00252 
00253     Base *new_base = (Base *)malloc(sizeof(Base));
00254     new_base->id = base.id;
00255     new_base->coord = base.coord;
00256 
00257     size_t len = strlen(base.label) + 1;
00258     new_base->label = (char *)malloc(len * sizeof(char));
00259     if (!new_base->label)
00260     {
00261         free(new_base);
00262         return NULL;
00263     }
00264     memcpy(new_base->label, base.label, len);
00265     new_base->ammoStock = base.ammoStock;
00266 
00267     /*
00268     // searches for the non NULL id order, and "moves" the target there
00269     Base_Node *target = list->head->next;
00270     for (; (target != list->tail); target = target->next)
00271         if ((target->baseItem && new_base))
00272             if (target->baseItem->id > new_base->id)
00273                 break;
00274 
00275     // insterts the new item before the target
00276     Base_Node *new = (Base_Node *)malloc(sizeof(Base_Node));
00277 
00278     new->baseItem = new_base;
00279     new->next = target->next;
00280     target->next = new;
00281     */
00282 
00283     Base_Node *target = list->head;
00284     for (; (target->next != list->tail); target = target->next)
```

```
00285         if ((target->next->baseItem && new_base))
00286             if (target->next->baseItem->id > new_base->id)
00287                 break;
00288
00289     // insterts the new item before the target
00290     Base_Node *new = (Base_Node *)malloc(sizeof(Base_Node));
00291     new->baseItem = new_base;
00292     new->next = target->next;
00293     target->next = new;
00294
00295     return new;
00296 }
00297
00298 int delete_Base(Base_List *list, unsigned id)
00299 {
00300     if (!list)
00301         return -1;
00302
00303     Base_Node *target = list->head->next;
00304     int no_id = 1;
00305     while (target != list->tail)
00306         if (target->next->baseItem && target->next->baseItem->id == id)
00307         {
00308             Base_Node *tmp = target->next;
00309             target->next = target->next->next;
00310
00311             free_Ammo_Stock_List(&(tmp->baseItem->ammoStock));
00312             free(tmp->baseItem->label); // free the string
00313             free(tmp->baseItem);        // free the struct
00314             free(tmp);
00315
00316             no_id = 0;
00317         }
00318         else
00319             target = target->next;
00320
00321     return (no_id) ? -1 : id;
00322 }
00323
00324 Base_Node popFront_Base(Base_List *list)
00325 {
00326     Base_Node value = {NULL, NULL};
00327     if (!list)
00328         return value;
00329     Base_Node *target = list->head->next;
00330     list->head->next = target->next;
00331     value = *target;
00332     free_Ammo_Stock_List(&(target->baseItem->ammoStock));
00333     free(target->baseItem->label); // free the string
00334     free(target->baseItem);        // free the struct
00335     free(target);                  // free the node
00336     return value;
00337 }
00338
00339 Base_Node popBack_Base(Base_List *list)
00340 {
00341     Base_Node value = {NULL, NULL};
00342     if (!list)
00343         return value;
00344     Base_Node *target = list->head->next;
00345     for (; target->next->next != list->tail; target = target->next)
00346         ;
00347     Base_Node *tmp = target->next;
00348     target->next = target->next->next;
00349     value = *tmp;
00350     free_Ammo_Stock_List(&(target->baseItem->ammoStock));
00351     free(tmp->baseItem->label); // free the string
00352     free(tmp->baseItem);        // free the struct
00353     free(tmp);                  // free the node
00354     return value;
00355 }
00356
00357 void print_Base_List(Base_List *list, PROCESS mode)
00358 {
00359     if (!list)
00360         return;
00361
00362     Base_Node *target = list->head->next;
00363
00364     for (; target != list->tail; target = target->next)
00365         if (target->baseItem)
00366         {
00367             printf("id: %5u | label: %20s | latitude: %10.3fdeg | longtitude: %10.3fdeg | altitude:
      %10.3fdeg\n",
00368                    target->baseItem->id, target->baseItem->label, target->baseItem->coord.latitude,
      target->baseItem->coord.longitude, target->baseItem->coord.altitude);
00369             if (mode == S_DEBUG)
```

```
00370                 print_Ammo_Stock_List(target->baseItem->ammoStock);
00371         }
00372         else
00373             printf("NULL ammo parameter\n");
00374 }
00375 // lifo
00376 HRESULT free_Base_List(Base_List **list)
00377 {
00378     if (!*list)
00379         return S_FALSE;
00380     while ((*list)->head->next != (*list)->tail)
00381         popFront_Base(*list);
00382 
00383     free((*list)->head);
00384     free((*list)->tail);
00385     free(*list);
00386 
00387     *list = NULL;
00388 
00389     return S_OK;
00390 }
00391 
00392 Base *FindBaseByID(Base_List *list, unsigned id)
00393 {
00394     if (!list || (list->head->next == list->tail))
00395         return NULL;
00396 
00397     Base_Node *target = list->head;
00398 
00399     while (target != list->tail)
00400         if (target->baseItem && target->baseItem->id == id)
00401         {
00402 
00403             return target->baseItem;
00404         }
00405         else
00406             target = target->next;
00407 
00408     return NULL;
00409 }
00410 
00411 // base ammo stock list functions=======================================================
00412 HRESULT init_Ammo_Stock_List(Ammo_Stock_List **list)
00413 {
00414     if (*list != NULL)
00415         return S_FALSE;
00416 
00417     Ammo_Stock *head = (Ammo_Stock *)malloc(sizeof(Ammo_Stock));
00418     Ammo_Stock *tail = (Ammo_Stock *)malloc(sizeof(Ammo_Stock));
00419 
00420     head->typePtr = NULL;
00421     head->quantity = 0;
00422     head->next = tail;
00423 
00424     tail->typePtr = NULL;
00425     tail->quantity = 0;
00426     tail->next = NULL;
00427 
00428     *list = (Ammo_Stock_List *)malloc(sizeof(Ammo_Stock_List));
00429     (*list)->head = head;
00430     (*list)->tail = tail;
00431 
00432     return S_OK;
00433 }
00434 // Ordered by ammoRef->id, duplicates → newest in front
00435 Ammo_Stock *add_Ammo_Stock(Ammo_Stock_List *list, Ammo_t *ammoRef, unsigned quantity)
00436 {
00437     if (!list || !ammoRef)
00438         return NULL;
00439 
00440     // Remove existing entry to avoid duplicates (your style)
00441     delete_Ammo_Stock(list, ammoRef);
00442 
00443     // Find ordered insertion point
00444     Ammo_Stock *target = list->head;
00445     for (; target->next != list->tail; target = target->next)
00446         if (target->next->typePtr && ammoRef->id < target->next->typePtr->id)
00447             break;
00448 
00449     // Create new node
00450     Ammo_Stock *newNode = (Ammo_Stock *)malloc(sizeof(Ammo_Stock));
00451     newNode->typePtr = ammoRef;
00452     newNode->quantity = quantity;
00453 
00454     // Insert before target->next
00455     newNode->next = target->next;
00456     target->next = newNode;
```

```
00457
00458     return newNode;
00459 }
00460
00461 int delete_Ammo_Stock(Ammo_Stock_List *list, Ammo_t *ammoRef)
00462 {
00463     if (!list || !ammoRef)
00464         return -1;
00465
00466     Ammo_Stock *target = list->head;
00467     int found = 0;
00468
00469     while (target->next != list->tail)
00470     {
00471         if (target->next->typePtr == ammoRef)
00472         {
00473             Ammo_Stock *tmp = target->next;
00474             target->next = target->next->next;
00475             free(tmp);
00476             found = 1;
00477             break; // only one possible
00478         }
00479         target = target->next;
00480     }
00481
00482     return found ? (int)ammoRef->id : -1;
00483 }
00484
00485 Ammo_Stock popFront_Ammo_Stock(Ammo_Stock_List *list)
00486 {
00487     Ammo_Stock value = {NULL, 0, NULL};
00488     if (!list)
00489         return value;
00490
00491     Ammo_Stock *target = list->head->next;
00492
00493     // empty?
00494     if (target == list->tail)
00495         return value;
00496
00497     // unlink
00498     list->head->next = target->next;
00499
00500     // copy the struct value
00501     value = *target;
00502
00503     // free the node itself (NOT the Ammo_t, you don't own it)
00504     free(target);
00505
00506     return value;
00507 }
00508
00509 Ammo_Stock popBack_Ammo_Stock(Ammo_Stock_List *list)
00510 {
00511     Ammo_Stock value = {NULL, 0, NULL};
00512     if (!list)
00513         return value;
00514
00515     Ammo_Stock *cur = list->head;
00516
00517     // empty?
00518     if (cur->next == list->tail)
00519         return value;
00520
00521     // walk until node BEFORE the last real node
00522     while (cur->next->next != list->tail)
00523         cur = cur->next;
00524
00525     Ammo_Stock *tmp = cur->next;
00526
00527     // unlink
00528     cur->next = tmp->next;
00529
00530     // copy out the struct
00531     value = *tmp;
00532
00533     // free only the node wrapper
00534     free(tmp);
00535
00536     return value;
00537 }
00538
00539 void print_Ammo_Stock_List(Ammo_Stock_List *list)
00540 {
00541     if (!list)
00542         return;
00543
```

```
00544     Ammo_Stock *target = list->head->next;
00545
00546     for (; target != list->tail; target = target->next)
00547     {
00548         if (target->typePtr)
00549             printf("\tAmmo ID: %5u | Name: %-20s | Qty: %5u\n",
00550                    target->typePtr->id,
00551                    target->typePtr->name,
00552                    target->quantity);
00553         else
00554             printf("NULL ammo stock entry\n");
00555     }
00556     printf("\n");
00557 }
00558
00559 HRESULT free_Ammo_Stock_List(Ammo_Stock_List **list)
00560 {
00561     if (!*list)
00562         return S_FALSE;
00563
00564     while ((*list)->head->next != (*list)->tail)
00565     {
00566         popFront_Ammo_Stock(*list);
00567     }
00568
00569     free((*list)->head);
00570     free((*list)->tail);
00571     free(*list);
00572
00573
00574     *list = NULL;
00575
00576     return S_OK;
00577 }
00578
00579 // target list functions=============================================
00580 HRESULT init_Target_List(Target_List **list)
00581 {
00582     if (*list != NULL)
00583         return S_FALSE;
00584     Target_Node *head = (Target_Node *)malloc(sizeof(Target_Node));
00585     Target_Node *tail = (Target_Node *)malloc(sizeof(Target_Node));
00586
00587     head->targetItem = NULL;
00588     head->next = tail;
00589     tail->targetItem = NULL;
00590     tail->next = NULL;
00591
00592     *list = (Target_List *)malloc(sizeof(Target_List));
00593     (*list)->head = head;
00594     (*list)->tail = tail;
00595     return S_OK;
00596 }
00597
00598 Target_Node *add_Target(Target_List *list, Target target)
00599 {
00600     if (!list)
00601         return NULL;
00602
00603     Target *new_target = (Target *)malloc(sizeof(Target));
00604     new_target->id = target.id;
00605     new_target->coord = target.coord;
00606
00607     size_t len = strlen(target.label) + 1;
00608     new_target->label = (char *)malloc(len * sizeof(char));
00609     if (!new_target->label)
00610     {
00611         free(new_target);
00612         return NULL;
00613     }
00614     new_target->area = target.area;
00615     memcpy(new_target->label, target.label, len);
00616
00617     Target_Node *target_n = list->head;
00618     for (; (target_n->next != list->tail); target_n = target_n->next)
00619         if ((target_n->next->targetItem && new_target))
00620             if (target_n->next->targetItem->id > new_target->id)
00621                 break;
00622
00623     // insterts the new item before the target
00624     Target_Node *new = (Target_Node *)malloc(sizeof(Target_Node));
00625     new->targetItem = new_target;
00626     new->next = target_n->next;
00627     target_n->next = new;
00628
00629     return new;
00630 }
```

```
00631
00632 int delete_Target(Target_List *list, unsigned id)
00633 {
00634     if (!list)
00635         return -1;
00636
00637     Target_Node *target = list->head->next;
00638     int no_id = 1;
00639     while (target != list->tail)
00640         if (target->next->targetItem && target->next->targetItem->id == id)
00641         {
00642             Target_Node *tmp = target->next;
00643             target->next = target->next->next;
00644
00645             free(tmp->targetItem->label); // free the string
00646             free(tmp->targetItem);        // free the struct
00647             free(tmp);
00648
00649             no_id = 0;
00650         }
00651         else
00652             target = target->next;
00653
00654     return (no_id) ? -1 : id;
00655 }
00656
00657 Target_Node popFront_Target(Target_List *list)
00658 {
00659     Target_Node value = {NULL, NULL};
00660     if (!list)
00661         return value;
00662     Target_Node *target = list->head->next;
00663     list->head->next = target->next;
00664     value = *target;
00665     free(target->targetItem->label); // free the string
00666     free(target->targetItem);        // free the struct
00667     free(target);                    // free the node
00668     return value;
00669 }
00670
00671 Target_Node popBack_Target(Target_List *list)
00672 {
00673     Target_Node value = {NULL, NULL};
00674     if (!list)
00675         return value;
00676     Target_Node *target = list->head->next;
00677     for (; target->next->next != list->tail; target = target->next)
00678         ;
00679     Target_Node *tmp = target->next;
00680     target->next = target->next->next;
00681     value = *tmp;
00682
00683     free(tmp->targetItem->label); // free the string
00684     free(tmp->targetItem);        // free the struct
00685     free(tmp);                    // free the node
00686     return value;
00687 }
00688
00689 void print_Target_List(Target_List *list)
00690 {
00691     if (!list)
00692         return;
00693
00694     Target_Node *target = list->head->next;
00695
00696     for (; target != list->tail; target = target->next)
00697         if (target->targetItem)
00698         {
00699             printf("id: %5u | label: %20s | latitude: %10.3fdeg | longtitude: %10.3fdeg | altitude:
      %10.3fdeg | area: %10.3fkm^2\n",
00700                    target->targetItem->id, target->targetItem->label,
      target->targetItem->coord.latitude, target->targetItem->coord.longitude,
      target->targetItem->coord.altitude, target->targetItem->area);
00701         }
00702         else
00703             printf("NULL ammo parameter\n");
00704 }
00705
00706 HRESULT free_Target_List(Target_List **list)
00707 {
00708     if (!*list)
00709         return S_FALSE;
00710     while ((*list)->head->next != (*list)->tail)
00711         popFront_Target(*list);
00712
00713     free((*list)->head);
00714     free((*list)->tail);
```

```
00715     free(*list);
00716
00717     *list = NULL;
00718
00719     return S_OK;
00720 }
00721
00722 Target *FindTargetByID(Target_List *list, unsigned id)
00723 {
00724     if (!list || (list->head->next == list->tail))
00725         return NULL;
00726
00727     Target_Node *target = list->head;
00728
00729     while (target != list->tail)
00730         if (target->targetItem && target->targetItem->id == id)
00731         {
00732
00733             return target->targetItem;
00734         }
00735         else
00736             target = target->next;
00737
00738     return NULL;
00739 }
00740
00741 HRESULT Save_BaseList(Base_List *list, char *path)
00742 {
00743     HRESULT hr = S_OK;
00744     FILE *f = fopen(path, "w");
00745     if (f)
00746     {
00747         Save_BaseList_file(f);
00748     }
00749     else
00750         hr = E_FAIL;
00751
00752     fclose(f);
00753     return hr;
00754 }
00755
00756 HRESULT Save_TargetList(Target_List *list, char *path)
00757 {
00758     HRESULT hr = S_OK;
00759     FILE *f = fopen(path, "w");
00760     if (f)
00761     {
00762         Save_TargetList_file(f);
00763     }
00764     else
00765         hr = E_FAIL;
00766
00767     fclose(f);
00768     return hr;
00769 }
00770
00771 HRESULT Save_Ammo_t_List(Ammo_t_List *list, char *path)
00772 {
00773     HRESULT hr = S_OK;
00774     FILE *f = fopen(path, "w");
00775     if (f)
00776     {
00777         Save_Ammo_t_List_file(f);
00778     }
00779     else
00780         hr = E_FAIL;
00781
00782     fclose(f);
00783     return hr;
00784 }
00785
00786 unsigned GetAmmoID(const Ammo_t *a)
00787 {
00788     return a ? a->id : 0;
00789 }
00790
00791 void Save_BaseList_file(FILE *fp)
00792 {
00793     if (!fp)
00794     {
00795         fprintf(stderr, "SaveBaseRomFile: NULL file pointer\n");
00796         return;
00797     }
00798
00799     fprintf(fp,
00800             "/*\n"
00801             "=====================================================\n"
```

```
00802             "--------------Base Parameters--------------------\n"
00803             "Use the following structure for inputs:\n"
00804             "    [I]   : id of base\n"
00805             "    [L]   : label (name)\n"
00806             "    [CLA] : latitude (degrees)\n"
00807             "    [CLO] : longitude (degrees)\n"
00808             "    [CA]  : altitude (meters)\n"
00809             "    [AMS] : ammo-stock map { ammo_id:count, ... }\n"
00810             "\n"
00811             "-End every line with ;\n"
00812             "=====================================================\n"
00813             "*/\n\n");
00814
00815     Base_Node *node = base_list->head;
00816
00817     RangeType lastRange = RANGE_NONE;
00818
00819     while (node)
00820     {
00821         Base *b = node->baseItem;
00822         if (b)
00823         {
00824             RangeType r = GetRange(b->id);
00825
00826             if (r != lastRange)
00827             {                           // <-- NEW RANGE DETECTED
00828                 PrintRangeHeader(fp, r); // <-- Print header ONCE
00829                 lastRange = r;
00830             }
00831
00832             fprintf(fp,
00833                     "I%u   L%s   CLA%.6f  CLO%.6f   CA%.1f  AMS:{",
00834                     b->id,
00835                     b->label,
00836                     b->coord.latitude,
00837                     b->coord.longitude,
00838                     b->coord.altitude);
00839
00840             Ammo_Stock *as = b->ammoStock->head;
00841             int first = 1;
00842
00843             while (as)
00844             {
00845                 unsigned ammo_id = GetAmmoID(as->typePtr);
00846                 unsigned qty = as->quantity;
00847
00848                 if (!first)
00849                     fprintf(fp, ",");
00850
00851                 fprintf(fp, "%u:%u", ammo_id, qty);
00852                 first = 0;
00853
00854                 as = as->next;
00855             }
00856
00857             fprintf(fp, "};\n");
00858         }
00859
00860         node = node->next;
00861     }
00862 }
00863
00864 void Save_TargetList_file(FILE *fp)
00865 {
00866     if (!fp)
00867     {
00868         fprintf(stderr, "TargetFile: NULL file pointer\n");
00869         return;
00870     }
00871
00872     fprintf(fp,
00873             "/*\n"
00874             "=====================================================\n"
00875             "--------------Target Parameters--------------------\n"
00876             "Use the following structure for inputs:\n"
00877             "    [I]   : id of base\n"
00878             "    [L]   : label (name)\n"
00879             "    [CLA] : latitude (degrees)\n"
00880             "    [CLO] : longitude (degrees)\n"
00881             "    [CA]  : altitude (meters)\n"
00882             "    [A]   : area (km^2)\n"
00883             "\n"
00884             "-End every line with ;\n"
00885             "=====================================================\n"
00886             "*/\n\n");
00887
00888     Target_Node *node = target_list->head;
```

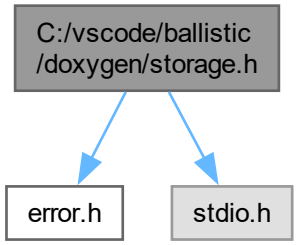
```
00889
00890     RangeType lastRange = RANGE_NONE;
00891
00892     while (node)
00893     {
00894         Target *b = node->targetItem;
00895         if (b)
00896         {
00897             RangeType r = GetRange(b->id);
00898
00899             if (r != lastRange)
00900             {                               // <-- NEW RANGE DETECTED
00901                 PrintRangeHeader(fp, r); // <-- Print header ONCE
00902                 lastRange = r;
00903             }
00904
00905             fprintf(fp,
00906                     "I%u   L%s   CLA%.6f  CLO%.6f   CA%.1f  A%.3f;\n",
00907                     b->id,
00908                     b->label,
00909                     b->coord.latitude,
00910                     b->coord.longitude,
00911                     b->coord.altitude,
00912                     b->area);
00913         }
00914
00915         node = node->next;
00916     }
00917 }
00918
00919 void Save_Ammo_t_List_file(FILE *fp)
00920 {
00921     if (!fp)
00922     {
00923         fprintf(stderr, "Ammo_t_ListFile: NULL file pointer\n");
00924         return;
00925     }
00926
00927     fprintf(fp,
00928             "/*\n"
00929             "=====================================================\n"
00930             "--------------Ammo Parameters-----------------------\n"
00931             "=====================================================\n"
00932             "Use the following structure for the inputs:\n"
00933             "--------The defaults--------------------------------\n"
00934             "\t[I]  : the id of the ammo\n"
00935             "\t[N]  : the name string\n"
00936             "\t[W]  : the weight\n"
00937             "\t[D]  : the diameter (caliber)\n"
00938             "\t[BR] : the blast radius\n"
00939             "\n"
00940             "--------The parameters------------------------------\n"
00941             "(valami még lesz itt)\n"
00942             "=====================================================\n"
00943             "-The data structure has no order\n"
00944             "-The id gets normalized\n"
00945             "-End every line with ;\n"
00946             "=====================================================\n"
00947             "*/\n\n");
00948
00949     Ammo_t_Node *node = ammo_t_list->head;
00950
00951     while (node)
00952     {
00953         Ammo_t *b = node->ammoItem;
00954         if (b)
00955         {
00956
00957             fprintf(fp,
00958                     "I%u   N%s   W%.3f  D%.3f   BR%.3f   MV%.3f   BC%.3f;\n",
00959                     b->id,
00960                     b->name,
00961                     b->weight,
00962                     b->diameter,
00963                     b->blastRadius,
00964                     b->muzzleVelocity,
00965                     b->ballistic_coefficient);
00966         }
00967
00968         node = node->next;
00969     }
00970 }
00971
00972 static void PrintRangeHeader(FILE *fp, RangeType r)
00973 {
00974     switch (r)
00975     {
```

```
00976     case RANGE_EUROPE:
00977         fprintf(fp, "\n// ==================== EUROPE (IDs 0-499) ====================\n");
00978         break;
00979
00980     case RANGE_ASIA:
00981         fprintf(fp, "\n// ==================== ASIA (IDs 500-999) ====================\n");
00982         break;
00983
00984     case RANGE_AMERICAS:
00985         fprintf(fp, "\n// ==================== AMERICAS (IDs 1000-1499) ====================\n");
00986         break;
00987
00988     case RANGE_AFRICA:
00989         fprintf(fp, "\n// ==================== AFRICA (IDs 1500-1999) ====================\n");
00990         break;
00991
00992     case RANGE_OCEANIA:
00993         fprintf(fp, "\n// ==================== AUSTRALIA / OCEANIA (IDs 2000-2499)
      ====================\n");
00994         break;
00995
00996     case RANGE_ANTARCTICA:
00997         fprintf(fp, "\n// ==================== ANTARCTICA (ID 2500) ====================\n");
00998         break;
00999
01000     default:
01001         break;
01002     }
01003 }
```

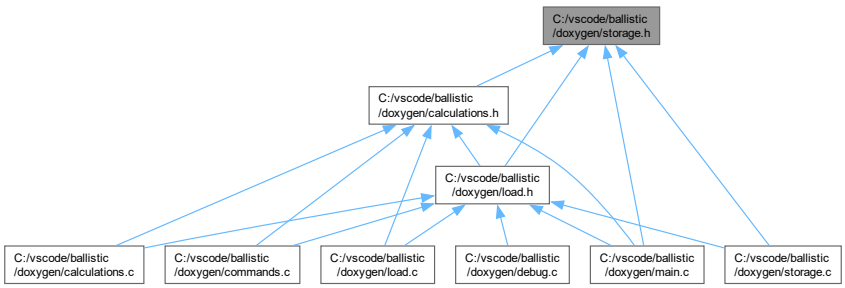
10.40 C:/vscode/ballistic/doxygen/storage.h File Reference

```
#include "error.h"
#include <stdio.h>
```

Include dependency graph for storage.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct Ammo_Stock

  Base ammunition stock entry.
- struct Ammo_Stock_List

  Ammunition stock list for a single base (sorted by ammo ID).
- struct Ammo_t

  Ammunition type definition.
- struct Ammo_t_List

  Global ammunition type list (sorted by ID).
- struct Ammo_t_Node

  Node for global ammunition type linked list.
- struct Base

  Military base structure.
- struct Base_List

  Base list (sorted by ID).
- struct Base_Node

  Node for base linked list.
- struct Coordinate

  Geographic coordinate structure.
- struct Target

  Target structure.
- struct Target_List

  Target list (sorted by ID).
- struct Target_Node

  Node for target linked list.

Typedefs

- typedef struct Ammo_Stock Ammo_Stock

  Base ammunition stock entry.
- typedef struct Ammo_t Ammo_t

  Ammunition type definition.
- typedef struct Ammo_t_Node Ammo_t_Node

  Node for global ammunition type linked list.
- typedef struct Base Base

  Military base structure.
- typedef struct Base_Node Base_Node

  Node for base linked list.
- typedef struct Target Target

  Target structure.
- typedef struct Target_Node Target_Node

  Node for target linked list.

Enumerations

- enum Default_flags {
  F_ID = 0 , F_LABEL = 1 , F_LONGITUDE = 2 , F_LATITUDE = 3 ,
  F_ALTITUDE = 4 }

  Default data field flags for parsing.
- enum InputFlags { T_ID , T_LABEL , T_DATA , T_WHITESPACE }

  Input parsing flags.
- enum RangeType {
  RANGE_NONE = 0 , RANGE_EUROPE , RANGE_ASIA , RANGE_AMERICAS ,
  RANGE_AFRICA , RANGE_OCEANIA , RANGE_ANTARCTICA }

  Regional categorization based on ID ranges.

Functions

Ammunition List Functions

- Ammo_t_Node * add_Ammo_t (Ammo_t_List *list, Ammo_t ammo)

  Add ammunition type to list (sorted by ID).
- int delete_Ammo_t (Ammo_t_List *list, unsigned id)

  Delete ammunition type by ID.
- Ammo_t * FindAmmoByID (Ammo_t_List *list, unsigned id)

  Find ammunition by ID.
- HRESULT free_Ammo_t_List (Ammo_t_List **list)

  Free all memory associated with ammunition list.
- HRESULT init_Ammo_t_List (Ammo_t_List **list)

  Initialize an empty ammunition type list.
- Ammo_t_Node popBack_Ammo_t (Ammo_t_List *list)

  Remove and return last ammunition from list.
- Ammo_t_Node popFront_Ammo_t (Ammo_t_List *list)

  Remove and return first ammunition from list.
- void print_Ammo_t_List (Ammo_t_List *list)

  Print all ammunition types to stdout.

Base List Functions

- Base_Node * add_Base (Base_List *list, Base base)

  Add base to list (sorted by ID).
- int delete_Base (Base_List *list, unsigned id)

  Delete base by ID.
- Base * FindBaseByID (Base_List *list, unsigned id)

  Find base by ID.
- HRESULT free_Base_List (Base_List **list)

  Free all memory associated with base list.
- Ammo_t * GetNode_Ammot_t_List (Ammo_t_List *list, unsigned id)

  Get ammunition type by ID from ammunition list.
- HRESULT init_Base_List (Base_List **list)

  Initialize an empty base list.
- Base_Node popBack_Base (Base_List *list)

  Remove and return last base from list.
- Base_Node popFront_Base (Base_List *list)

  Remove and return first base from list.
- void print_Base_List (Base_List *list, PROCESS mode)

  Print all bases to stdout.

Ammunition Stock List Functions

- Ammo_Stock * add_Ammo_Stock (Ammo_Stock_List *list, Ammo_t *ammoRef, unsigned quantity)
  Add ammunition to base stock (sorted by ammo ID).
- int delete_Ammo_Stock (Ammo_Stock_List *list, Ammo_t *ammoRef)
  Delete ammunition from stock by reference.
- HRESULT free_Ammo_Stock_List (Ammo_Stock_List **list)
  Free all memory associated with ammunition stock list.
- HRESULT init_Ammo_Stock_List (Ammo_Stock_List **list)
  Initialize an empty ammunition stock list.
- Ammo_Stock popBack_Ammo_Stock (Ammo_Stock_List *list)
  Remove and return last ammunition from stock.
- Ammo_Stock popFront_Ammo_Stock (Ammo_Stock_List *list)
  Remove and return first ammunition from stock.
- void print_Ammo_Stock_List (Ammo_Stock_List *list)
  Print ammunition stock to stdout.

Target List Functions

- Target_Node * add_Target (Target_List *list, Target target)
  Add target to list (sorted by ID).
- int delete_Target (Target_List *list, unsigned id)
  Delete target by ID.
- Target * FindTargetByID (Target_List *list, unsigned id)
  Find target by ID.
- HRESULT free_Target_List (Target_List **list)
  Free all memory associated with target list.
- HRESULT init_Target_List (Target_List **list)
  Initialize an empty target list.
- Target_Node popBack_Target (Target_List *list)
  Remove and return last target from list.
- Target_Node popFront_Target (Target_List *list)
  Remove and return first target from list.
- void print_Target_List (Target_List *list)
  Print all targets to stdout.

File I/O Functions

- unsigned GetAmmoID (const Ammo_t *a)
  Get ammunition ID (safe wrapper).
- HRESULT Save_Ammo_t_List (Ammo_t_List *list, char *path)
  Save ammunition list to file path.
- void Save_Ammo_t_List_file (FILE *fp)
  Save ammunition list to open file.
- HRESULT Save_BaseList (Base_List *list, char *path)
  Save base list to file path.
- void Save_BaseList_file (FILE *fp)
  Save base list to open file.
- HRESULT Save_TargetList (Target_List *list, char *path)
  Save target list to file path.
- void Save_TargetList_file (FILE *fp)
  Save target list to open file.

Variables

- Ammo_t_List * ammo_t_list
  Global ammunition type database.
- Base_List * base_list
  All military bases.
- Target_List * target_list
  All targets.

10.41 storage.h

Go to the documentation of this file.

```
00001
00081
00082 #ifndef STORAGE_H
00083 #define STORAGE_H
00084
00085 #include "error.h"
00086 #include <stdio.h>
00087
00094 typedef enum
00095 {
00096     RANGE_NONE = 0,
00097     RANGE_EUROPE,
00098     RANGE_ASIA,
00099     RANGE_AMERICAS,
00100     RANGE_AFRICA,
00101     RANGE_OCEANIA,
00102     RANGE_ANTARCTICA
00103 } RangeType;
00104
00111 static RangeType GetRange(unsigned id)
00112 {
00113     if (id <= 999)
00114         return RANGE_EUROPE;
00115     if (id <= 1999)
00116         return RANGE_ASIA;
00117     if (id <= 2999)
00118         return RANGE_AMERICAS;
00119     if (id <= 3999)
00120         return RANGE_AFRICA;
00121     if (id <= 4999)
00122         return RANGE_OCEANIA;
00123     if (id >= 5000)
00124         return RANGE_ANTARCTICA;
00125
00126     return RANGE_NONE;
00127 }
00128
00133 typedef struct
00134 {
00135     double latitude;
00136     double longitude;
00137     double altitude;
00138 } Coordinate;
00139
00147 typedef struct Ammo_t
00148 {
00149     unsigned id;
00150     char *name;
00151     double weight;
00152     double diameter;
00153     double blastRadius;
00154     double muzzleVelocity;
00155     double ballistic_coefficient;
00156 } Ammo_t;
00157
00162 typedef struct Ammo_t_Node
00163 {
00164     Ammo_t *ammoItem;
00165     struct Ammo_t_Node *prev;
00166     struct Ammo_t_Node *next;
00167 } Ammo_t_Node;
00168
00175 typedef struct
00176 {
00177     Ammo_t_Node *head;
00178     Ammo_t_Node *tail;
00179 } Ammo_t_List;
00180
00187 typedef struct Ammo_Stock
00188 {
00189     Ammo_t *typePtr;
00190     unsigned quantity;
00191     struct Ammo_Stock *next;
00192 } Ammo_Stock;
00193
00198 typedef struct
00199 {
00200     Ammo_Stock *head;
00201     Ammo_Stock *tail;
00202 } Ammo_Stock_List;
00203
00210 typedef struct Base
```

```
00211 {
00212     unsigned id;
00213     char *label;
00214     Coordinate coord;
00215     Ammo_Stock_List *ammoStock;
00216 } Base;
00217
00222 typedef struct Base_Node
00223 {
00224     Base *baseItem;
00225     struct Base_Node *next;
00226 } Base_Node;
00227
00232 typedef struct
00233 {
00234     Base_Node *head;
00235     Base_Node *tail;
00236 } Base_List;
00237
00244 typedef struct Target
00245 {
00246     unsigned id;
00247     char *label;
00248     Coordinate coord;
00249     double area;
00250 } Target;
00251
00256 typedef struct Target_Node
00257 {
00258     Target *targetItem;
00259     struct Target_Node *next;
00260 } Target_Node;
00261
00266 typedef struct
00267 {
00268     Target_Node *head;
00269     Target_Node *tail;
00270 } Target_List;
00271
00272 // Global lists (defined in storage.c)
00273 extern Ammo_t_List *ammo_t_list;
00274 extern Base_List *base_list;
00275 extern Target_List *target_list;
00276
00281
00288 HRESULT init_Ammo_t_List(Ammo_t_List **list);
00289
00297 Ammo_t_Node *add_Ammo_t(Ammo_t_List *list, Ammo_t ammo);
00298
00306 int delete_Ammo_t(Ammo_t_List *list, unsigned id);
00307
00314 Ammo_t_Node popFront_Ammo_t(Ammo_t_List *list);
00315
00322 Ammo_t_Node popBack_Ammo_t(Ammo_t_List *list);
00323
00329 void print_Ammo_t_List(Ammo_t_List *list);
00330
00337 HRESULT free_Ammo_t_List(Ammo_t_List **list);
00338
00346 Ammo_t *FindAmmoByID(Ammo_t_List *list, unsigned id);
00347
00349
00354
00361 HRESULT init_Base_List(Base_List **list);
00362
00370 Base_Node *add_Base(Base_List *list, Base base);
00371
00379 int delete_Base(Base_List *list, unsigned id);
00380
00387 Base_Node popFront_Base(Base_List *list);
00388
00395 Base_Node popBack_Base(Base_List *list);
00396
00403 void print_Base_List(Base_List *list, PROCESS mode);
00404
00412 Ammo_t *GetNode_Ammot_t_List(Ammo_t_List *list, unsigned id);
00413
00420 HRESULT free_Base_List(Base_List **list);
00421
00429 Base *FindBaseByID(Base_List *list, unsigned id);
00430
00432
00437
00444 HRESULT init_Ammo_Stock_List(Ammo_Stock_List **list);
00445
00454 Ammo_Stock *add_Ammo_Stock(Ammo_Stock_List *list, Ammo_t *ammoRef, unsigned quantity);
00455
```

```
00463 int delete_Ammo_Stock(Ammo_Stock_List *list, Ammo_t *ammoRef);
00464
00471 Ammo_Stock popFront_Ammo_Stock(Ammo_Stock_List *list);
00472
00479 Ammo_Stock popBack_Ammo_Stock(Ammo_Stock_List *list);
00480
00486 void print_Ammo_Stock_List(Ammo_Stock_List *list);
00487
00494 HRESULT free_Ammo_Stock_List(Ammo_Stock_List **list);
00495
00497
00502
00509 HRESULT init_Target_List(Target_List **list);
00510
00518 Target_Node *add_Target(Target_List *list, Target target);
00519
00527 int delete_Target(Target_List *list, unsigned id);
00528
00535 Target_Node popFront_Target(Target_List *list);
00536
00543 Target_Node popBack_Target(Target_List *list);
00544
00550 void print_Target_List(Target_List *list);
00551
00558 HRESULT free_Target_List(Target_List **list);
00559
00567 Target *FindTargetByID(Target_List *list, unsigned id);
00568
00570
00575
00582 unsigned GetAmmoID(const Ammo_t *a);
00583
00589 void Save_BaseList_file(FILE *fp);
00590
00598 HRESULT Save_BaseList(Base_List *list, char *path);
00599
00605 void Save_TargetList_file(FILE *fp);
00606
00614 HRESULT Save_TargetList(Target_List *list, char *path);
00615
00621 void Save_Ammo_t_List_file(FILE *fp);
00622
00630 HRESULT Save_Ammo_t_List(Ammo_t_List *list, char *path);
00631
00633
00638 typedef enum
00639 {
00640     T_ID,
00641     T_LABEL,
00642     T_DATA,
00643     T_WHITESPACE
00644 } InputFlags;
00645
00650 typedef enum
00651 {
00652     F_ID = 0,
00653     F_LABEL = 1,
00654     F_LONGITUDE = 2,
00655     F_LATITUDE = 3,
00656     F_ALTITUDE = 4
00657 } Default_flags;
00658
00659 #endif /* STORAGE_H */
```

Index

_USE_MATH_DEFINES
calculations.c, 128

add_Ammo_Stock
Data Storage Structures, 67
add_Ammo_t
Data Storage Structures, 68
add_Base
Data Storage Structures, 69
add_seperator
Command Processing, 24
add_Target
Data Storage Structures, 69
AIR_DENSITY
calculations.c, 128
all_alloc_bytes
DebugmallocData, 118
all_alloc_count
DebugmallocData, 118
alloc_bytes
DebugmallocData, 118
alloc_count
DebugmallocData, 118
altitude
Data Storage Structures, 96
Ammo_Stock, 105
Data Storage Structures, 65
Ammo_Stock_List, 106
Ammo_t, 107
Data Storage Structures, 65
Ammo_t_List, 108
ammo_t_list
Data Storage Structures, 96
Ammo_t_Node, 109
Data Storage Structures, 65
ammoItem
Data Storage Structures, 96
ammoStock
Data Storage Structures, 96
ANSI_COLOR_BLUE
color.h, 140
ANSI_COLOR_CYAN
color.h, 140
ANSI_COLOR_GREEN
color.h, 140
ANSI_COLOR_MAGENTA
color.h, 140
ANSI_COLOR_RED
color.h, 140
ANSI_COLOR_RESET
color.h, 140
ANSI_COLOR_YELLOW
color.h, 140
area
Data Storage Structures, 96
azimuth_deg
BallisticResult, 110

Ballistic Calculations, 15
compute_fire_solution, 16
SuggestBestBaseAndAmmo, 17
SuggestBestBaseAndAmmo_RegionAware, 18
Ballistic Trajectory Calculator, 1
ballistic_coefficient
Data Storage Structures, 96
BallisticResult, 109
azimuth_deg, 110
elevation_deg, 110
impact_velocity, 110
time_of_flight, 110
valid, 111
Base, 111
Data Storage Structures, 65
Base_List, 112
base_list
Data Storage Structures, 96
Base_Node, 114
Data Storage Structures, 65
baseItem
Data Storage Structures, 97
blastRadius
Data Storage Structures, 97
BLOCK_COMMENT
File Loading and Configuration, 42
bool
debug.h, 155

C:/vscode Directory Reference, 104
C:/vscode/ballistic Directory Reference, 103
C:/vscode/ballistic/doxygen Directory Reference, 104
C:/vscode/ballistic/doxygen/calculations.c, 127, 130
C:/vscode/ballistic/doxygen/calculations.h, 135, 136
C:/vscode/ballistic/doxygen/color.c, 136, 138
C:/vscode/ballistic/doxygen/color.h, 139, 142
C:/vscode/ballistic/doxygen/commands.c, 142, 144
C:/vscode/ballistic/doxygen/commands.h, 148, 150
C:/vscode/ballistic/doxygen/data_structures.dox, 151
C:/vscode/ballistic/doxygen/debug.c, 151, 153
C:/vscode/ballistic/doxygen/debug.h, 154, 156
C:/vscode/ballistic/doxygen/debugmalloc.h, 156, 159

C:/vscode/ballistic/doxygen/doxygen_main.h, 166
C:/vscode/ballistic/doxygen/error.c, 166, 167
C:/vscode/ballistic/doxygen/error.h, 167, 168
C:/vscode/ballistic/doxygen/helpers.c, 168, 171
C:/vscode/ballistic/doxygen/helpers.h, 172, 176
C:/vscode/ballistic/doxygen/load.c, 176, 178
C:/vscode/ballistic/doxygen/load.h, 187, 189
C:/vscode/ballistic/doxygen/main.c, 189, 192
C:/vscode/ballistic/doxygen/main.h, 194, 195
C:/vscode/ballistic/doxygen/storage.c, 195, 198
C:/vscode/ballistic/doxygen/storage.h, 210, 214
calculations.c
    _USE_MATH_DEFINES, 128
    AIR_DENSITY, 128
    DEG2RAD, 129
    G, 129
    RAD2DEG, 129
    sq, 129
calloc
    debugmalloc.h, 157
case_mask
    Command, 115
CleanUp
    main.c, 190
clear_terminal
    Command Processing, 24
CM_COLOUR
    color.h, 141
CM_MONO
    color.h, 141
CMD_ADD
    Command Processing, 23
CMD_AMMO
    Command Processing, 23
cmd_ammo
    Command Processing, 25
CMD_BACK
    Command Processing, 23
CMD_BASE
    Command Processing, 23
cmd_base
    Command Processing, 26
CMD_C_ALL
    Command Processing, 22
CMD_C_BACK
    Command Processing, 23
CMD_C_CALC
    Command Processing, 23
CMD_C_CANCEL
    Command Processing, 23
CMD_C_DEFAULT
    Command Processing, 23
CMD_C_EXIT
    Command Processing, 23
CMD_C_GENERIC
    Command Processing, 23
CMD_C_LOAD
    Command Processing, 23
CMD_C_LOOP
    Command Processing, 23
CMD_C_SELECT
    Command Processing, 23
CMD_C_SETUP
    Command Processing, 23
CMD_C_SYSTEM
    Command Processing, 23
CMD_C_YESNO
    Command Processing, 23
CMD_CALCULATE
    Command Processing, 23
cmd_calculate
    Command Processing, 27
CMD_CANCEL
    Command Processing, 23
CMD_CLEAR_TERMINAL
    Command Processing, 24
cmd_clr_ter
    Command Processing, 28
CMD_COLOUR
    Command Processing, 24
cmd_colour
    Command Processing, 28
cmd_config
    Command Processing, 29
CMD_CONTINUE
    Command Processing, 23
CMD_DELETE
    Command Processing, 24
CMD_EXIT
    Command Processing, 23
cmd_exit
    Command Processing, 30
CMD_HELP
    Command Processing, 23
cmd_help
    Command Processing, 30
CMD_INVALID
    Command Processing, 23
CMD_LIST
    Command Processing, 23
CMD_MONO
    Command Processing, 24
cmd_mono
    Command Processing, 31
CMD_NO
    Command Processing, 23
CMD_RESET
    Command Processing, 23
CMD_SAVE
    Command Processing, 23
CMD_SELECT
    Command Processing, 23
CMD_TARGET
    Command Processing, 23
cmd_target
    Command Processing, 32

CMD_TEST
Command Processing, 23
cmd_test
Command Processing, 32
CMD_YES
Command Processing, 23
color.c
get_color_mode, 137
print_colored, 137
set_color_mode, 137
color.h
ANSI_COLOR_BLUE, 140
ANSI_COLOR_CYAN, 140
ANSI_COLOR_GREEN, 140
ANSI_COLOR_MAGENTA, 140
ANSI_COLOR_RED, 140
ANSI_COLOR_RESET, 140
ANSI_COLOR_YELLOW, 140
CM_COLOUR, 141
CM_MONO, 141
ColorMode, 140
get_color_mode, 141
print_colored, 141
set_color_mode, 141
ColorMode
color.h, 140
Command, 115
case_mask, 115
description, 115
handler, 116
name, 116
type, 116
Command Processing, 20
add_seperator, 24
clear_terminal, 24
CMD_ADD, 23
CMD_AMMO, 23
cmd_ammo, 25
CMD_BACK, 23
CMD_BASE, 23
cmd_base, 26
CMD_C_ALL, 22
CMD_C_BACK, 23
CMD_C_CALC, 23
CMD_C_CANCEL, 23
CMD_C_DEFAULT, 23
CMD_C_EXIT, 23
CMD_C_GENERIC, 23
CMD_C_LOAD, 23
CMD_C_LOOP, 23
CMD_C_SELECT, 23
CMD_C_SETUP, 23
CMD_C_SYSTEM, 23
CMD_C_YESNO, 23
CMD_CALCULATE, 23
cmd_calculate, 27
CMD_CANCEL, 23
CMD_CLEAR_TERMINAL, 24
cmd_clr_ter, 28
CMD_COLOUR, 24
cmd_colour, 28
cmd_config, 29
CMD_CONTINUE, 23
CMD_DELETE, 24
CMD_EXIT, 23
cmd_exit, 30
CMD_HELP, 23
cmd_help, 30
CMD_INVALID, 23
CMD_LIST, 23
CMD_MONO, 24
cmd_mono, 31
CMD_NO, 23
CMD_RESET, 23
CMD_SAVE, 23
CMD_SELECT, 23
CMD_TARGET, 23
cmd_target, 32
CMD_TEST, 23
cmd_test, 32
CMD_YES, 23
Command_case, 22
command_count, 37
Command_t, 23
commands, 37
GetCommand, 33
NormalizeInput, 34
ParseCommandFromBuffer, 36
Command_case
Command Processing, 22
command_count
Command Processing, 37
Command_t
Command Processing, 23
commands
Command Processing, 37
compute_fire_solution
Ballistic Calculations, 16
coord
Data Storage Structures, 97
Coordinate, 116

Data Storage Structures, 58
add_Ammo_Stock, 67
add_Ammo_t, 68
add_Base, 69
add_Target, 69
altitude, 96
Ammo_Stock, 65
Ammo_t, 65
ammo_t_list, 96
Ammo_t_Node, 65
ammoItem, 96
ammoStock, 96
area, 96
ballistic_coefficient, 96
Base, 65

base_list, 96
Base_Node, 65
baseItem, 97
blastRadius, 97
coord, 97
Default_flags, 66
delete_Ammo_Stock, 70
delete_Ammo_t, 71
delete_Base, 71
delete_Target, 73
diameter, 97
F_ALTITUDE, 66
F_ID, 66
F_LABEL, 66
F_LATITUDE, 66
F_LONGITUDE, 66
FindAmmoByID, 74
FindBaseByID, 74
FindTargetByID, 76
free_Ammo_Stock_List, 76
free_Ammo_t_List, 77
free_Base_List, 78
free_Target_List, 79
GetAmmoID, 80
GetNode_Ammot_t_List, 80
head, 97, 98
id, 98
init_Ammo_Stock_List, 81
init_Ammo_t_List, 82
init_Base_List, 82
init_Target_List, 84
InputFlags, 66
label, 99
latitude, 99
longitude, 99
muzzleVelocity, 99
name, 99
next, 100
popBack_Ammo_Stock, 84
popBack_Ammo_t, 85
popBack_Base, 85
popBack_Target, 86
popFront_Ammo_Stock, 86
popFront_Ammo_t, 87
popFront_Base, 87
popFront_Target, 88
prev, 100
print_Ammo_Stock_List, 89
print_Ammo_t_List, 89
print_Base_List, 90
print_Target_List, 91
quantity, 100
RANGE_AFRICA, 67
RANGE_AMERICAS, 67
RANGE_ANTARCTICA, 67
RANGE_ASIA, 67
RANGE_EUROPE, 67
RANGE_NONE, 67
RANGE_OCEANIA, 67
RangeType, 67
Save_Ammo_t_List, 91
Save_Ammo_t_List_file, 92
Save_BaseList, 93
Save_BaseList_file, 93
Save_TargetList, 94
Save_TargetList_file, 95
T_DATA, 67
T_ID, 67
T_LABEL, 67
T_WHITESPACE, 67
tail, 101
Target, 65
target_list, 101
Target_Node, 66
targetItem, 101
typePtr, 102
weight, 102
debug.c
run_pre_test, 152
debug.h
bool, 155
EMPTY_MESSAGE, 154
false, 155
run_pre_test, 155
true, 155
debugmalloc.h
calloc, 157
debugmalloc_canary_char, 159
debugmalloc_canary_size, 159
debugmalloc_max_block_size_default, 159
debugmalloc_tablesize, 159
DebugmallocData, 159
DebugmallocEntry, 159
free, 157
malloc, 158
putenv, 159
realloc, 158
strdup, 158
strndup, 158
debugmalloc_canary_char
debugmalloc.h, 159
debugmalloc_canary_size
debugmalloc.h, 159
debugmalloc_max_block_size_default
debugmalloc.h, 159
debugmalloc_tablesize
debugmalloc.h, 159
DebugmallocData, 117
all_alloc_bytes, 118
all_alloc_count, 118
alloc_bytes, 118
alloc_count, 118
debugmalloc.h, 159
head, 118
logfile, 118
max_block_size, 118

tail, 118
DebugmallocEntry, 119
debugmalloc.h, 159
expr, 119
file, 119
func, 120
line, 120
next, 120
prev, 120
real_mem, 120
size, 120
user_mem, 120
Default_flags
Data Storage Structures, 66
DEG2RAD
calculations.c, 129
delete_Ammo_Stock
Data Storage Structures, 70
delete_Ammo_t
Data Storage Structures, 71
delete_Base
Data Storage Structures, 71
delete_Target
Data Storage Structures, 73
description
Command, 115
diameter
Data Storage Structures, 97

E_FAIL
Utility Functions, 39
elevation_deg
BallisticResult, 110
EMPTY_MESSAGE
debug.h, 154
ensure_trailing_backslash
helpers.c, 169
helpers.h, 174
expr
DebugmallocEntry, 119

F_ALTITUDE
Data Storage Structures, 66
F_ID
Data Storage Structures, 66
F_LABEL
Data Storage Structures, 66
F_LATITUDE
Data Storage Structures, 66
F_LONGITUDE
Data Storage Structures, 66
FAILED
Utility Functions, 39
false
debug.h, 155
file
DebugmallocEntry, 119
File Loading and Configuration, 40
BLOCK_COMMENT, 42
folder_path, 56
format_basic_error_string, 42
free_folder_path, 43
generate_config, 43
handle_ammo_t, 44
handle_base, 45
handle_config, 46
handle_target, 48
HandleParameterSelection, 48
LINE_COMMENT, 42
LogResult, 49
NORMAL, 42
ParseState, 42
peek, 50
read_config, 51
RECORD, 42
RecordHandler, 42
recreate_config, 52
scan, 53
set_folder_path, 55
test_and_run_config, 55
FindAmmoByID
Data Storage Structures, 74
FindBaseByID
Data Storage Structures, 74
FindTargetByID
Data Storage Structures, 76
folder_path
File Loading and Configuration, 56
format_basic_error_string
File Loading and Configuration, 42
free
debugmalloc.h, 157
free_Ammo_Stock_List
Data Storage Structures, 76
free_Ammo_t_List
Data Storage Structures, 77
free_Base_List
Data Storage Structures, 78
free_folder_path
File Loading and Configuration, 43
free_Target_List
Data Storage Structures, 79
func
DebugmallocEntry, 120

G
calculations.c, 129
generate_config
File Loading and Configuration, 43
get_color_mode
color.c, 137
color.h, 141
GetAmmoID
Data Storage Structures, 80
GetCommand
Command Processing, 33
GetNode_Ammot_t_List
Data Storage Structures, 80

handle_ammo_t
File Loading and Configuration, 44
handle_base
File Loading and Configuration, 45
handle_config
File Loading and Configuration, 46
handle_target
File Loading and Configuration, 48
HandleParameterSelection
File Loading and Configuration, 48
handler
Command, 116
head
Data Storage Structures, 97, 98
DebugmallocData, 118
helpers.c
ensure_trailing_backslash, 169
norm_strncmp, 169
Normalize_path, 169
remove_quotes, 170
strtoken, 170
helpers.h
ensure_trailing_backslash, 174
norm_strncmp, 174
Normalize_path, 174
remove_quotes, 175
strtoken, 175
HRESULT
Utility Functions, 38

id
Data Storage Structures, 98
impact_velocity
BallisticResult, 110
init_Ammo_Stock_List
Data Storage Structures, 81
init_Ammo_t_List
Data Storage Structures, 82
init_Base_List
Data Storage Structures, 82
init_Target_List
Data Storage Structures, 84
InputFlags
Data Storage Structures, 66

label
Data Storage Structures, 99
latitude
Data Storage Structures, 99
line
DebugmallocEntry, 120
LINE_COMMENT
File Loading and Configuration, 42
logfile
DebugmallocData, 118
LogResult
File Loading and Configuration, 49
longitude
Data Storage Structures, 99

main
main.c, 191
Main Program, 57
Setup, 57
main.c
CleanUp, 190
main, 191
malloc
debugmalloc.h, 158
max_block_size
DebugmallocData, 118
muzzleVelocity
Data Storage Structures, 99

name
Command, 116
Data Storage Structures, 99
next
Data Storage Structures, 100
DebugmallocEntry, 120
norm_strncmp
helpers.c, 169
helpers.h, 174
NORMAL
File Loading and Configuration, 42
Normalize_path
helpers.c, 169
helpers.h, 174
NormalizeInput
Command Processing, 34

ParseCommandFromBuffer
Command Processing, 36
ParseState
File Loading and Configuration, 42
peek
File Loading and Configuration, 50
pop_BackAmmo_t
storage.c, 198
pop_FrontAmmo_t
storage.c, 198
popBack_Ammo_Stock
Data Storage Structures, 84
popBack_Ammo_t
Data Storage Structures, 85
popBack_Base
Data Storage Structures, 85
popBack_Target
Data Storage Structures, 86
popFront_Ammo_Stock
Data Storage Structures, 86
popFront_Ammo_t
Data Storage Structures, 87
popFront_Base
Data Storage Structures, 87
popFront_Target
Data Storage Structures, 88
prev
Data Storage Structures, 100

DebugmallocEntry, 120
print_Ammo_Stock_List
Data Storage Structures, 89
print_Ammo_t_List
Data Storage Structures, 89
print_Base_List
Data Storage Structures, 90
print_colored
color.c, 137
color.h, 141
print_Target_List
Data Storage Structures, 91
PROCESS
Utility Functions, 39
putenv
debugmalloc.h, 159

quantity
Data Storage Structures, 100

RAD2DEG
calculations.c, 129
RANGE_AFRICA
Data Storage Structures, 67
RANGE_AMERICAS
Data Storage Structures, 67
RANGE_ANTARCTICA
Data Storage Structures, 67
RANGE_ASIA
Data Storage Structures, 67
RANGE_EUROPE
Data Storage Structures, 67
RANGE_NONE
Data Storage Structures, 67
RANGE_OCEANIA
Data Storage Structures, 67
RangeType
Data Storage Structures, 67
read_config
File Loading and Configuration, 51
real_mem
DebugmallocEntry, 120
realloc
debugmalloc.h, 158
RECORD
File Loading and Configuration, 42
RecordHandler
File Loading and Configuration, 42
recreate_config
File Loading and Configuration, 52
remove_quotes
helpers.c, 170
helpers.h, 175
run_pre_test
debug.c, 152
debug.h, 155

S_DEBUG
Utility Functions, 39
S_END
Utility Functions, 39
S_FALSE
Utility Functions, 38
S_OK
Utility Functions, 38
S_PARTIAL
Utility Functions, 38
S_RUNNING
Utility Functions, 39
S_TERMINATE
Utility Functions, 39
Save_Ammo_t_List
Data Storage Structures, 91
Save_Ammo_t_List_file
Data Storage Structures, 92
Save_BaseList
Data Storage Structures, 93
Save_BaseList_file
Data Storage Structures, 93
Save_TargetList
Data Storage Structures, 94
Save_TargetList_file
Data Storage Structures, 95
scan
File Loading and Configuration, 53
set_color_mode
color.c, 137
color.h, 141
set_folder_path
File Loading and Configuration, 55
Setup
Main Program, 57
size
DebugmallocEntry, 120
sq
calculations.c, 129
State, 121
vx, 121
vy, 121
vz, 121
x, 122
y, 122
z, 122
storage.c
pop_BackAmmo_t, 198
pop_FrontAmmo_t, 198
strdup
debugmalloc.h, 158
strndup
debugmalloc.h, 158
strtoken
helpers.c, 170
helpers.h, 175
SUCCEEDED
Utility Functions, 39
SuggestBestBaseAndAmmo
Ballistic Calculations, 17

SuggestBestBaseAndAmmo_RegionAware
Ballistic Calculations, 18

T_DATA
Data Storage Structures, 67
T_ID
Data Storage Structures, 67
T_LABEL
Data Storage Structures, 67
T_WHITESPACE
Data Storage Structures, 67
tail
Data Storage Structures, 101
DebugmallocData, 118
Target, 122
Data Storage Structures, 65
Target_List, 123
target_list
Data Storage Structures, 101
Target_Node, 124
Data Storage Structures, 66
targetItem
Data Storage Structures, 101
test_and_run_config
File Loading and Configuration, 55
time_of_flight
BallisticResult, 110
Todo List, 5
true
debug.h, 155
type
Command, 116
typePtr
Data Storage Structures, 102

user_mem
DebugmallocEntry, 120
Utility Functions, 38
E_FAIL, 39
FAILED, 39
HRESULT, 38
PROCESS, 39
S_DEBUG, 39
S_END, 39
S_FALSE, 38
S_OK, 38
S_PARTIAL, 38
S_RUNNING, 39
S_TERMINATE, 39
SUCCEEDED, 39

valid
BallisticResult, 111
Vec3, 125
x, 125
y, 125
z, 125
vx
State, 121
vy
State, 121
vz
State, 121

weight
Data Storage Structures, 102

x
State, 122
Vec3, 125

y
State, 122
Vec3, 125

z
State, 122
Vec3, 125